

Practical hands-on experience learning how to approach the Business Understanding and the Analytic Approach stage tasks and the Data Requirements and Collection stage tasks for any data science problem.

## Module 1: Python Basics

This module teaches the basics of Python and begins by exploring some of the different data types such as integers, real numbers, and strings. Continue with the module and learn how to use expressions in mathematical operations, store values in variables, and the many different ways to manipulate strings.

### Learning Objectives

- Demonstrate an understanding of types in Python by converting or casting data types such as strings, floats, and integers.
- Interpret variables and solve expressions by applying mathematical operations.
- Describe how to manipulate strings by using a variety of methods and operations.
- Build a program in JupyterLab to demonstrate your knowledge of types, expressions, and variables.
- Work with, manipulate, and perform operations on strings in Python.

## About the course

### Python for Data Science & AI

- Super easy to learn
- One of the first languages to start with when learning to code
- Very powerful
- Huge ecosystem of libraries that help you get complex things done with just a few lines of code

Python is great for a huge range of tasks:



Data Analysis



Web Scraping



Big Data



Finance



Computer Vision



Natural Language  
Processing



Machine Learning



Deep Learning

This course is designed for beginners, but if you know how to program, you can also take this course to quickly learn Python.

#### Course Overview

Welcome to the Python for Data Science, AI, and Development course. After completing this course, you'll possess the **basic knowledge of Python** and acquire a good understanding of different **data types**. You'll also learn to use lists and tuples, dictionaries,

and Python sets. Additionally, you'll acquire the concepts of condition and branching and will know how to implement loops, create functions, perform exception handling, and create objects. Furthermore, you'll be proficient in reading and writing files and will be able to implement unique ways to collect data using APIs and web scraping. In addition to the module labs, you'll prove your skills in a peer-graded project and your overall knowledge with the final quiz.

## **Course Content**

This course is divided into five modules. You should set a goal to complete at least one module per week.

### **Module 1: Python Basics**

- About the Course
- Types
- Expressions and Variables
- String Operations

### **Module 2: Python Data Structures**

- Lists and Tuples
- Dictionaries
- Sets

### **Module 3: Python Programming Fundamentals**

- Conditions and Branching
- Loops
- Functions
- Exception Handling
- Objects and Classes
- Practice with Python Programming Fundamentals

### **Module 4: Working with Data in Python**

- Reading and Writing Files with Open
- Pandas
- Numpy in Python

## Module 5: APIs and Data Collection

- Simple APIs
- REST APIs, Web Scraping, and Working with Files
- Final Exam

The course contains a variety of learning assets: Videos, activities, labs, projects, practice, graded quizzes, and readings. The videos and readings present the instruction. Labs and activities support that instruction with hands-on learning experiences. Discussions allow you to interact and learn from your peers. **A peer-review project** that mimics real-world scenarios encourage you to showcase your skills, Practice quizzes enable you to test your knowledge of what you learned. Finally, graded quizzes indicate how well you have learned the course concepts



## HELPFUL TIPS *for* COURSE COMPLETION

**1. FAMILIARIZE YOURSELF WITH THE COURSE CONTENT**

- Browse module overviews and objectives
- Understand topics and associated assets
- Familiarize with content order
- Identify upcoming topics
- Connect ideas to create a completion plan for the course

**2. FORM YOUR PLAN AND MAKE A ROUGH TIMELINE FOR COURSE COMPLETION**

- Review completion time estimates for module assets
- Set reasonable time goal for each module
- Determine course completion deadline
- Schedule daily study time

**3. ACTIVELY MANAGE YOUR LEARNING**

**Complete your independent tasks:**

- Take notes during the course
- Download transcripts for reference
- Highlight important parts in transcripts
- Complete all labs
- Review terms using glossaries

**Get support:**

- Actively participate in discussion forums

**Pass your quizzes:**

- Review study notes
- Complete practice quizzes, Review feedback
- Complete graded quizzes
- Use related videos or readings for incorrect answers
- Review video/transcript/reading for correct answers
- Retake quizzes until passing

**4. TALK WITH YOUR FRIENDS AND FAMILY ABOUT THE COURSE**

- Stay accountable, commit to the course
- Talk to friends and family about it
- Engage in conversations about interesting topics
- Seek beneficial perspectives from others

**5. FOLLOW YOUR PLAN**

- Stay motivated with your plan
- Set achievable goals in step 2
- Reward yourself upon achieving the goals

**IBM** Skills Network

## Getting Starting with Python and Jupyter

It is the most widely used and most popular programming language used in the data science industry.

According to the 2019 Kaggle Data Science and Machine Learning Survey,  $\frac{3}{4}$  of the over 10,000 respondents worldwide reported that they use Python regularly.

Glassdoor reported that in 2019 more than 75% of data science positions listed included Python in their job descriptions.

When asked which language an aspiring data scientist should learn first, most data scientists say Python.

## Who is python for?

For programmers: it uses **clear and readable syntax**. You can develop the same programs from other languages with **lesser code** using Python.

For beginners, Python is a good language to start with because of the **huge global community and wealth of documentation**.

Several different surveys done in 2019 established that over 80% of data professionals use Python worldwide.

Python is useful in many areas including data science, AI and machine learning, web development, and Internet of Things (IoT) devices, like the Raspberry Pi.

Large organizations that heavily use python include

IBM, Wikipedia, Google, Yahoo!, CERN, NASA, Facebook, Amazon, Instagram, Spotify, and Reddit.

Python is widely supported by a global community and shepherded by the **Python Software Foundation**.

## What makes python great

Python is a **high-level, general-purpose programming language** that can be applied to many different classes of problems.

It has a large, **standard library** that provides tools suited to many different tasks including but not limited to Databases, Automation, Web scraping, Text processing, Image processing, Machine learning, and Data analytics.

For **data science**, you can use Python's scientific computing libraries like Pandas, NumPy, SciPy, and Matplotlib.

For **artificial intelligence**, it has TensorFlow, PyTorch, Keras, and Scikit-learn.

For **Natural Language Processing** (NLP) using the Natural Language Toolkit (NLTK).

## Diversity and inclusion efforts

Another great selling point for Python is that the Python community has a well-documented history of paving the way for diversity and inclusion efforts in the tech industry as a whole.

The Python language has a code of conduct executed by the Python Software Foundation that seeks to ensure safety and inclusion for all, in both online and in-person Python communities.

Communities like PyLadies seek to create spaces for people interested in learning Python in safe and inclusive environments.

PyLadies is an international mentorship group with a focus on helping more women become active participants and leaders in the Python open-source community.

## Introduction to Jupyter

Jupyter is a freely available **web application** that enables creation and sharing of documents containing equations, live coding, visualizations, and narrative text. Jupyter provides an interactive computing environment that supports multiple programming languages, including Python, R, Julia, and more, but it shines brightest when used with Python. Jupyter revolves around **notebooks, documents containing a mix** of code, visualizations, narrative text, equations, and multimedia content. These notebooks allow users to **create, share, and collaborate on computational projects seamlessly**.

## Why Jupyter?

Jupyter's popularity stems from its **flexibility** and ease of use. Regardless of your level of programming expertise, whether you're an experienced coder or embarking on your data science journey, Jupyter offers an **intuitive platform for writing, testing, and sharing code**. Its interactive interface enables data exploration, algorithm experimentation, and result visualization—all seamlessly integrated within a unified environment.

## Key Features of Jupyter

Here are some key features and advantages of Jupyter:

1. **Interactive Computing:** Jupyter notebooks enable users to write and execute code interactively. This means you can run code cells individually and see the output immediately, fostering an iterative approach to coding and experimentation.
2. **Support for Multiple Languages:** While Jupyter was initially developed for Python (hence the name, which stands for Julia, Python, and R), it now supports various programming languages **through its kernel system**. This flexibility makes Jupyter suitable for various computational tasks and interdisciplinary collaboration.
3. **Rich Output:** Jupyter Notebooks support rich media integration, allowing users to generate interactive plots, charts, images, videos, and more directly within the document. This makes visualizing data, communicating findings, and creating compelling narratives easier.
4. **Integration with Data Science Libraries:** Jupyter seamlessly integrates with popular libraries and frameworks used in the data science ecosystem, such as NumPy, Pandas, Matplotlib, sci-kit-learn, TensorFlow, and PyTorch. This allows users to leverage the full power of these tools within the notebook environment for tasks like data manipulation, visualization, machine learning, and deep learning.

5. **Collaboration and Sharing:** Jupyter promotes collaboration and reproducibility by allowing users to share their notebooks with others via email, GitHub, or the Jupyter Notebook Viewer. This facilitates knowledge sharing, peer review, and interdisciplinary collaboration, as users can easily exchange ideas, code snippets, and best practices.

## Jupyter in Data Science

Jupyter has become an indispensable **tool for researchers, analysts, and developers in data science**. Its seamless integration with popular libraries such as NumPy, pandas, and sci-kit-learn makes it the go-to choice for data manipulation, analysis, and machine learning. Jupyter provides a user-friendly interface, interactive capabilities, and robust collaboration features, making it an essential tool for anyone involved in data analysis, scientific research, education, or software development. Whether you're exploring data, building machine learning models, teaching a class, or conducting research, Jupyter empowers you to work more efficiently and share your insights with others.

## Getting Started with Jupyter

Now that you can glimpse what Jupyter offers, it's time to dive in and experience its capabilities firsthand. Our Getting Started with Jupyter video will walk you through the basics of setting up and using Jupyter, empowering you to unleash the full potential of Python and embark on your data science journey with confidence.

So, let's jump into the world of Jupyter and unlock a world of possibilities in Python and data science!

## Basic jupyter operations

Launch notebook

Rename notebook

Run selected cells

Insert cells

Delete cells

Work with multiple (side by side)

Presenting notebook

Markdown cells

Slide functionality

Shutdown notebook

## Hands-on Lab

*View saved notebook*

## Types

A type is how Python represents different types of data. The **expression and the data type**.  
Type command.

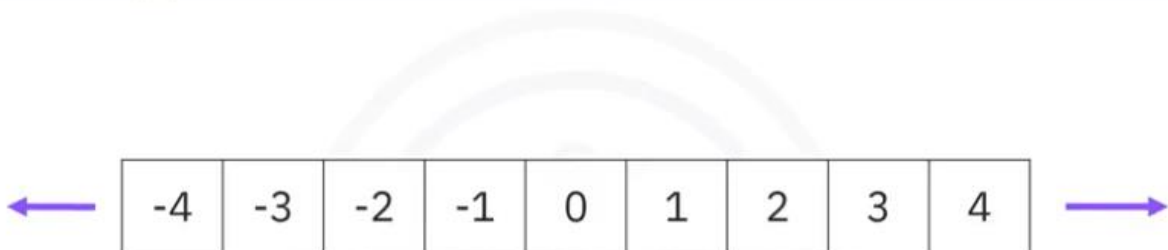
|                                       |       |
|---------------------------------------|-------|
| <code>type(11)</code>                 | int   |
| <code>type(21.213)</code>             | float |
| <code>type("Hello Python 101")</code> | str   |

We can have int, which stands for an integer and float that stands for float, essentially a real number.

The type string is a sequence of characters.

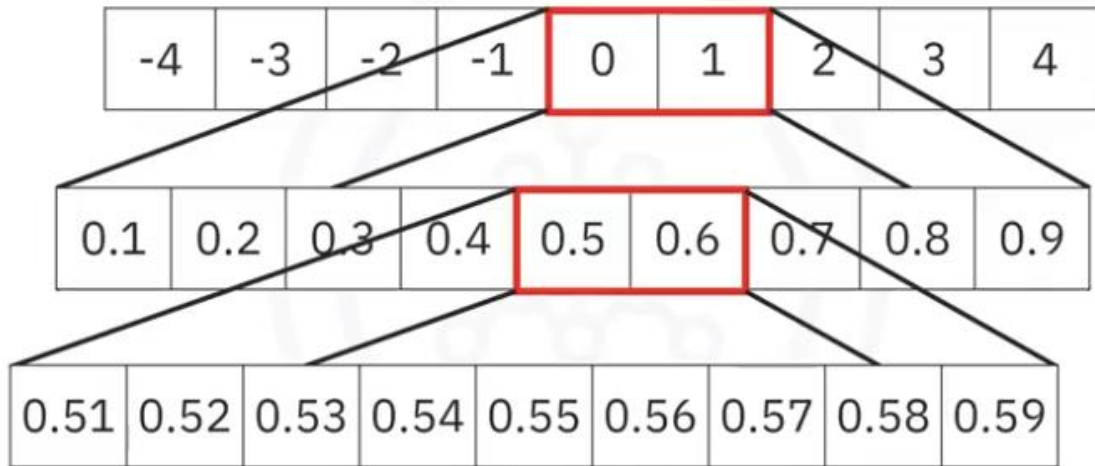
[Tip:] Note that strings can be represented with single quotes ('1.2') or double quotes ("1.2"), but you can't mix both (e.g., "1.2').

## Data types: int



Integers can be **negative or positive**. There is a finite quite large range of integers.

# Data types: float



Floats are real numbers. Include the integers but also numbers in between the integers. We can zoom in. There is a quite small limit

**sys.float\_info**(max=1.7976931348623157e+308, max\_exp=1024, max\_10\_exp=308, min=2.2250738585072014e-308, min\_exp=-1021, min\_10\_exp=-307, dig=15, mant\_dig=53, epsilon=2.220446049250313e-16, radix=2, rounds=1)

typecasting

## Changing expression type

`float(2):2.0`

`int(1.1):1`

`int('1'):1`

~~`int('A')`~~

Changes the type of the expression in Python.

You can convert an int (2) to a float (2.0).

Casting a float (1.1) to an integer (1), You lose some information.

If a string contains an integer value, you can convert it to int.

If we convert a string that contains a non-integer value, we get an error.

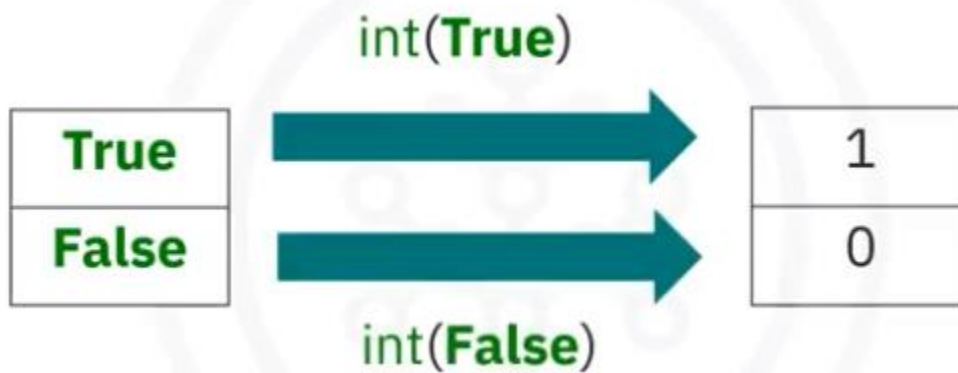
ValueError: invalid literal for int() with base 10: 'A'

```
str(1): "1"
```

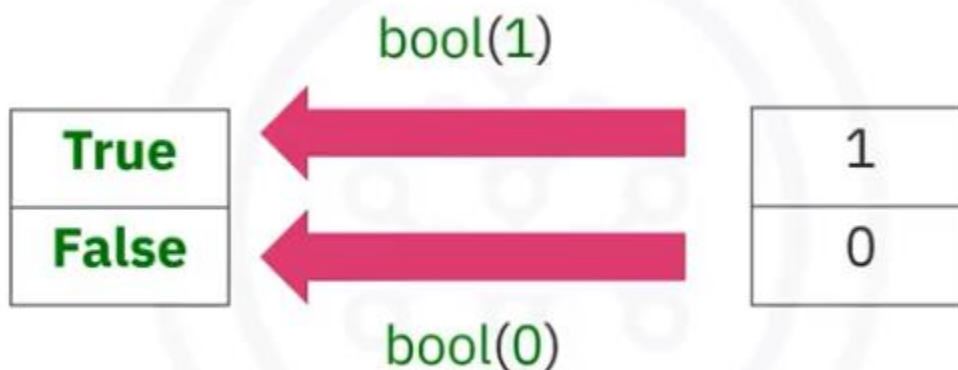
```
str(4.5): '4.5'
```

You can convert an int or a float to a string.

Boolean can take on two values: True and False. Type command on a Boolean value, we obtain the term bool.



Cast True to an integer or float get 1. Cast False to an integer or float, get 0.



Cast 1 to a Boolean get True. Cast 0 to a Boolean get False.

## Expressions and variables

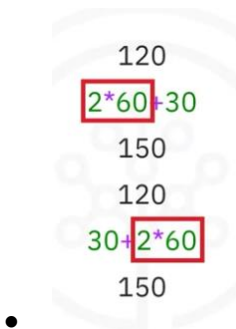
In this video, we will cover expressions and variables.

## Expressions

Describe a type of operation (addition (+), subtraction (-), multiplication (\*), division (/), integer division (//)) that computers perform.

We call the numbers **operands**, and the math symbols in this case, addition, are called **operators**.

Python follows mathematical conventions (BODMAS) when performing mathematical expressions. The following operations are in a different order. Then addition, to obtain the final result. The expressions in the parentheses are performed first.



## BODMAS

- **Brackets (Parentheses):** Operations within parentheses are evaluated first.
- **Orders (Exponents):** Powers and roots are evaluated next.
- **Division and Multiplication:** These operations are performed from left to right.
- **Addition and Subtraction:** These operations are performed from left to right.

## Variables.

To store values usable somewhere else in the code using the **assignment operator (=)**.

We can:

- store the results of expressions.
- perform operations on x and save the result to a new variable-y.
- perform operations on x and assign the value x.
- use the type command in variables as well.

It's good practice to **use meaningful variable names**. Use the underscore or a capital letter to represent the start of a new word.



## String Operations

A string is a sequence of characters (spaces, digits special characters), contained within two quotes or single quotes we can bind to a variable

- A string can be spaces or digits

`"1 2 3 4 5 6"`

- A String can also be special characters :

``@#2_#]&*^%$'`

A string is an ordered sequence. Each element accessible with an index represented by the array of numbers.

Name= "Michael Jackson"

|   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |
|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|
| M | i | c | h | a | e | l |   | J | a | c  | k  | s  | o  | n  |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |

Name[0]:M

Name[6]:l

Name[13]:o

Name= "Michael Jackson"

|     |     |     |     |     |     |    |    |    |    |    |    |    |    |    |
|-----|-----|-----|-----|-----|-----|----|----|----|----|----|----|----|----|----|
| M   | i   | c   | h   | a   | e   | l  |    | J  | a  | c  | k  | s  | o  | n  |
| -15 | -14 | -13 | -12 | -11 | -10 | -9 | -8 | -7 | -6 | -5 | -4 | -3 | -2 | -1 |

Name[-15]:M

Name[-1]:n

String Slicing

Name= "Michael Jackson"

|   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |
|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|
| M | i | c | h | a | e | l |   | J | a | c  | k  | s  | o  | n  |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |

Name[0:4] =Mich

Name[8:12] =Jack

Name= "Michael Jackson"

|   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |
|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|
| M | i | c | h | a | e | l |   | J | a | c  | k  | s  | o  | n  |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |

Name[::2]: "McalJcsn"    Name[0:5:2]: "Mca"

We can use the **len** command to obtain the length of the string.

We can **concatenate** (+) or combine strings.

We can **replicate** (\*) values of a string.

3\* "Michael Jackson "



"Michael Jackson Michael Jackson Michael Jackson "

Strings are immutable.

This means you cannot change the value of the string, but you can create a new string.

## Back slashes (Escape Sequences)

Strings that may be difficult to input. **new line (\n) or tab (\t).**

To place a backslash in your string, use a double backslash, or place an "r" in front of the string.

## String Methods and sequence methods

Strings as sequences have **methods that work on lists and tuples**. Strings also have a second set of methods that just work on strings.

### Method



When we apply a method to the string A, we get a new string B that is different from A.



The method `find`, finds substrings. The output is the first index of the sequence if found , -1 otherwise.

## Additional Reading

### Format Strings in Python

Format strings are a way to **inject variables into a string** in Python. They are used to format strings and produce more human-readable outputs. There are several ways to format strings in Python:

#### String interpolation (f-strings)

Introduced in Python 3.6, f-strings are a new way to format strings in Python. They are prefixed with **'f'** and use curly braces `{}` to enclose the variables that will be formatted. For example:

1. `name = "John"`
2. `age = 30`
3. `print(f"My name is {name} and I am {age} years old.")`

This will output:

1. My name is John and I am 30 years old.

#### `str.format()`

This is another way to format strings in Python. It uses curly braces `{}` as placeholders for variables which are passed as arguments in the **`format()` method**. For example:

1. `name = "John"`
2. `age = 50`
3. `print("My name is {} and I am {} years old.".format(name, age))`

This will output:

1. My name is John and I am 50 years old.

## % Operator

This is one of **the oldest ways to format strings** in Python. It uses the % operator to replace variables in the string. For example:

1. `name = "Johnathan"`
2. `age = 30`
3. `print("My name is %s and I am %d years old." % (name, age))`

This will output:

1. My name is Johnathan and I am 30 years old.

**“My name is %s and I am %d years old.”**: This is a string that includes format specifiers:

- **%s**: This is a placeholder for a string.
- **%d**: This is a placeholder for an integer.

**% (name, age)**: This is a tuple containing the variables name and age. The values of these variables will replace the placeholders in the string.

Each of these methods has its own advantages and use cases. However, f-strings are generally considered the most modern and preferred way to format strings in Python due to their readability and performance.

## Additional capabilities

F-strings are also able to evaluate expressions inside the curly braces, which can be very handy. For example:

1. `x = 10`
2. `y = 20`
3. `print(f"The sum of x and y is {x+y}.")`

This will output:

1. The sum of x and y is 30.

## Raw String (r’)

In Python, raw strings are a powerful tool for **handling textual data, especially when dealing with escape characters**. By prefixing a string literal with the letter ‘r’, Python

treats the string as raw, meaning it interprets backslashes as literal characters rather than escape sequences.

Consider the following examples of regular string and raw string:

#### **Regular string:**

1. `regular_string = "C:\new_folder\file.txt"`
2. `print("Regular String:", regular_string)`

This will output:

1. Regular String: C:
2. ew\_folderile.txt

In the regular string `regular_string` variable, the backslashes (`\n`) are interpreted as escape sequences. Therefore, `\n` represents a newline character, which would lead to an incorrect file path representation.

#### **Raw string:**

1. `raw_string = r"C:\new_folder\file.txt"`
2. `print("Raw String:", raw_string)`

This will output:

1. Raw String: C:\new\_folder\file.txt

However, in the raw string `raw_string`, the backslashes are treated as literal characters. This means that `\n` is not interpreted as a newline character, but rather as two separate characters, “`\`” and “`n`”. Consequently, the file path is represented exactly as it appears.

[Summary. Cheatsheet, Grades Quiz, Glossary](#)

## Module 2: Python Data Structures

This module begins a journey into Python data structures by explaining the use of lists and tuples and how they are able to store collections of data in a single variable. Next learn about dictionaries and how they function by storing data in pairs...

### Learning Objectives

- Describe and manipulate tuple combinations and list data structures.
- Execute basic tuple operations in Python.
- Perform list operations in Python.
- Write structures with correct keys and values to demonstrate understanding of dictionaries.
- Work with and perform operations on dictionaries in Python.
- Create sets to demonstrate understanding of the differences between sets, tuples, and lists.
- Work with sets in Python, including operations and logic operations.

### List and tuples

They are **Compound data types** in Python.

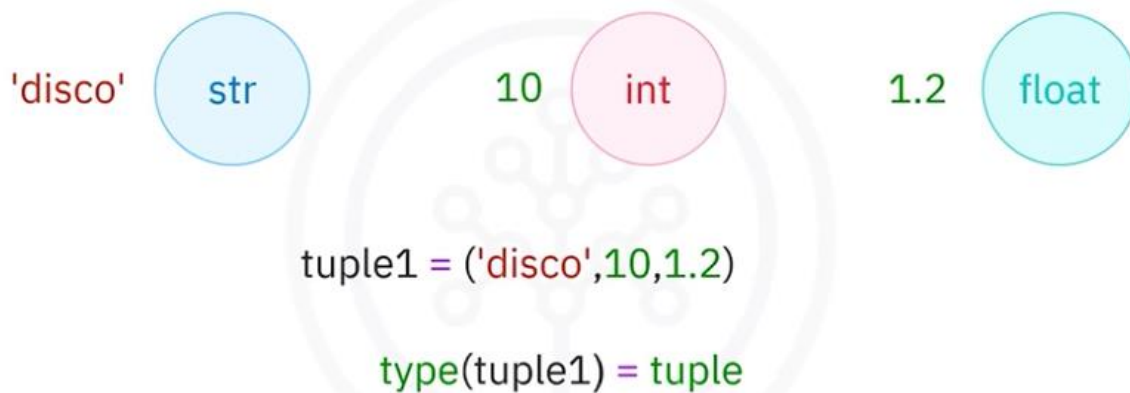
#### Tuples.

Tuples are an ordered sequence. Tuples are expressed as comma-separated elements within **parentheses**.



# Tuples

---



Different types can be contained in a tuple, but the **type** of the variable is **tuple**.

Each element of a tuple can be accessed via an **index(positive or negative)**.

# Tuples

---

`tuple1 = ("disco", 10, 1.2)`

|   |         |
|---|---------|
| 0 | "disco" |
| 1 | 10      |
| 2 | 1.2     |

`tuple1[0]: "disco"`

`tuple1[1]: 10`

`tuple1[2]: 1.2`

The following table represents the relationship between the index and the elements in the tuple. We can concatenate or combine tuples by adding them.

The result is the following with the following index.

If we would like multiple elements from a tuple, we can also **slice tuples**.

# Tuples: Slicing

("disco", 10, 1.2, "hard rock", 10)

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

tuple2[3:5] => ("hard rock", 10)

We can use the len command to obtain the **length** of a tuple.

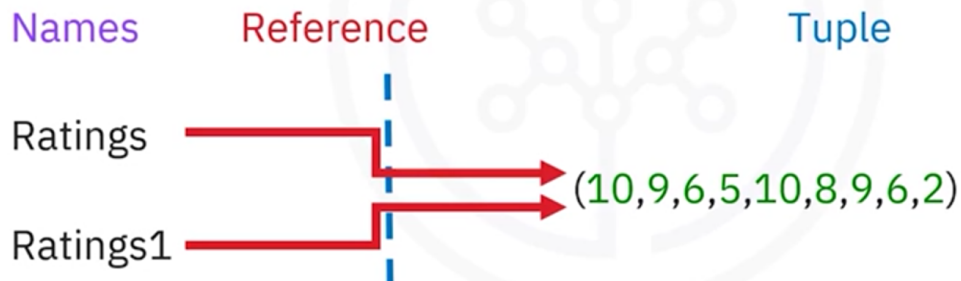
## *Immutability*

Tuples are **immutable**, which means we can't change them.

# Tuples: Immutable

Ratings = (10, 9, 6, 5, 10, 8, 9, 6, 2)

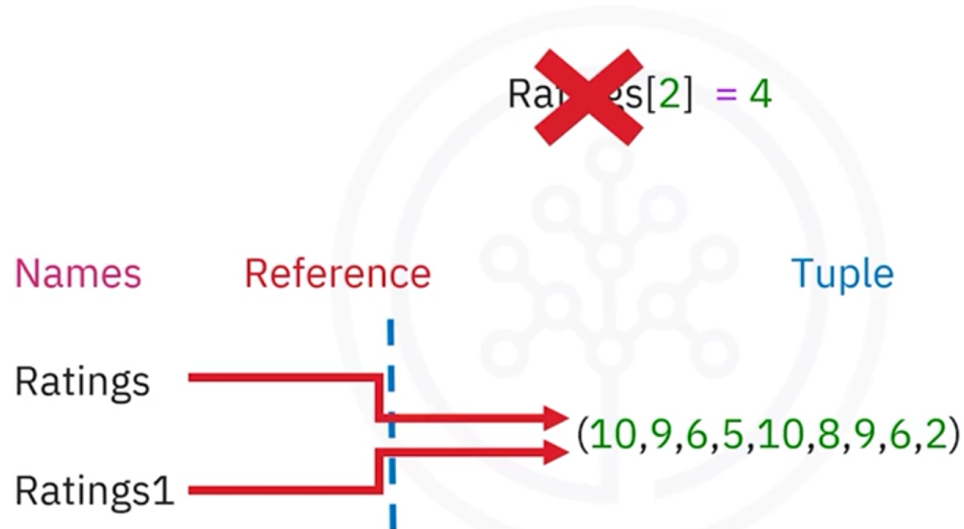
Ratings1 = Ratings



when we set the variable ratings1 to ratings. Each variable does not contain a tuple but references the same immutable tuple object.

## Tuples: Immutable

---

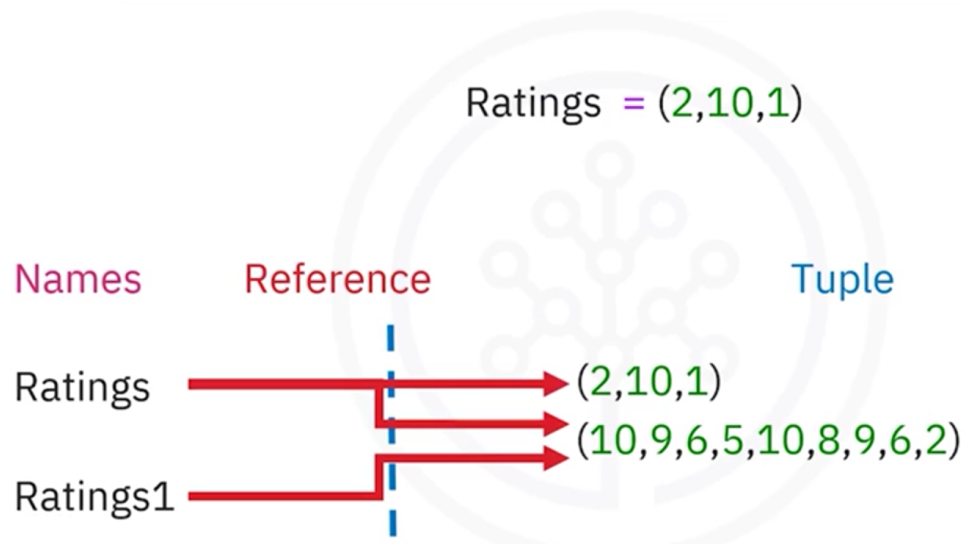


If we want to change the element at index two. Because tuples are immutable, we can't. Therefore, ratings1 will not be affected by a change in rating because the tuple is immutable, i.e. we can't change it.

We can assign a different tuple to the ratings variable. The variable ratings now references another tuple. As a consequence of immutability, if we would like to manipulate a tuple, we must create a new tuple instead.

## Tuples: Immutable

---



For example, if we would like to sort a tuple, we use the function sorted. The input is the original tuple. The output is a new sorted list.

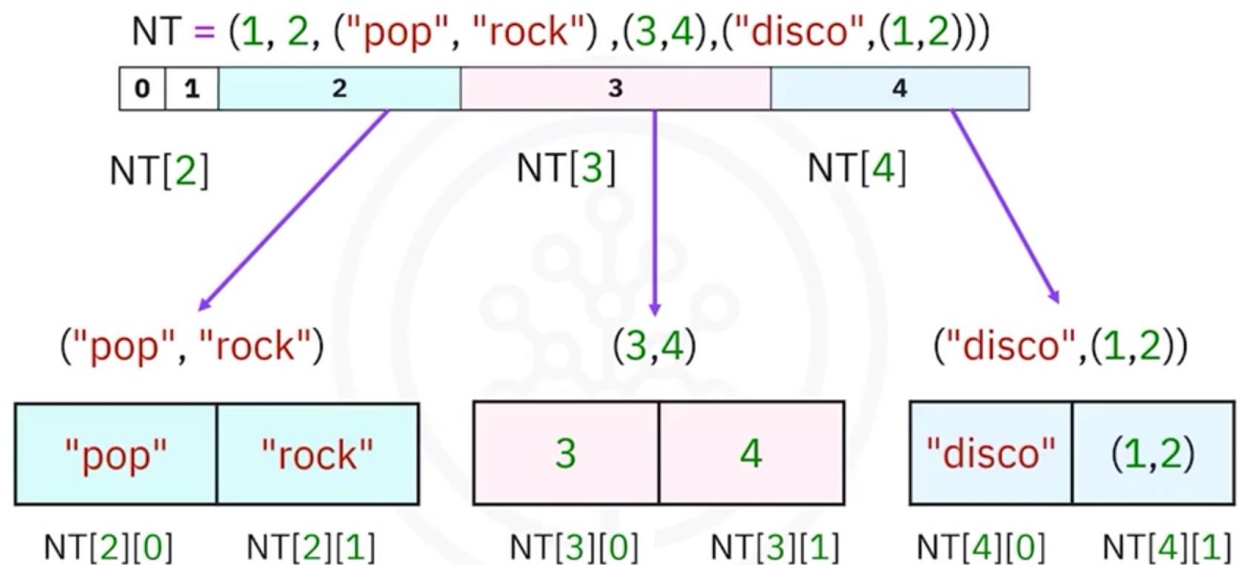
# Tuples: Immutable

Ratings = (10,9,6,5,10,8,9,6,2)

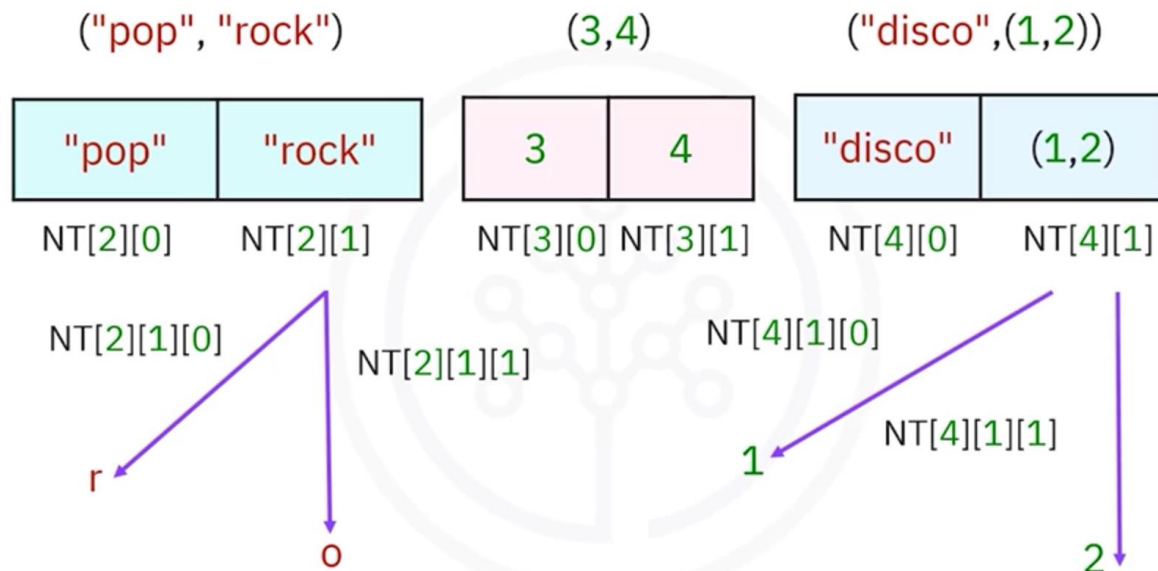
RatingsSorted = sorted(Ratings)

(2, 5, 6, 6, 8, 9, 9, 10, 10)

## Nesting



A tuple can contain other tuples as well as other complex data types. This is called nesting. We can access these elements using the standard indexing methods. If we select an index with a tuple, the same index convention applies. As such, we can then access values in the tuple. For example, we could access the second element. It is helpful to visualize this as a tree.



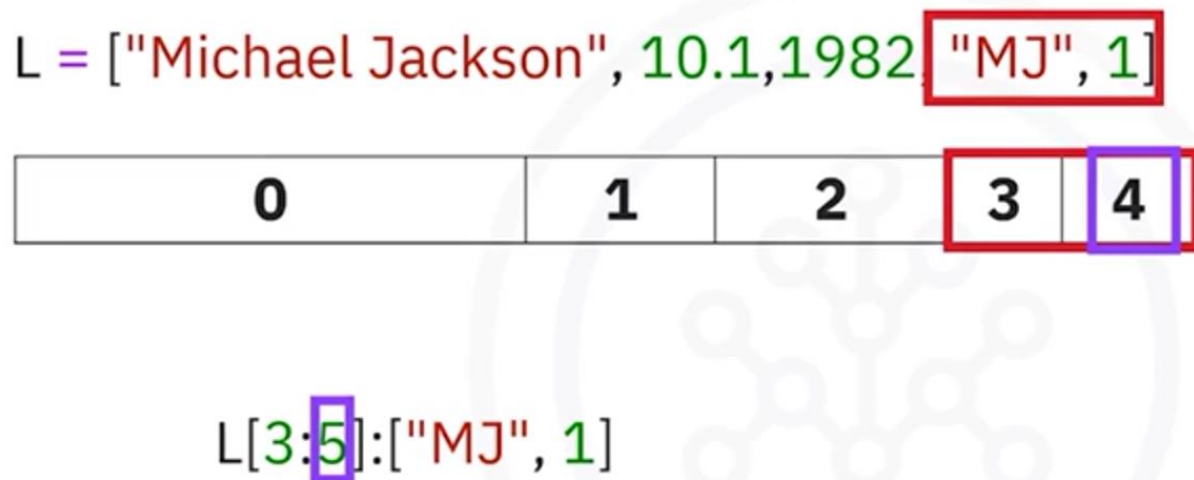
## List

Lists are also an ordered sequence, represented with **square brackets**.

Lists are like tuples but mutable. Can contain different types, either nest other lists or tuples. Have The same indexing conventions apply for nesting. Like tuples, each element of a list can be accessed via an index.

We can also perform slicing in lists.

## Lists: Slicing



Notice how the last index is one larger than the length of the list

### *Concatenate*

We can **concatenate** or combine lists by adding them.

## Lists

---

```
L = ["Michael Jackson", 10.1, 1982]
```

```
L1 = L + ["pop", 10]
```

```
L1 = ["Michael Jackson", 10.1, 1982, "pop", 10]
```

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

### *Extend method*

Lists are mutable, therefore, we can change them.

# Lists

---

```
L = ["Michael Jackson", 10.1, 1982]
```

```
L.extend(["pop", 10])
```

```
L = ["Michael Jackson", 10.1, 1982, "pop", 10]
```

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

For example, we apply the method `extend` by adding a dot followed by the name of the method, and then parentheses. The argument inside the parentheses is a **new list that we are going to concatenate to the original list**. In this case, instead of creating a new list, `L1`, the original list, `L`, is modified by adding two new elements. To learn more about methods, check out our video on objects and classes.

# Lists

---

```
L = ["Michael Jackson", 10.1, 1982]
```

```
L.append(["pop", 10])
```

```
L = ["Michael Jackson", 10.1, 1982, ["pop", 10]]
```

|   |   |   |   |
|---|---|---|---|
| 0 | 1 | 2 | 3 |
|---|---|---|---|

If we apply append instead of extended, we **add one element to the list**. If we look at the index, there is only one more element. Index 3 contains the list we appended.

Every time we apply a method, the list changes. If we apply extend, we add two new elements to the list. The list L is modified by adding two new elements. If we append the string A, we further change the list adding the string A.

As lists are mutable, we can change them.

For example, we can change the first element as follows.

The list now becomes HardRock 10 1.2.

## Delete

We can delete an element of a list using the del command.



# Lists

---

A = ["hard rock", 10, 1.2]



del(A[0])



A: [10, 1.2]

We simply indicate the list item we would like to remove as an argument. For example, if we would like to remove the first element, the result becomes 10 1.2. We can delete the second element. This operation removes the second element off the list.

*Split*

## Lists

---

"A,B,C,D".split(",")

["A", "B", "C", "D"]

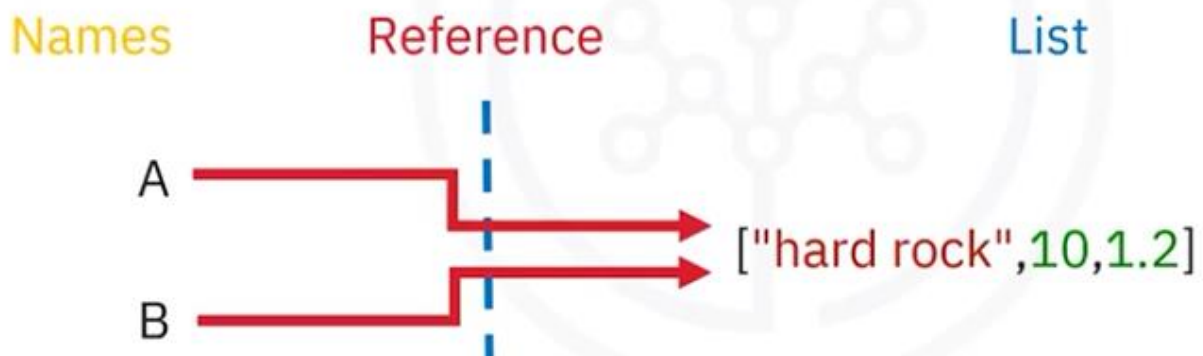
We can use the split function to separate strings on a specific character, known as a delimiter. We simply pass the delimiter we would like to split on as an argument, in this case a comma. The result is a list, each element corresponds to a set of characters that have been separated by a comma.

## Lists: Aliasing

---

```
A = ["hard rock",10,1.2]
```

```
B = A
```



When we set one variable, b, equal to a, both a and b are referencing the same list. Multiple names referring to the same object is known as aliasing. We know from the list slide that the first element in b is set as HardRock.

# Lists: Aliasing

B[0] = "hard rock"

A[0] = " banana "



B[0]: " banana "

Names

Reference

List



If we change the first element in a to banana, we get a **side effect**. The value of b will change as a consequence. a and b are referencing the same list, therefore if we change a, list b also changes. If we check the first element of b after changing list a, we get banana instead of HardRock.

## Cloning

You can clone list a by using the following syntax.

# Lists: Clone

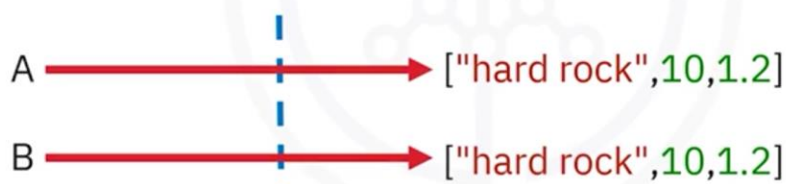
A = ["hard rock", 10, 1.2]

B = A[:]

Names

Reference

List



Variable a references one list. Variable b references a new copy or clone of the original list. Now if you change a, b will not change.

We can get more info on list, tuples and many other objects in Python using the **help(A)** command. Simply pass in the list, tuple or any other Python object.

Help on list object:

```
class list(object)
  list(iterable=(), /)

  Built-in mutable sequence.

  If no argument is given, the constructor creates a new empty list.
  The argument must be an iterable if specified.

  Methods defined here:

  __add__(self, value, /)
      Return self+value.

  __contains__(self, key, /)
      Return bool(key in self).

  __delitem__(self, key, /)
      Delete self[key].

  __eq__(self, value, /)
      Return self==value.

  __ge__(self, value, /)
      Return self>=value.

  __getattr__(self, name, /)
      Return getattr(self, name).

  __getitem__(self, index, /)
      Return self[index].

  __gt__(self, value, /)
      Return self>value.
```

## Dictionaris

Dictionaries are a type of collection in Python.

A dictionary has **keys** and **values**. The key is analogous to the index. They are like addresses, but they don't have to be integers. They are usually characters. The values are similar to the element in a list and contain information.

# Dictionaries

- Dictionaries are denoted with curly brackets {}.
- The keys have to be immutable and unique.
- The values can be immutable, mutable, and duplicates.
- Each key and value pair is separated by a comma.

```
{"key1":1, "key2": "2", "key3": [3,3,3], "key4": (4,4,4), ('key5'):5}
```

Consider the following example of a dictionary.

|                             |        |                   |
|-----------------------------|--------|-------------------|
| "Thriller"                  | "1982" | dict["Thriller"]: |
| "Back in Black"             | "1980" |                   |
| "The Dark Side of the Moon" | "1973" |                   |
| "The Bodyguard"             | "1992" |                   |
| "Bat Out of Hell"           | "1977" |                   |
| "Their Greatest..."         | "1976" |                   |
| "Saturday Night Fever"      | "1977" |                   |
| "Rumors"                    | "1977" |                   |

The album title is the key, and the value is the released data.

**Read:** We use square brackets. The argument is the key. This outputs the value. Dict["Back in Black"] : "1980"

**Add** a new entry dict["Graduation"] = "2007"

**Delete** an entry del(dict["Thriller"])

**Exist** an entry? "Key" in dict returns True if "key" is in the dictionary, False otherwise.,

Get the keys dict.keys() returns a list with all keys.

Get the values dict.values() returns a list with all values

## Sets

Sets are a type of collection holding different Python types. They are unordered, do not record element position. Sets only have **unique elements**.

This means there is only one of a particular element in a set.

To define a set, you use **curly brackets**.

```
Set1={"pop", "rock", "soul", "hard rock", "rock", "R&B", "rock", "disco" }
```

```
Set1: {"rock", "R&B", "disco", "hard rock", "pop", "soul"}
```

The duplicated elements you place within the curly brackets, when the actual set is created, will not be present.

### Set from list

```
album_list = ["Michael Jackson", "Thriller", "Thriller", 1982]
```

```
album_set = set(album_list)
```

```
album_set : {'Michael Jackson', 'Thriller', 1982}
```

set()

album\_set

### set operations.

**Add** : album\_list.add("new\_element")

**Remove** : album\_list.remove("one\_element")

**Exist** : "one\_element" in A

*mathematical set operations.*

Intersection & and `set1.intersection(set2)`

```
album_set_1 = {"AC/DC", "Back in Black", "Thriller"}
```

```
album_set_2 = {"AC/DC", "Back in Black", "The Dark Side of the Moon"}
```

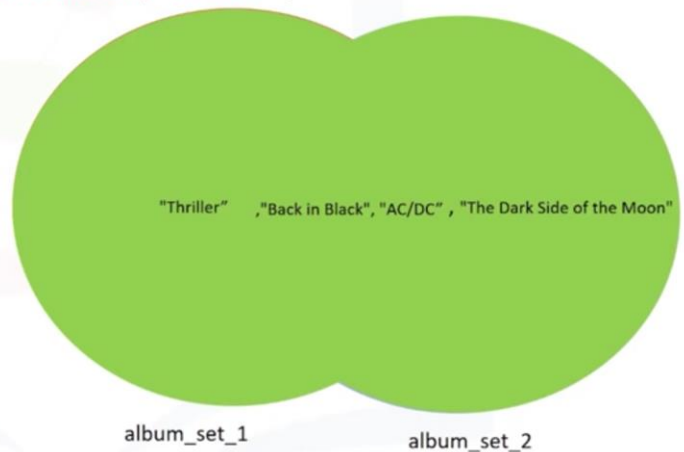
```
album_set_3 = album_set_1 & album_set_2
```

```
album_set_3: {"AC/DC", "Back in Black"}
```

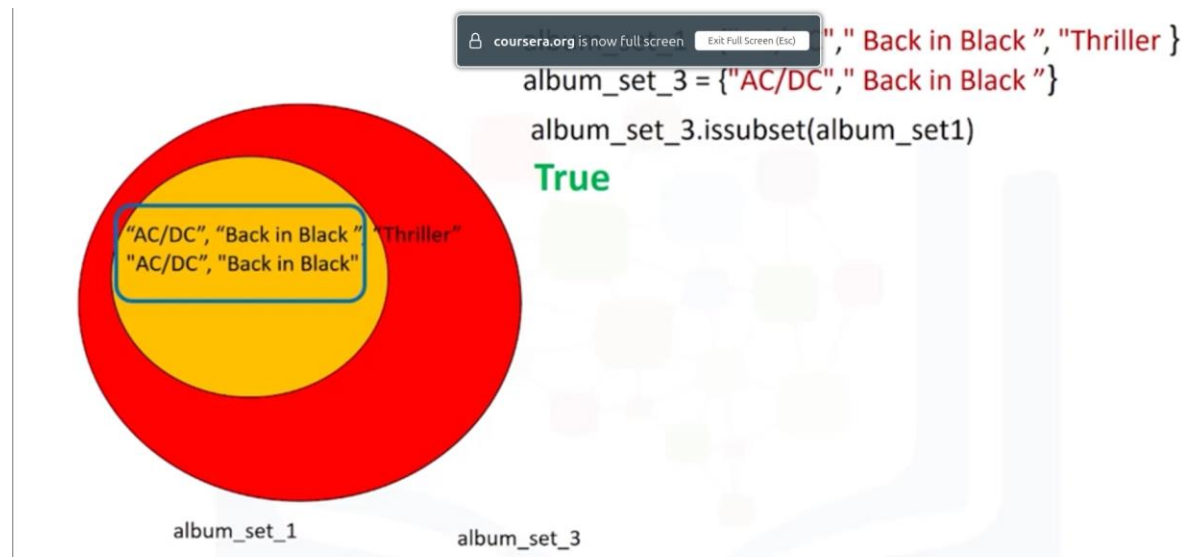
Union

```
album_set_1.union(album_set_2)
```

```
{"AC/DC", "Back in Black", "The Dark Side of the Moon", "Thriller"}
```



## Inclusion



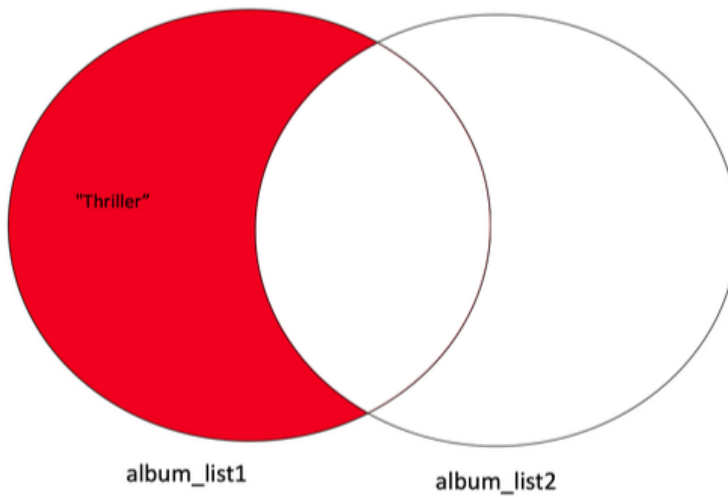
## Difference



```
album_set1.difference(album_set2)
```

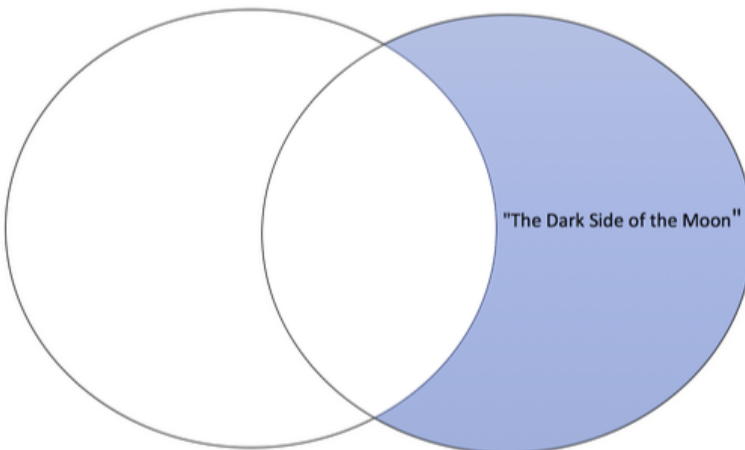
```
[12]: {'Thriller'}
```

You only need to consider elements in `album_set1`; all the elements in `album_set2`, including the intersection, are not included.



The elements in `album_set2` but not in `album_set1` is given by:

```
[ ]: album_set2.difference(album_set1)
```

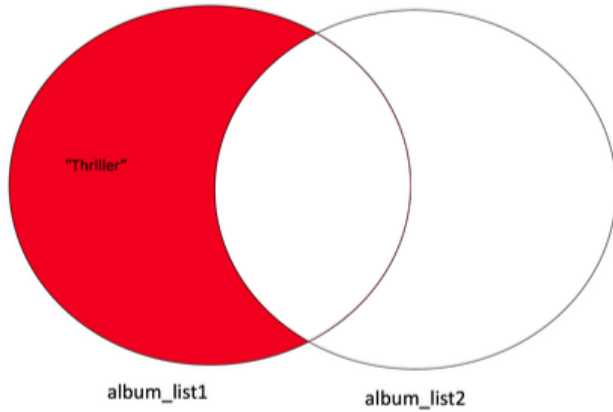


```
# Find the difference in set1 but not set2
```

```
album_set1.difference(album_set2)
```

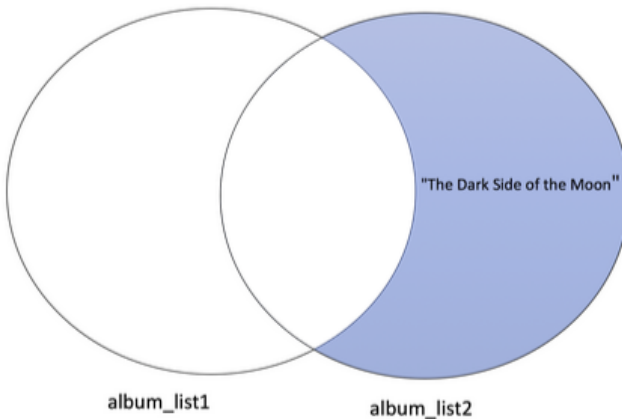
```
{'Thriller'}
```

You only need to consider elements in `album_set1`; all the elements in `album_set2`, including the intersection, are not included.



The elements in `album_set2` but not in `album_set1` is given by:

```
album_set2.difference(album_set1)
```



You can also find the intersection of `album_list1` and `album_list2`, using the `intersection` method:

```
# Use intersection method to find the intersection of album list1 and album list2
```

## Summary. Cheatsheet, Grades Quiz, Glossary

## Module3: Python Programming fundamentals

This module discusses Python fundamentals and begins with the concepts of conditions and branching. Continue through the module and learn how to implement loops to iterate over sequences, create functions to perform a specific task, perform exception handling to catch errors, and how classes are needed to create objects.

## Learning Objectives

- Classify conditions and branching by identifying structured scenarios with outputs.
- Work with objects and classes.
- Explain objects and classes by identifying data types and creating a class.
- Use exception handling in Python.
- Explain what functions do.
- Build a function using inputs and outputs.
- Explain how for loops and while loops work.
- Work with condition statements in Python, including operators and branching.
- Create and use loop statements in Python.

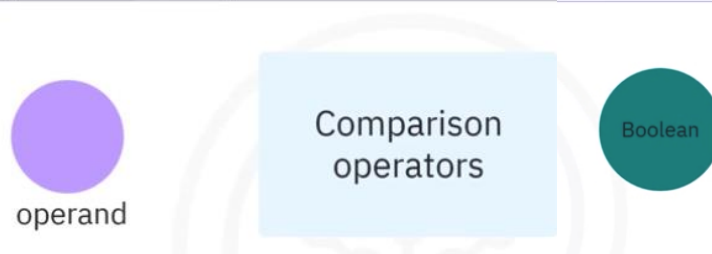
## Conditions and branching

### Conditions

Then based on some condition, they produce a Boolean.

### Comparison operators

---



Let's say we assign a value of `i` to six.

`i=6`

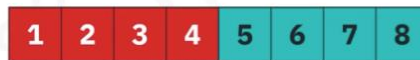
`i>5`

**True**

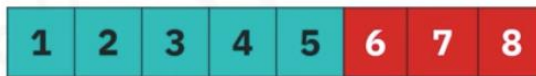


True

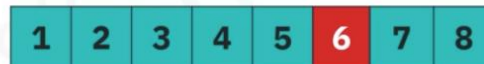
`i>=5`



`i<6`



`i!=6`



`"AC/DC"=="Michael Jackson"`

**False**

`"AC/DC"!="Michael Jackson"`

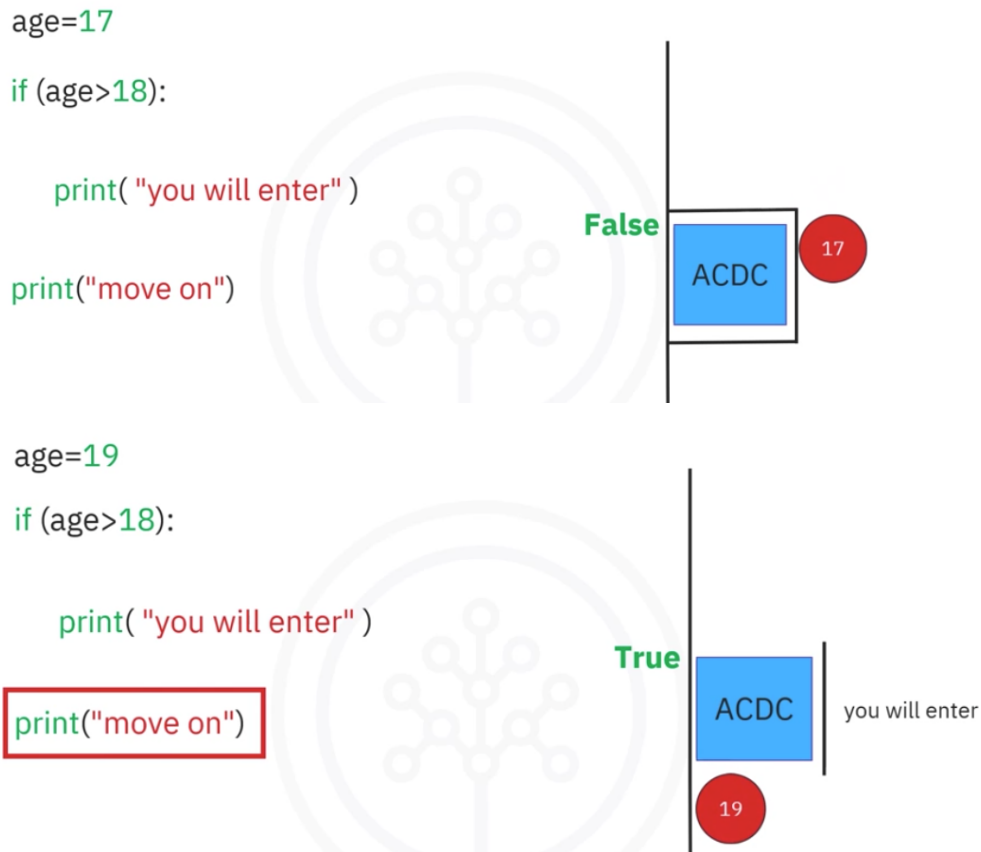
**True**

See the labs for more examples.

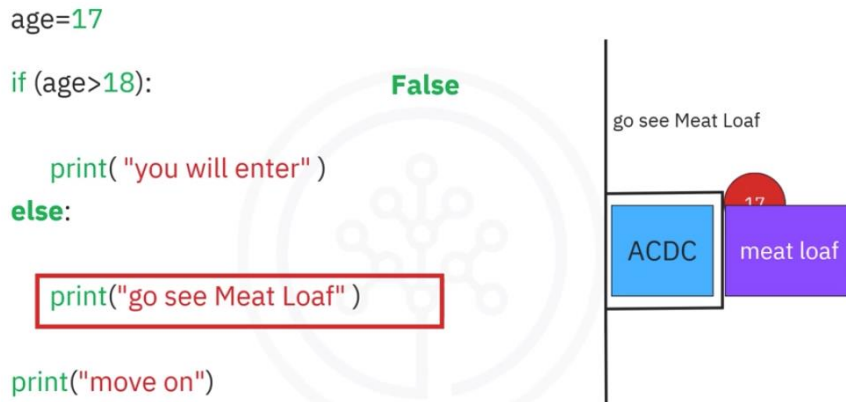
## Branchs

Branching allows us to run different statements for a different input. It's helpful to think of an if statement as a locked room.

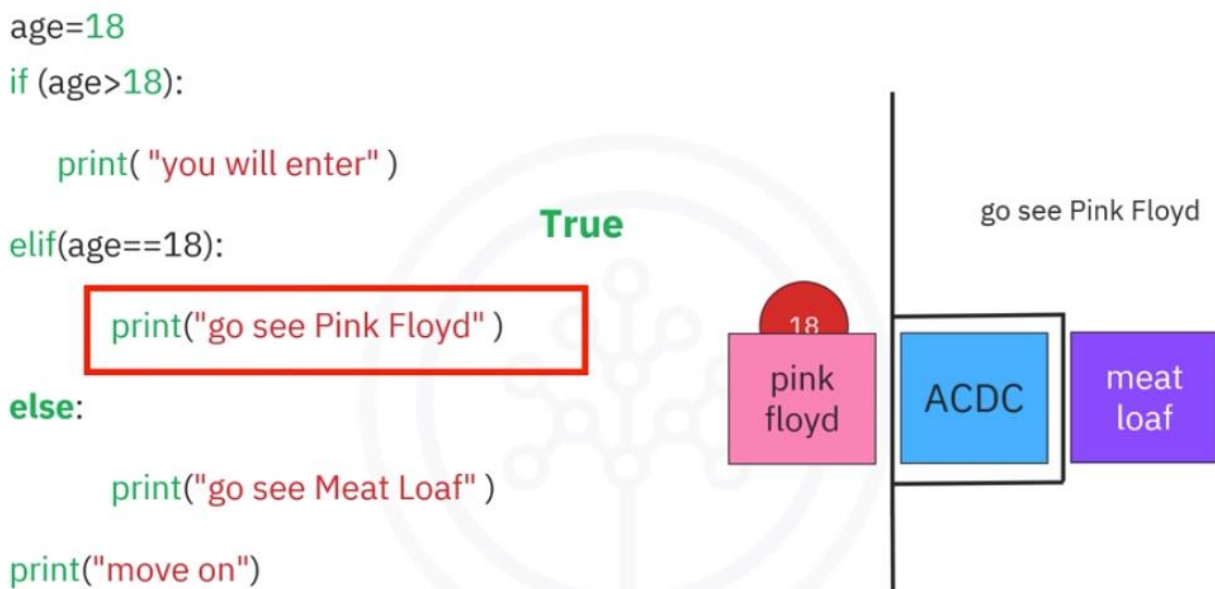
If this statement is true, you can enter the room and your program can run some predefined task. If the statement is false, your program will skip the task.



The else statement will run a different block of code if the same condition is false.

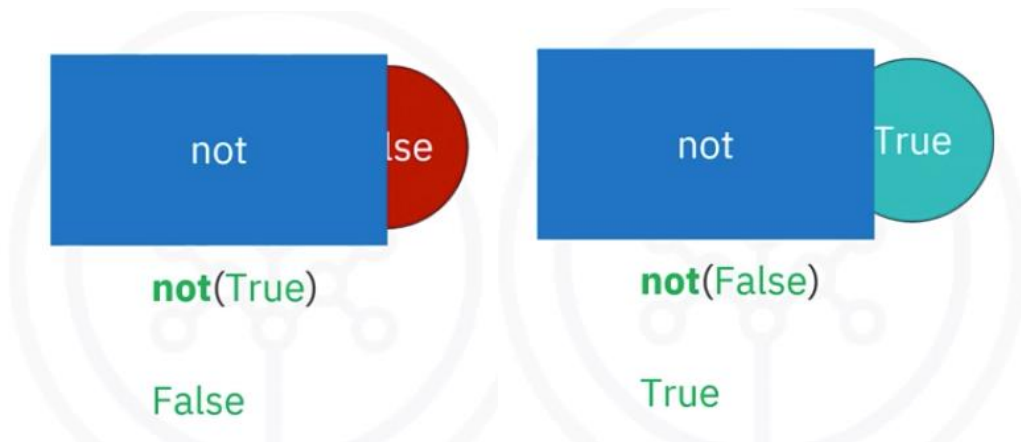


The syntax of the elif statement is similar. We simply add the statement elif with the condition. We, then add the expression we would like to execute if the statement is true with an indent.



## Logic Operators

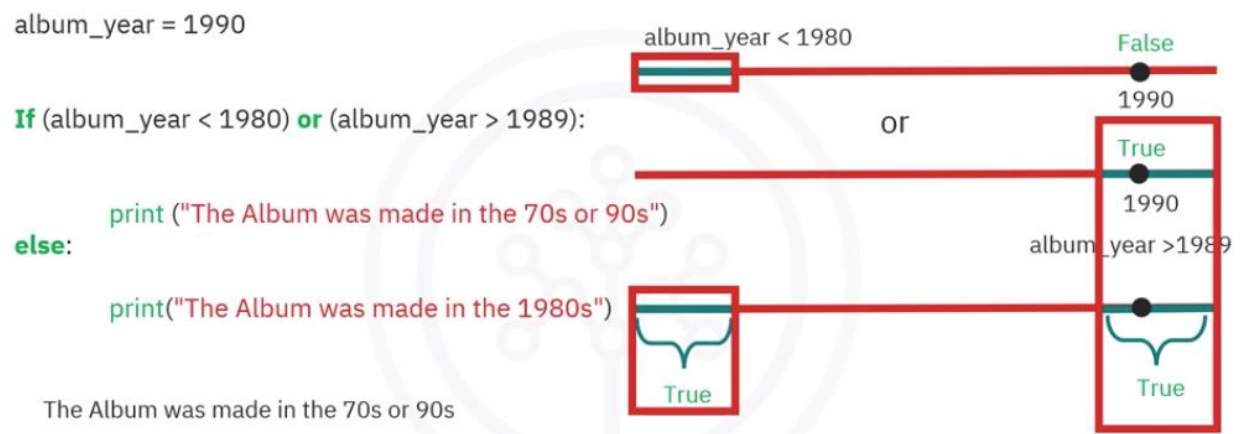
Logic operations take Boolean values and produce different Boolean values.



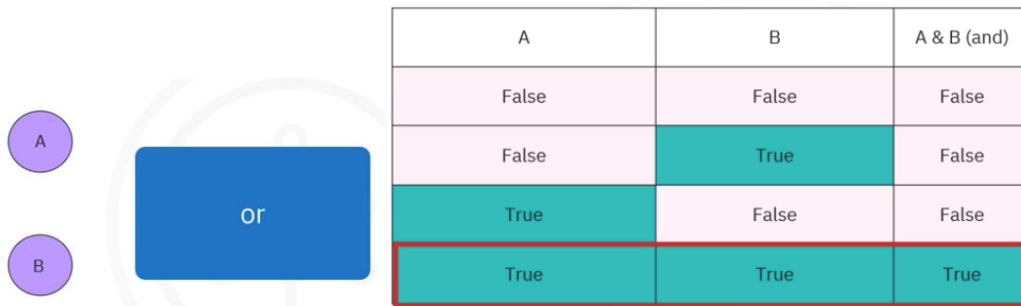
|  | A     | B     | A or B |
|--|-------|-------|--------|
|  | False | False | False  |
|  | False | True  | True   |
|  | True  | False | True   |
|  | True  | True  | True   |

The first operation is the not operator. If the input is true, the result is false. Similarly, if the input is false, the result is a true.

The OR operator takes in the two values and produces a new Boolean value. The OR operator only produces a false if all the Boolean values are false.



The AND operator takes in the two values and produces a new Boolean value.



We see the AND operator only produces a true if all the Boolean values are true.

album\_year = 1983

if(album\_year > 1979) and (album\_year < 1990):

print ("This album was made in the 80s")

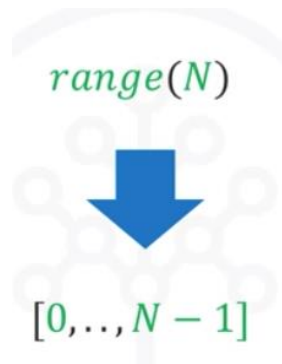


## Loops

### Range

The range function outputs an **ordered sequence as a list**.



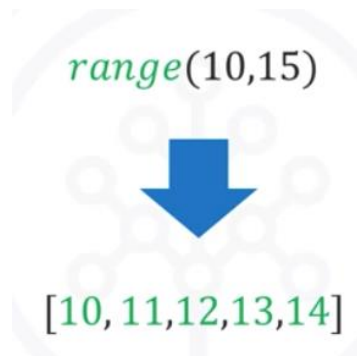


If the input is a positive integer, the output is a sequence.

The sequence contains the same number of elements as the input but starts at zero.

For example, if the input is three the output is the sequence zero, one, two.

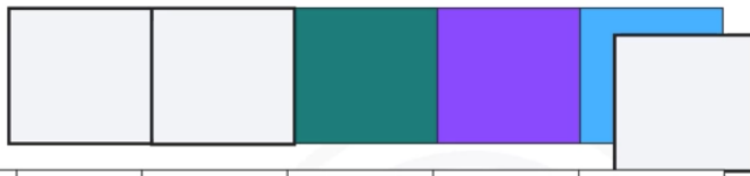
If the range function has two inputs where the first input is smaller than the second input, the output is a sequence that starts at the first input. Then the sequence iterates up to **but not including the second number**.



## For loop

In this section, we will cover for loops. We will focus on lists, but many of the procedures can be used on tuples. Loops perform a task over and over. Consider the group of colored squares. Let's say we would like to **replace each colored square with a white square**.

Let's give each square a number to make things a little easier and refer to all the group of squares as squares.



| squares | 0 | 1 | 2 | 3 | 4 |
|---------|---|---|---|---|---|
|---------|---|---|---|---|---|

For square 0 in squares, square 0 = white square

For square 1 in squares, square 1 = white square

For square 2 in squares, square 2 = white square

If we wanted to tell someone to replace squares zero with a white square, we would say equals replace square zero with a white square or we can say, For square zero in squares, square zero equals white square. Similarly, for the next square we can say for square one in squares, square one equals white square. For the next square we can say for square two in squares, square two equals white square. We repeat the process for each square.

The only thing that changes is the index of the square we are referring to.



| squares | 0 | 1 | 2 | 3 | 4 |
|---------|---|---|---|---|---|
|---------|---|---|---|---|---|

For square 0 in squares, square 0 = white square

For square 1 in squares, square 1 = white square

For square 2 in squares, square 2 = white square

For square 3 in squares, square 3 = white square

For square 4 in squares, square 4 = white square

If we're going to perform a similar task in Python, we cannot use actual squares.

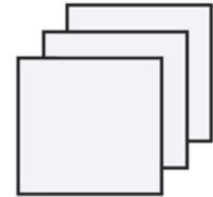


|                |     |        |       |        |      |
|----------------|-----|--------|-------|--------|------|
| <b>squares</b> | red | yellow | green | purple | blue |
|                | 0   | 1      | 2     | 3      | 4    |

```
squares=["red", "yellow", "green", "purple", "blue"]
```

```
for i in range(0,5):
```

```
    squares[i]="white"
```



|                |       |       |       |        |      |
|----------------|-------|-------|-------|--------|------|
| <b>squares</b> | white | white | green | purple | blue |
|                | 0     | 1     | 2     | 3      | 4    |

```
squares=["red", "yellow", "green", "purple", "blue"]
```

```
for i in range(0,5):
```

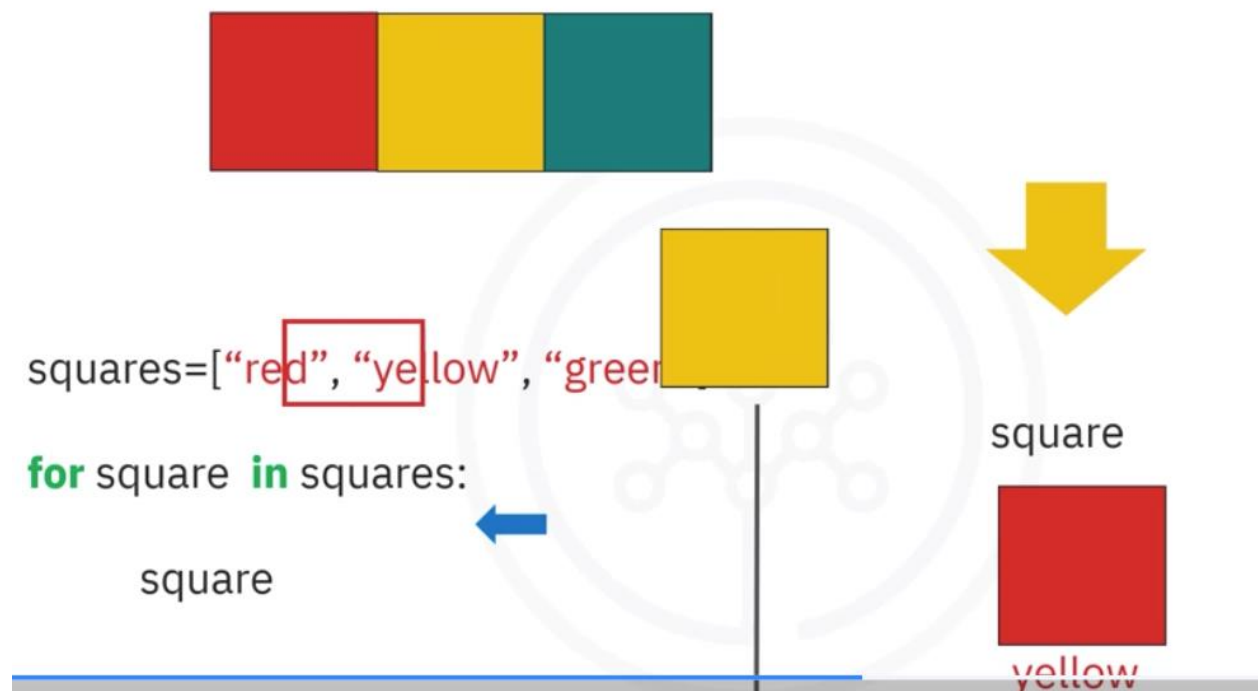
```
    squares[1]="white" ←
```

So let's use a list to represent the boxes. Each element in the list is a string representing the color. We want to change the name of the color in each element to white. Each element in the list has the following index. This is a syntax to perform a loop in Python. Notice the indent, the range function generates a list. The code will simply repeat everything in the indent five times. If you were to change the value to six it would do it 6 times. However, the value of `i` is incremented by one each time. In this segment we change the `i` element of the list to the string white. The value of `i` is set to zero. Each iteration of the loop starts at the beginning of the indent. We then run everything in the indent. The first element in

the list is set to white. We then go to the start of the indent, we progress down each line. When we reach the line to change the value of the list, we set the value of index one to white.

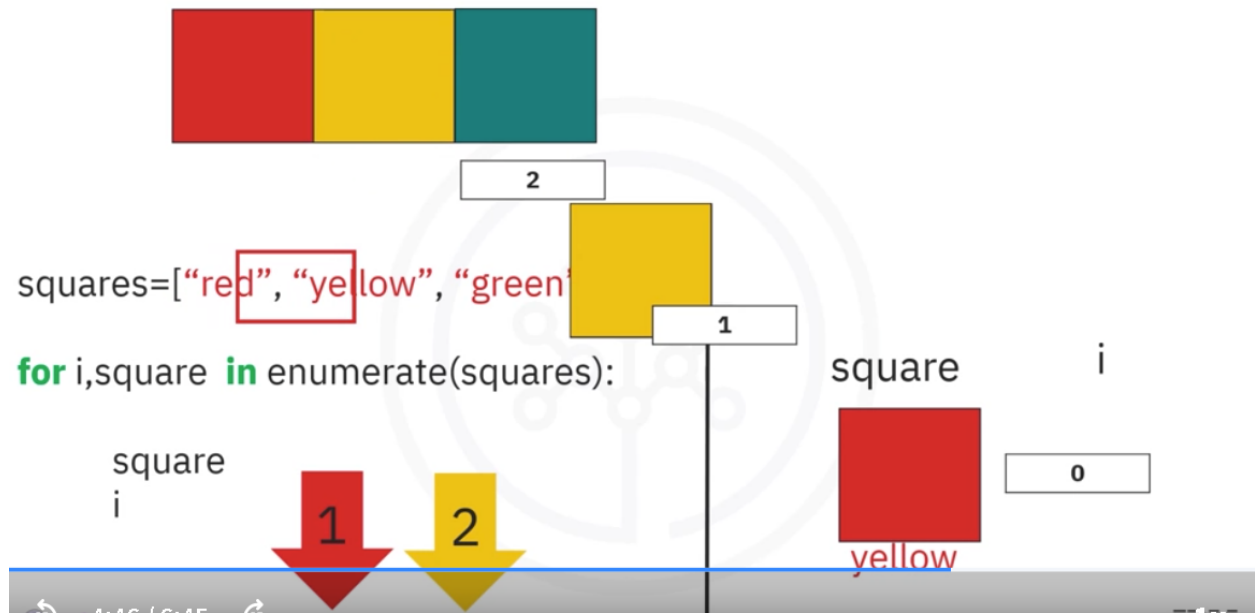
The value of `i` increases by one. We repeat the process for index two. The process continues for the next index, until we've reached the final element.

We can also iterate through a list or tuple directly in Python, **we do not even need to use indices**.



Here is the list `squares`. Each iteration of the list we pass one element of the list `squares` to the variable `square`. Let's display the value of the variable `square` on this section. For the first iteration, the value of `square` is red, we then start the second iteration. For the second iteration, the value of `square` is yellow. We then start the third iteration. For the final iteration, the value of `square` is green,

A useful function for iterating data is **`enumerate`**. It can be used to obtain the index and the element in the list. Let's use the box analogy with the numbers representing the index of each square.

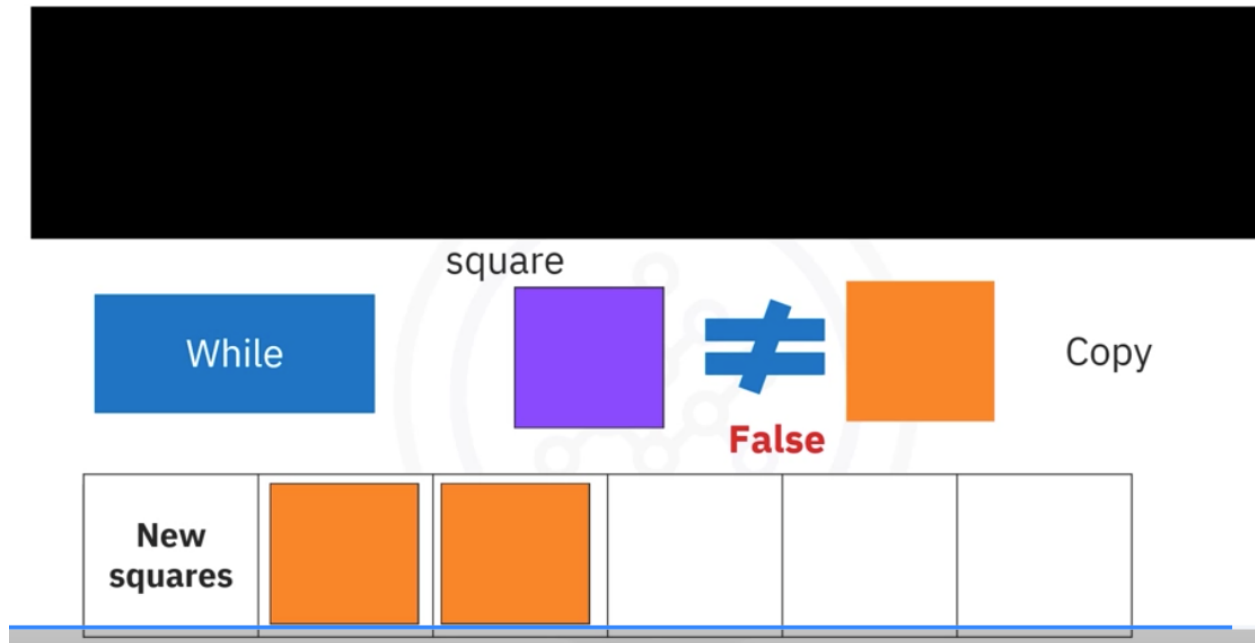


This is the syntax to iterate through a list and provide the index of each element. We use the list `squares` and use the names of the colors to represent the colored squares. The argument of the function `enumerate` is the list. In this case `squares` the variable `i` is the index and the variable `square` is the corresponding element in the list. Let's use the left part of the screen to display the different values of the variable `square` and `i` for the various iterations of the loop. For the first iteration, the value of the variable is `red` corresponding to the zeroth index, and the value for `i` is zero. For the second iteration, the value of the variable `square` is `yellow`, and the value of `i` corresponds to its index, i.e., 1. We repeat the process for the last index.

## While loop

While loops are similar to for loops but instead of executing a statement a set number of times a while loop **will only run if a condition is met**.





Let's say we would like to **copy all the orange squares from the list squares to the list New squares**.

But we would like to stop if we encounter a non-orange square. We don't know the value of the squares beforehand. We would simply continue the process while the square is orange or see if the square equals orange. If not, we would stop. For the first example, we would check if the square was orange. It satisfies the conditions so we would copy the square. We repeat the process for the second square. The condition is met. So we copy the square. In the next iteration, we encounter a purple square. The condition is not met. So we stop the process. This is essentially what a while loop does.

```
squares=['orange','orange','purple','orange','blue']
```

```
Newsquares=[]
```

```
i=0
```

```
while(squares[i]=='orange'):
```

```
    Newsquares.append(squares[i])
```

```
    i=i+1
```



```
while( squares[i] =='orange'):
```



Let's use the figure on the left to represent the code. We will use a list with the names of the color to represent the different squares. We create an empty list of new squares. In reality, the list is of indeterminate size. We start the index at zero the while statement will repeatedly execute the statements within the indent until the condition inside the bracket is false. We append the value of the first element of the list squares to the list Newsquares. We increase the value of I by one. We append the value of the second element of the list squares to the list new squares. We increment the value of I. Now the value in the array squares is purple; therefore, the condition for the while statement is false and we exit the loop.

## When to Use Each

**For Loops:** Use for loops when you know the number of iterations in advance and want to process each element in a sequence. They are best suited for iterating over collections and sequences where the length is known.

**While Loops:** Use while loops when you need to perform a task repeatedly as long as a certain condition holds true. While loops are particularly useful for situations where the number of iterations is uncertain or where you're waiting for a specific condition to be met.

## Functions

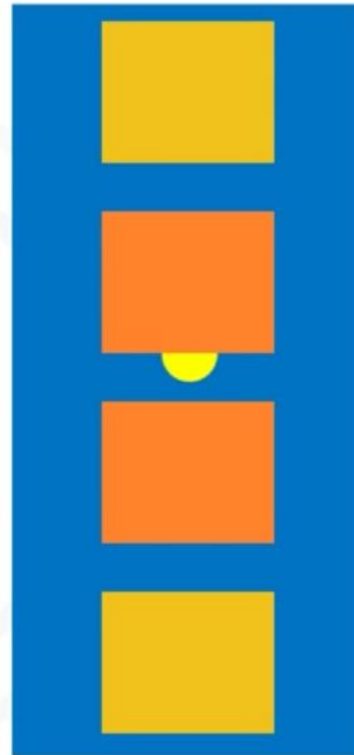
Functions **take some input** then **produce some output** or change. The function is just a piece of code you can reuse.

Let the orange and yellow squares represent similar blocks of code. We can run the code using some input and get an output. If we define a function to do the task, we just must call the function.

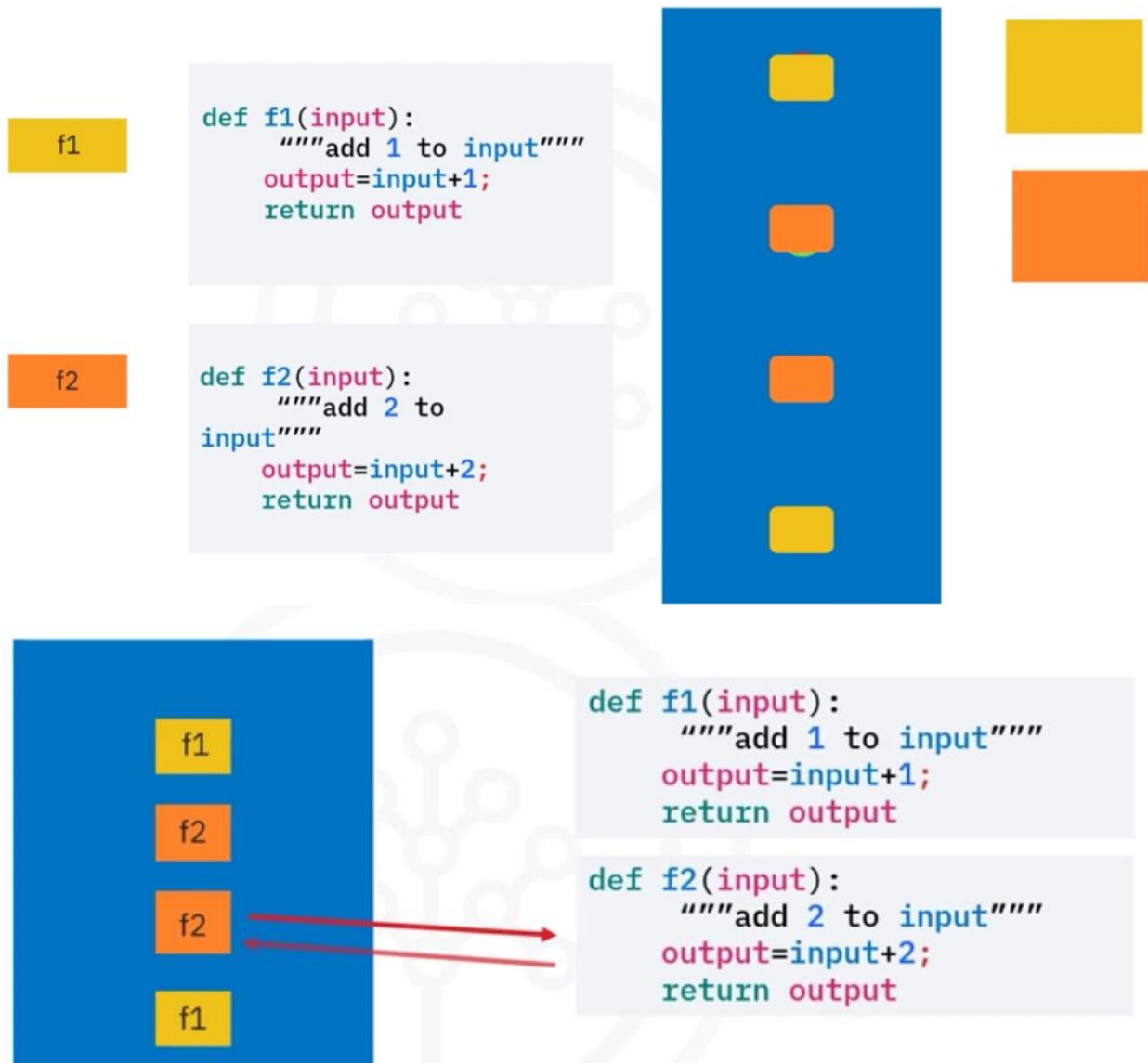
Let the small squares represent the lines of code used to call the function. We can replace these long lines of code by just calling the function a few times. Now we can just call the function; our code is much shorter. The code performs the same task.

```
def f1(input):  
    """add 1 to input"""  
    output=input+1;  
    return output
```

```
def f2(input):  
    """add 2 to  
input"""  
    output=input+2;  
    return output
```







You can think of the process like this: when we call the function f1, we pass an input to the function. These values are passed to all those lines of code you wrote. This returns a value; you can use the value. For example, you can input this value to a new function f2. When we call this new function f2, the value is passed to another set of lines of code. The function returns a value. The process is repeated passing the values to the function you call. You can save these functions and reuse them or use other people's functions.

## Built-in function

Python has many built-in functions; you don't have to know how those functions work internally, but simply what task those functions perform.

*Len()*

## Len

```
album_ratings = [10.0,8.5,9.5,7.0,7.0,9.5,9.0,9.5]
```

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|

```
L=len(album_ratings)
```

L:8

*Sum()*

## Sum

 coursera.org está ahora en pantalla completa [Salir de pantalla completa](#)

```
album_ratings = [10.0,8.5,9.5,7.0,7.0,9.5,9.0,9.5]
```

```
10.0+8.5+9.5+7.0+7.0+9.5+9.0+9.5
```

```
S=sum(album_ratings)
```

S:70

*Sort() versus sorted()*

There are two ways to sort a list.

- **function sorted**. The function sorted **returns a new sorted list or tuple**.

## Sorted versus sort

---

```
album_ratings = [10.0,8.5,9.5,7.0,7.0,9.5,9.0,9.5]
```

```
sorted_album_rating = sorted (album_ratings)
```

```
sorted_album_rating:
```

```
[7.0, 7.0, 8.5, 9.0, 9.5, 9.5, 9.5, 10.0]
```

```
album_ratings :
```

```
[10.0,8.5,9.5,7.0,7.0,9.5,9.0,9.5]
```

- list method sort.

## Sorted versus sort

---

```
album_ratings = [10.0,8.5,9.5,7.0,7.0,9.5,9.0,9.5]
```

```
album_ratings.sort()
```

```
album_ratings:
```

```
[7.0, 7.0, 8.5, 9.0, 9.5, 9.5, 9.5, 10.0]
```

**Methods are similar to functions.**

## Making functions

You can implement your own function, but in many cases, you use other people's functions. In this case, you just must know **how the function works** and, in some cases, **how to import the functions**.

# Building functions in Python

```
def add1(a):  
    b=a+1  
    return b
```

```
add1(5)  
6  
c=add1(10)  
c:11
```

This is an example of a function in Python that returns its input value + 1.

Define a function:

- 1.- **keyword** def.
- 2.-The **name** of the function should be **descriptive** of what it does.
- 3.-We have the function **formal parameter** "a" in parentheses.
- 4.-Followed by a **colon** ":".
- 5.- a **code block** with an **indent**, for this case, we add 1 to "a" and assign it to b.
- 6.-We **return** or output the value for b.

After we define the function, we can call it. The function will add 1 to 5 and return a 6. We can call the function again; this time assign it to the variable "c" The value for c is 11.

```
def add1(8):
```

```
    b=a+1
```

```
    return b
```

```
c=add1(5)
```

```
c:6
```

```
d=add1(8)
```

It should be noted that this is a **simplified model of Python**, and Python does not work like this under the hood.

We call the function giving it an input, 5. It helps to think of the value of 5 as being passed to the function. Now the sequences of commands are run, the value of "a" is 5. "b" would be assigned a value of 6. We then return the value of b, in this case, as b was assigned a value of 6, the function returns a 6. If we call the function again, the process starts from scratch;

we pass in an 8. The subsequent operations are performed. Everything that happened in the last call will happen again with a different value of "A" The function returns a value, in this case, 9.

## Adding documentation to functions

---

```
def add1(a):  
    """  
    add 1 to a  
    """  
    b=a+1;  
    return b
```

Documentation String

```
help(add1)
```

```
Help on function add1 in module __main__: add1(a) add 1 to a
```

It's customary to **document the function on the first few lines**; this tells anyone who uses the function what it does. This documentation is surrounded in **triple quotes**. You can use the help command on the function to display the documentation as follows. This will print out the function name and the documentation. We will not include the documentation in the rest of the examples.

*Multiple parameters.*

```
def Mult(a,b):  
    c=a*b  
    return c
```

*Multiplication of integer and strings*

## Multiplying integer and string in functions

```
def Mult(a,b):  
    c=a*b  
    return c
```



2\*"Michael Jackson "

"Michael Jackson Michael Jackson "

Mult(2,"Michael Jackson ")

"Michael Jackson Michael Jackson "

If we pass in the integer two and the string "Michael Jackson," the string Michael Jackson is repeated two times. This is because **the multiplication symbol can also mean repeat a sequence**. If you accidentally multiply an integer with a string instead of two integers, you won't get an error. Instead, you will get a string, and your program will progress, potentially failing later because you have a string where you expected an integer. This property will make coding simpler, but **you must test your code more thoroughly**.

### *Functions without return*

In many cases, a function does not have a return statement. In these cases, Python will return the **special "None" object**. Practically speaking, if your function has no return statement, you can treat it as if the function returns nothing at all.

### *Functions with Empty Body: Pass*

```
def NoWork():  
    pass  
print(NoWork())
```



```
def NoWork():  
    pass  
    return None
```

None

Let's define the function "No work" that performs no task. **Python doesn't allow a function to have an empty body**, so we can use the **keyword pass**, which doesn't do anything, but satisfies the requirement of a non-empty body. If we call the function and print it out, the function returns a None. In the background, if the return statement is not called, Python will automatically return a None. It is helpful to view the function No Work with the following return statement.

Usually, functions perform more than one task.

## Functions performing multiple tasks

```
def add1(a):  
    b=a+1;  
    print(a, "plus 1 equals ",b)  
    return b
```

|                        |                   |
|------------------------|-------------------|
| a                      | 2                 |
| b                      | 3                 |
| output of<br>print(... | 2 plus 1 equals 3 |
| value<br>returned      | 3                 |

```
add1(2)  
2 plus 1 equals 3  
3
```

## Using loops in functions

```
def printStuff(Stuff):  
    for i,s in enumerate(Stuff):  
        print("Album", i , "Rating is ", s)
```

Stuff: [10.0, 8.5, 9.5]  
Index: 

|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
|---|---|---|

```
album_ratings = [10.0,8.5,9.5]  
printStuff(album_ratings)
```

```
Album 0 Rating is 10  
Album 1 Rating is 8.5  
Album 2 Rating is 9.5
```

### *Variadic parameters*

Variadic parameters allow us to **input a variable number of elements**.

Consider the following function: the function has **an asterisk on the parameter names**. When we call the function, three parameters are packed into the tuple names. We then iterate through the loop; the values are printed out accordingly.



# Collecting arguments

```
def ArtistNames (*names):  
    for name in names:  
        print(name)
```



names=("Michael Jackson","AC/DC","Pink Floyd")

ArtistNames("Michael Jackson","AC/DC","Pink Floyd")

Michael Jackson  
AC/DC  
Pink Floyd

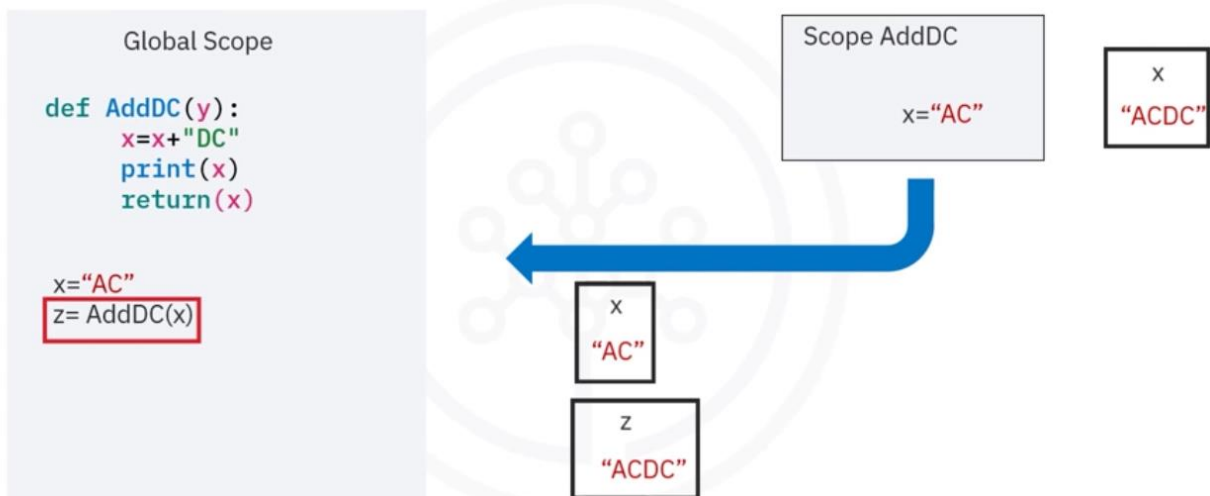
If we call the same function with only two parameters as inputs, the variable names only contain two elements. The result is only two values are printed out.

## Scope

The scope of a variable is the part of the program **where that variable is accessible**.

**Variables** that are **defined outside of any function** are said to be within the **global scope**, meaning they can be accessed anywhere after they are defined.

## Scope



Here we have a function that adds the string DC to the parameter x. When we reach the part where the value of x is set to AC, this is within the global scope, meaning x is accessible anywhere after it is defined. A variable defined in the global scope is called a global variable. When we call the function, we enter a new scope or the scope of AddDC. We pass as an

argument to the AddDC function, in this case, AC. Within the scope of the function, the value of x is set to ACDC. The function returns the value and is assigned to z. Within the global scope, the value z is set to ACDC. After the value is returned, the scope of the function is deleted.

### Local variables

Local variables **only exist within the scope of a function**. Consider the function Thriller; the local variable Date is set to 1982. When we call the function, we **create a new scope**. Within that scope of the function, the value of the date is set to 1982. The value of date does not exist within the global scope. **Variables inside the global scope can have the same name as variables in the local scope with no conflict.**

## Scope: Local variables

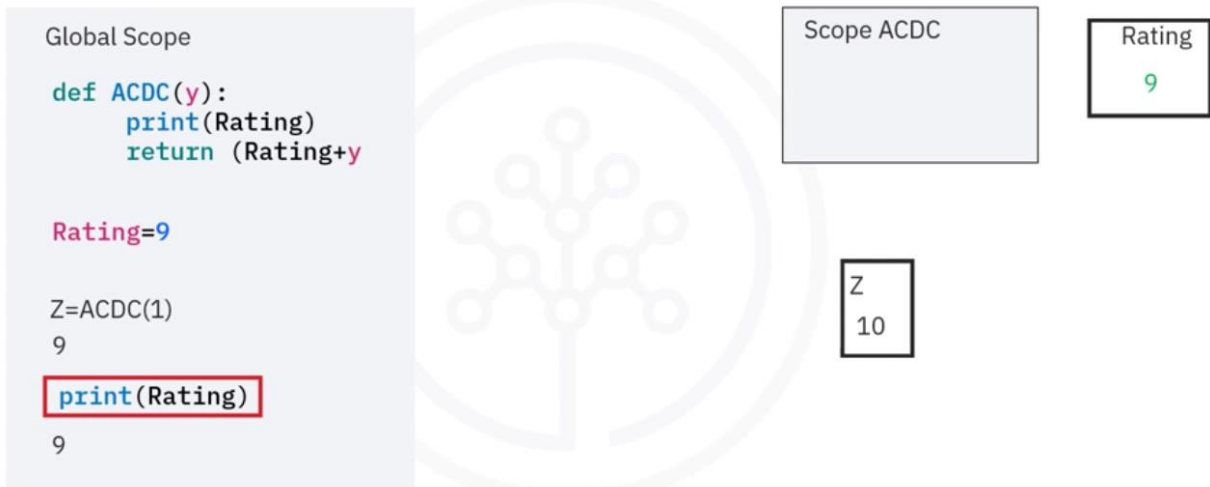


Consider the function Thriller; the local variable Date is set to 1982. The global variable date is set to 2017. When we call the function, we create a new scope. Within that scope, the value of the date is set to 1982. If we call the function, it returns the value of Date in the local scope, in this case, 1982. When we print in the global scope, we use the global variable value. The global value of the variable is 2017. Therefore, the value is set to 2017.

If a variable is not defined within a function, **Python will check the global scope**.

Consider the function ACDC.

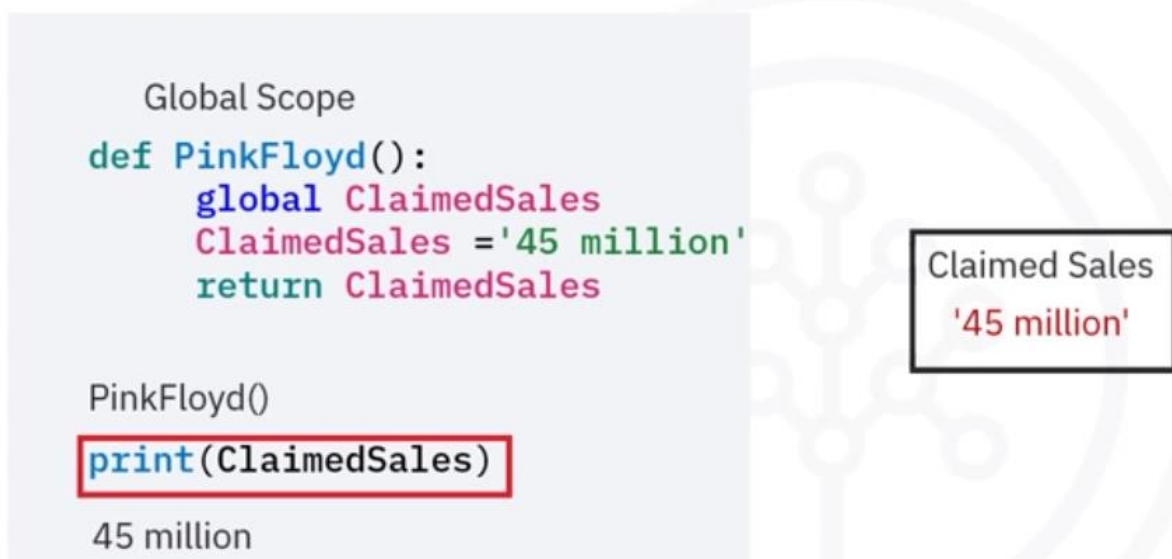
## Scope: Variables



The function has the variable `rating`, with no value assigned. If we define the variable `rating` in the global scope, then call the function, Python will see there is no value for the variable `Rating`. As a result, Python will leave the scope and check if the variable `Ratings` exists in the global scope. It will use this value of `Ratings` in the global scope within the scope of `ACDC`. In the function, will print out a `9`. The value of `z` in the global scope will be `10`, as we added one. The value of `rating` will be unchanged within the global scope.

Consider the function `PinkFloyd`.

## Scope: Local variables



If we define the variable ClaimedSales with the **keyword global**, the variable will be a global variable. We call the function PinkFloyd. The variable ClaimedSales is set to the string “45 million” in the global scope. When we print the variable, we get a value of “45 million.” There is a lot more you can do with functions.

*Default argument's values*

*My errors*

len([sum([1,1,1])) is different from len(sum([1,1,1])). First one is valid. The second one no. I did not pay attention to external square brackets

## Exception Handling

After watching this video, you will be able to **explain** exception handling, **demonstrate** the use of exception handling, and **understand** the basics of exception handling.

Have you ever mistakenly entered a number when you were supposed to enter text in an input field? Most of us have either in error or when testing out a program, but do you know why it gave an error message instead of completing and terminating the program?

For **the error message to appear, an event was triggered in the background**. This event was activated because the program tried to perform a computation on the name entry and realized the entry contained numbers and not letters.

By **encasing this code in an exception handler** the program knew how to deal with this type of error and was able to output the error message to continue along with the program.

This is one of many errors that can happen when asking for user input, so let's see how exception handling works.

We will first explore the try...except statement.

```
try:
    .....
except:
    .....
```

This type of statement will first attempt to execute the code in the “try” block, but if an error occurs it will kick out and **begin searching for the exception that matches** the error. Once it finds the correct exception to handle the error, it will then execute that line of code.

For example, perhaps you are writing a program that will open and write a file. After starting the program, an error occurred as the data was not able to be read. Because of this error the program skipped over the code lines under the “try” statement and went directly to the exception line. Since this error fell within the IOError guidelines it printed “Unable to open or read the data in the file.” to our console.

When writing simple programs, we can sometimes get away with only one except statement, but what happens if another error occurs that is not caught by the IOError? If that happened, we would need to add another **except statement**. For this except statement you will notice that the **type of error to catch is not specified**. While this may seem a logical step so the program will catch all errors and not terminate, this is **not a best practice**.

```
try:
    getfile = open("myfile", "r")
    getfile.write("My file for exception handling.")
except IOError:
    print("Unable to open or read the data in the file.")
except:
    print("Some other error occurred!")
```

For example, let’s say our small program was just one section of a much larger program that was over a thousand lines of code. Our task was to debug the program as it kept throwing an error causing a disruption for our users. When investigating the program you found this error kept appearing. Because this error had no details you ended up spending hours trying to pinpoint and fix the error.

So far in our program we have defined that an error message should print out if an error occurs, but we do not receive any **messages that the program executed properly**. This is where we can now add an **else statement** to give us that notification.

For this example, since we are opening a file the last thing, we need to do is close the file.

```
try:
    getfile = open("myfile", "r")
    getfile.write("My file for exception handling.")
except IOError:
    print("Unable to open or read the data in the file.")
else:
    print("The file was written successfully.")
finally:
    getfile.close()
    print("File is now closed.")
```

By adding a **finally statement** it will tell the program to close the file no matter the result and print "File is now closed" to our console.

It is **important to always define errors when creating exceptions**, and how to add an else and finally statement.

## Errors vs Exceptions

What is the difference between errors and exceptions? Well, **errors** are usually **big problems** that come from the computer or the system. They often make the program stop working completely. On the other hand, **exceptions** are more like **issues we can control**. They happen because of something we did in our code and can usually be fixed, so the program keeps going.

| Aspect   | Errors                                                                           | Exceptions                                                                                         |
|----------|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Origin   | Errors are typically caused by the environment, hardware, or operating system.   | Exceptions are usually a result of problematic code execution within the program.                  |
| Nature   | Errors are often severe and can lead to program crashes or abnormal termination. | Exceptions are generally less severe and can be caught and handled to prevent program termination. |
| Handling | Errors are <b>not usually caught or handled</b> by the program itself.           | Exceptions can be <b>caught using try-except blocks</b> and dealt with                             |

|                       |                                                                                                       |                                                                                                                                   |
|-----------------------|-------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
|                       |                                                                                                       | gracefully, allowing the program to continue execution.                                                                           |
| <b>Examples</b>       | Examples include “SyntaxError” due to incorrect syntax or “NameError” when a variable is not defined. | Examples include “ZeroDivisionError” when dividing by zero, or “FileNotFoundError” when attempting to open a non-existent file.   |
| <b>Categorization</b> | Errors are not classified into categories.                                                            | Exceptions are categorized into various classes, such as “ArithmeticError,” “IOError,” “ValueError,” etc., based on their nature. |

## Common Exceptions in Python

**ZeroDivisionError:** This error arises when an attempt is made to divide a number by zero.

**ValueError:** This error occurs when an inappropriate value is used within the code.

**FileNotFoundError:** This exception is encountered when an attempt is made to access a file that does not exist.

**IndexError:** An IndexError occurs when an index is used to access an element in a list that is outside the valid index range.

**KeyError:** The KeyError arises when an attempt is made to access a non-existent key in a dictionary.

**TypeError:** The TypeError occurs when an object is used in an incompatible manner.

**AttributeError:** An AttributeError occurs when an attribute or method is accessed on an object that doesn't possess that specific attribute or method.

**ImportError:** This error is encountered when an attempt is made to import a module that is unavailable.



## Exception's group

```
# potential code before try catch

try:
    # code to try to execute
except (ZeroDivisionError, NameError):
    # code to execute if there is an exception of the given types

# code that will execute if there is no exception or a one that we are handling
```

---

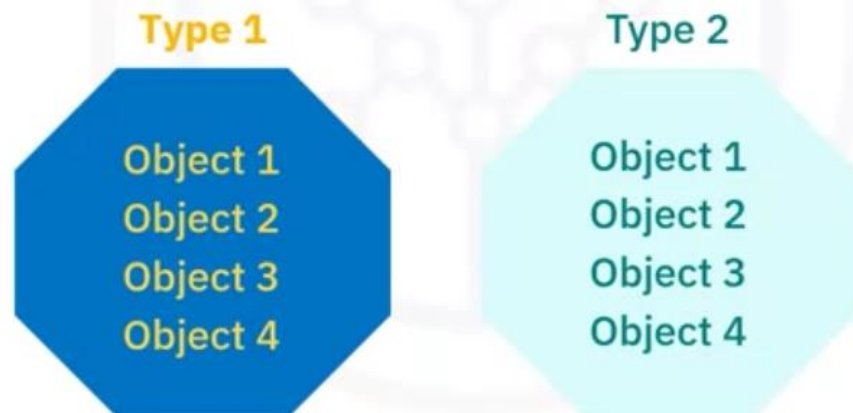
```
# potential code before try catch

try:
    # code to try to execute
except ZeroDivisionError:
    # code to execute if there is a ZeroDivisionError
except NameError:
    # code to execute if there is a NameError

# code that will execute if there is no exception or a one that we are handling
```

## Objects and classes

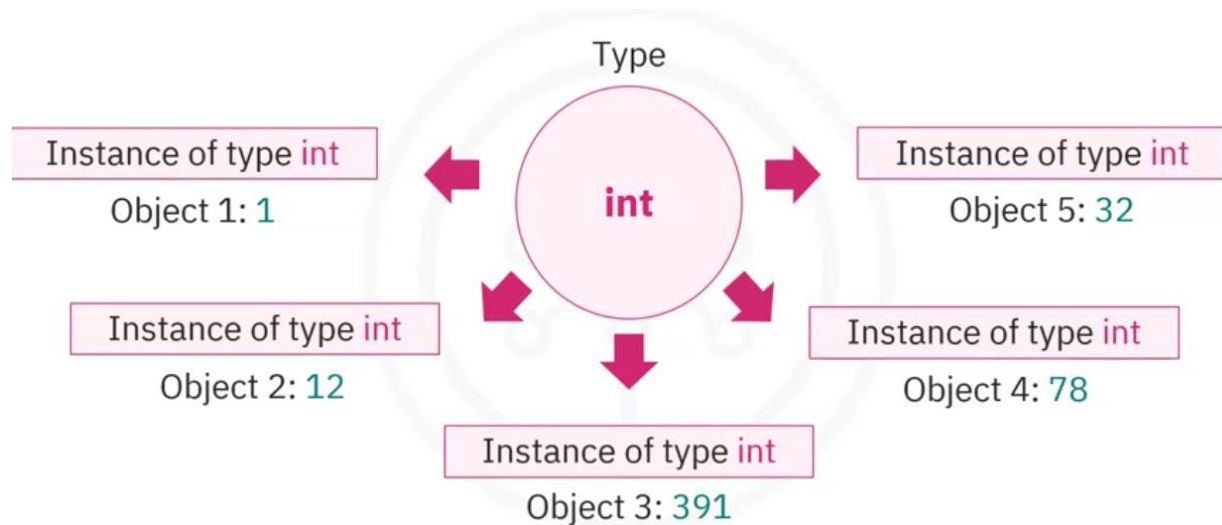
- Every **object** has:
  - A **type**
  - An internal data representation (a blueprint )
  - A set of procedures for interacting with the object (**method**)
- An **object** is an **instance** of a particular **type**



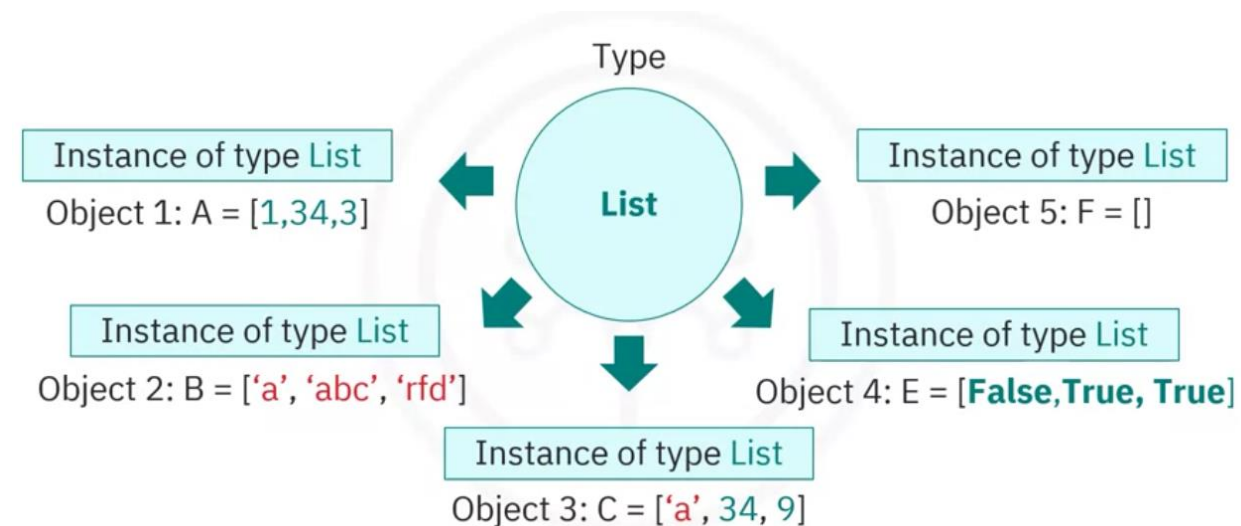


Let's do several less abstract examples.

Every time we create an **integer**, we are creating an **instance of type integer**, or we are creating an **integer object**. In this case, we are creating five instances of type integer or five integer objects.



Every time we create a **list**, we are creating an **instance of type list**, or we are creating a **list object**. In this case, we are creating five instances of type list or five list objects.



We could find out the type of an object by using the `type ()` command. In this case, we have an object of type list, we have an object of type integer, we have an object of type string. Finally, we have an object of type dictionary.

- You can find the type of an object by using the command `type()`

```
>>type([1,34,3])  
<class 'list'>
```

Instance of type `List`

`List`

```
>>type('The cat is  
yellow')  
<class 'str'>
```

Instance of type `str`

`str`

```
>>type(1)  
<class 'int'>
```

Instance of type `int`

`int`

```
>>type( {"dog": 1,  
"Cat": 2})  
<class 'dict'>
```

Instance of type `dict`

`dict`

## Methods

A class or type's methods **are functions** that every instance of that class or type provides. It's how you interact with the object. We have been using methods all this time, for example, on lists.

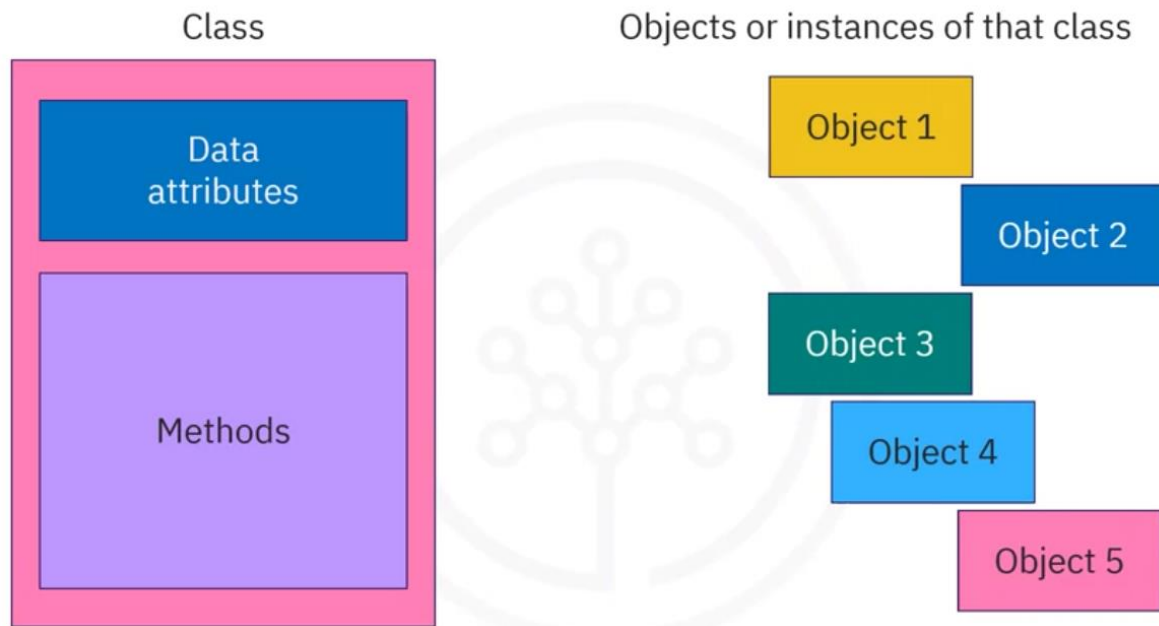
- `List.sort()`
- `List.reverse()`

We call the method **by adding a period at the end of the object's name**, and the method's name we would like to call with parentheses.

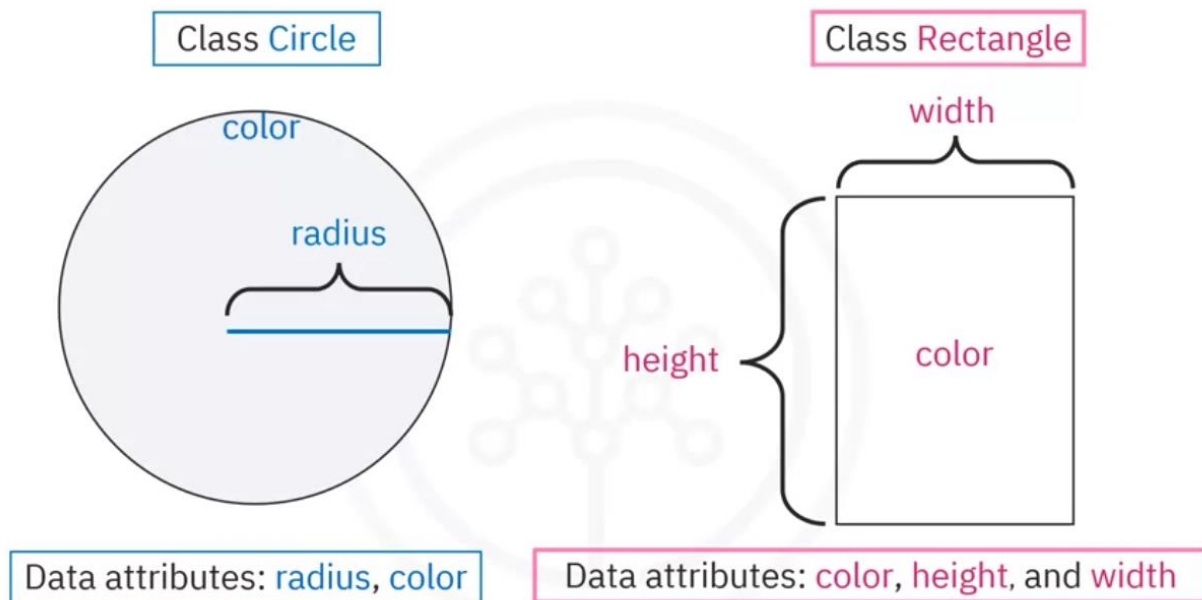
In many cases, you **don't have to know the inner workings** of the class and its methods, you just must know how to use them.

## Built own class

The class has data **attributes**. The class has **methods**. We then create **instances** or **instances of that class** or **objects**.



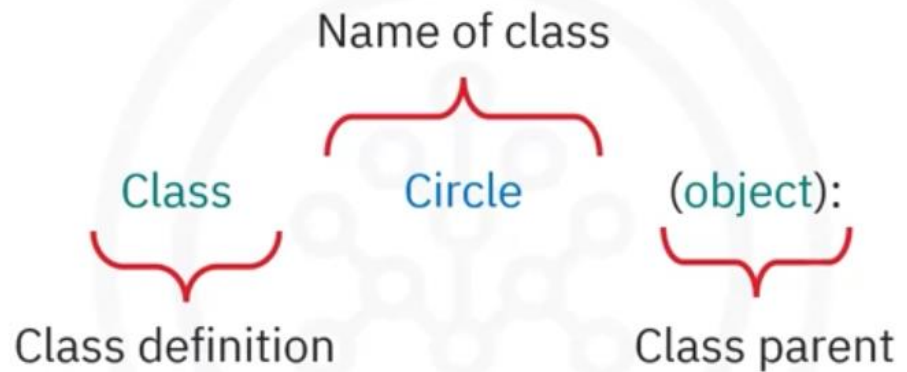
The class **data attributes** define the class.



To create the class circle,

# Create a class: Circle

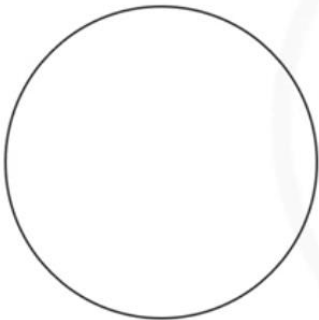
---



you will need to include the **class definition**. This tells Python you're creating your own class, the **name of the class**. For this course in parentheses, you will always place the term object, this is the **parent of the class**.

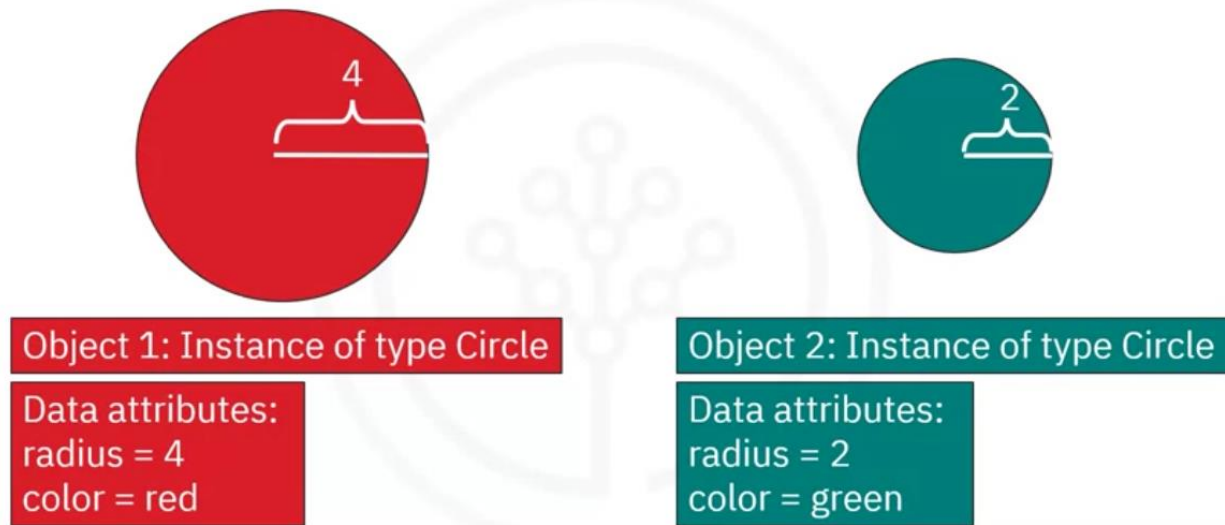
**Classes are outlines** we have to set the attributes to create objects.

Class Circle(object):



Class Rectangle(object):



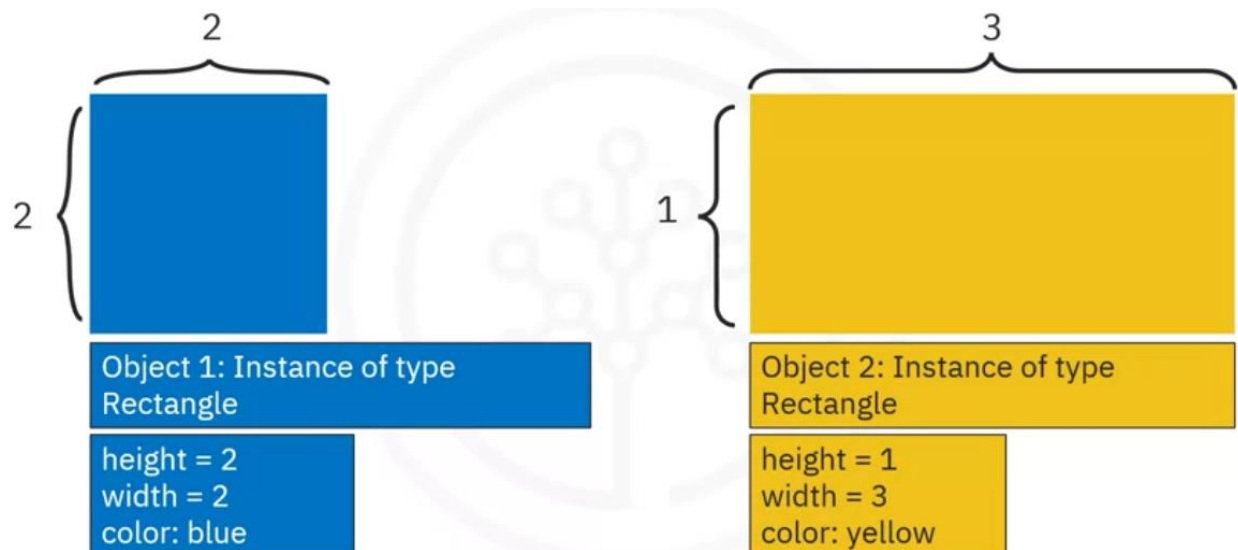


We can create an object that is an instance of type circle.

The color data attribute is red, and the data attribute radius is four. We could also create a second object that is an instance of type circle. In this case, the color data attribute is green, and the data attribute radius is two.

We can also create an object that is an instance of type rectangle.

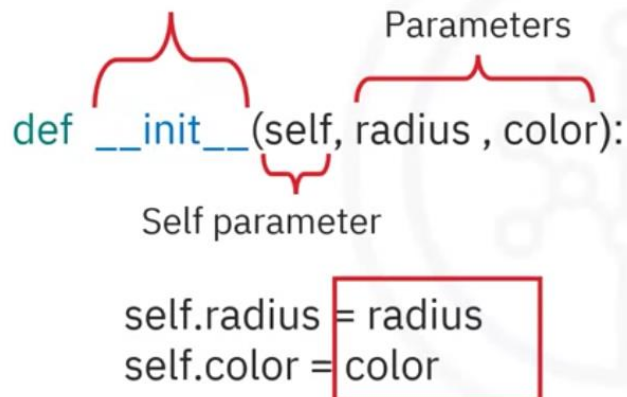
The color data attribute is blue, and the data attribute of height and width is two. The second object is also an instance of type rectangle. In this case, the color data attribute is yellow, and the height is one, and the width is three.



## Class definition

Special method or constructor used to initialize some data attributes

```
def __init__(self, radius, color):  
    self.radius = radius  
    self.color = color
```



5:40

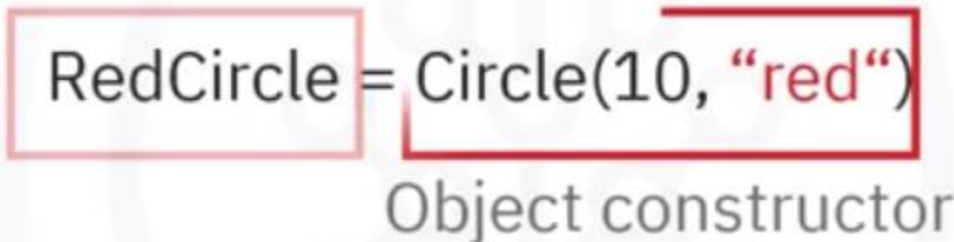
We then initialize each instance of the class with data attributes, radius and color, using the **class constructor**. The function **init** is a constructor. It's a special function that tells Python you are making a new class.

The radius and color parameters are used to initialize the radius and color data attributes of the class instance. The **self parameter refers to the newly created instance** of the class. The **parameters**, radius, and color can be used in the constructor's body to access the values passed to the class constructor when the class is constructed. We could set the value of the radius and color data **attributes** to the values passed to the constructor method.

After we've created the class, to create an object of class circle, we introduce a variable. This will be the **name of the object**. We create the object by using the **object constructor**.

Name of object

```
RedCircle = Circle(10, "red")
```



Object constructor

The object constructor consists of the **name of the class** as well as the parameters. These are the data attributes.

Name of class

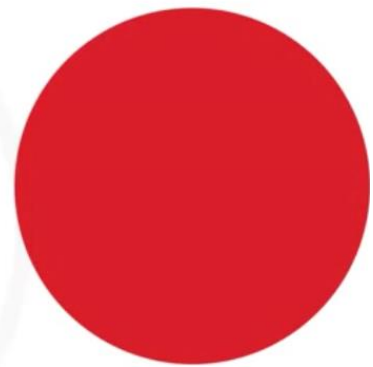
```
RedCircle = Circle(10, "red")
```

Attributes

When we create a circle object, we call the code like a function. The arguments passed to the circle constructor are used to initialize the data attributes of the newly created circle instance.

```
C1=Circle(10,'red')
```

```
class Circle(object):  
    def __init__(self, 10, 'red'):  
        self.radius = 10  
        self.color = 'red'
```



```
self.radius = 10  
self.color = 'red'
```

It is helpful to think of **self as a box that contains all the data attributes** of the object. Typing the object's name followed by a dot and the data attribute name gives us the data attribute value, for example, radius.

### Methods

Methods are functions that interact and change the data attributes, changing or using the data attributes of the object.



```
C1=Circle(2,'red' )
```

```
C1=add_radius(8)
```

```
self.radius = 2  
self.color = 'red'
```

```
def add_radius(self,8):  
    self.radius = 2+8  
    return (10)
```

```
self.radius = 10  
self.color = 'red'
```

In the constructor's body, the data attributes are set. We can use the box analogy to see the current state of the object. We **call the method by adding a dot followed by the method, name, and parentheses**.

Python will take care of self-parameter when calling the method. In many cases, there may not be any parameters other than self-specified in the method's definition. So we don't pass any arguments when calling the function.

Internally, the method is called with a value of eight, and the proper self-object. The method assigns a new value to self-radius. This changes the object, in particular, the radius data attribute.

When we call the add\_radius method, this changes the object by changing the value of the radius data attribute. We can add **default values to the parameters** of a class as constructor.

```
class Circle(object):  
    def __init__(self, radius=3 , color='red'):  
        self.radius = radius  
        self.color = color  
    def add_radius(self,r):  
        self.radius= self.radius +r  
  
    def drawCircle(self):
```

} We can add default values for parameters

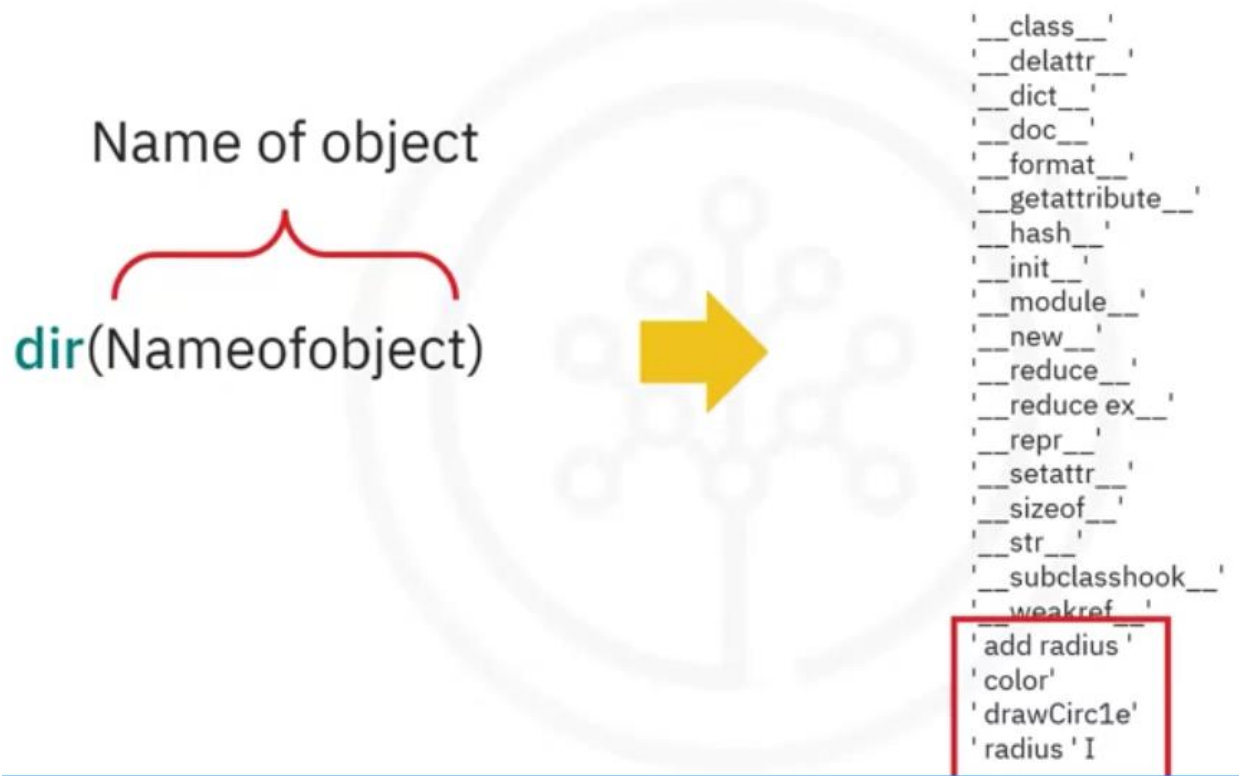
} New method

### *Dir function*

The dir function is useful for obtaining the list of data attributes and methods associated with a class. The object you're interested in is passed as an argument. The return value is a



list of the object's data attributes. The **attribute surrounded by underscores** are for internal use, and you shouldn't have to worry about them.



The regular looking attributes are the ones you should concern yourself with. These are the objects, methods, and data attributes.

## Summary. Cheatsheet, Grades Quiz, Glossary

- Python **conditions** use "if" statements to execute code based on true/false conditions created by comparisons and Boolean expressions.
- **Comparison** operations require using comparison operators equal to "=", greater than ">", less than "<".
- An exclamation mark "!" is used to define **inequalities** of a variable.
- You can compare integers, strings, and floats.
- Python **branching** directs program flow by using conditional statements (for example, if, else, elif) to execute different code blocks based on conditions or tests.
- You can use the "if" statement with conditions to define actions if true.

- To perform actions based on true or false output, you can use the "else" statement with conditions.
- The **elif statement** allows for additional checks only if the initial condition is false.
- To execute various operations on Boolean values, we use Boolean logic operators.
- Python **loops** are control structures that automate repetitive tasks and iterate over data structures like lists or dictionaries.
- The range() function generates a sequence of numbers with a specified start, stop, and step value for loops in Python.
- A **for loop** in Python iterates over a sequence, such as a list, tuple, or string, and executes a block of code for each item in the sequence.
- A **while loop** in Python executes a block of code as long as a specified condition remains true.
- Python functions are reusable code blocks that perform specific tasks, take input parameters, and often return results, enhancing code modularity and reusability.
- You may or may not have written the codes that are often included in functions.
- Python has a set of **built-in functions** such as "len" to find the length of a sequence or "sum" to find the total sum of a sequence.
- The "sorted" function creates a new sorted list, while "sort" sorts items in the original list.
- You can also create your own functions in Python.
- To ensure clarity and organization and facilitate understanding and maintenance of the code, developers must document functions using a documentation string enclosed in three quotes.
- The help command will return the documentation defined for a particular function.
- A function can have multiple parameters.
- "No return" statement in the function means that the function will return nothing.
- The "No work" function does not execute any task. You can use the "pass" keyword to meet the requirement of a non-empty body.
- A function will usually perform more than one task.
- In Python, the scope of a variable determines where you can access or modify that variable. Global scope allows access from anywhere, while local scope restricts it to a block or function.
- In Python, a programmer defines a local variable within a specific block or function, which can only be accessed or modified within that block or function.

- In Python, a global variable is a variable defined at the top level of a program that any part of the code can access or modify.
- Exception handling in Python is a mechanism for managing and responding to errors and exceptions that may occur during program execution, preventing them from crashing the program.
- In Python, you use the "try-except" statement to attempt a block of code and specify alternative actions to execute if an error occurs, allowing you to handle exceptions.
- In Python, you use the "try-except-else" statement to attempt a block of code, handle exceptions in the "except" block, and execute code in the "else" block when no exceptions occur.
- Python developers use the "try-except-else-finally" statement to attempt a block of code, catch exceptions in the "except" block, execute code in the "else" block when no exceptions occur, and ensure that the "finally" block always runs, regardless of whether an exception raised or not.
- In Python, objects are instances of classes that encapsulate data and behavior, serving as the foundation for creating and working with various data types and custom data structures.
- To determine the type of an object in Python, you can use the `type()` command.
- Any changes made within the method of the object may result in a change in object type.
- Classes in Python are blueprints for creating objects, defining their attributes and methods, enabling code organization, and object-oriented programming.
- Function "init" is a special method used to initialize data attributes.
- We can create instances of a class in Python.
- Data attributes consist of the data defining the objects.
- Methods are functions that interact and change the data attributes.
- The method has a function that requires the self as well as other parameters.

## Module 4: Working with Data in Python

This module explains the basics of working with data in Python and begins the path with learning how to read and write files. Continue the module and uncover the best Python libraries that will aid in data manipulation and mathematical operations.

## Learning Objectives

- Explain how Pandas use data frames.
- Use Pandas library for data analysis.
- Read text files using Python libraries including "open" and "with".
- Utilize NumPy to create one-dimensional and two-dimensional arrays.
- Write and save files in Python.

## Reading and writing files with open

Use Python's **built-in open ()** function to create a file object, and obtain the data from a "txt" file. apply a method to that object to read data from the file.

We can open the file, Example1.txt, as follows. We use the open function. The first argument is the **file path**. This is made up of the file **name**, and the file **directory**. The second parameter is the **mode**. Common values used include 'r' for reading, 'w' for writing, and 'a' for appending.

*File object*



```
file1 = open("/resources/data/Example1.txt", "r")
```

We can now **use the file object to obtain information about the file**.

file1



file1.name

'/resources/data/Example1.txt'

file1.mode

'r'

file1.close()

You should always close the file object using the method close.

This may get tedious sometimes,

*With Statement*

Indented block

```
with open("Example1.txt","r") as file1:
    file_stuff=file1.read()
    print(file_stuff)
    print(file1.closed)
    print(file_stuff)
```

Using a "with" statement to open a file is better practice because it **automatically closes the file**. The code will run everything in the indent block, then closes the file.

Reading

*Reading all file*

```
with open("Example1.txt","r") as file1:  
  
    file_stuff=file1.read()  
  
    print(file_stuff)  
  
print(file1.closed)  
print(file_stuff)
```

The method "read" **stores the values of the file** in the variable "file\_stuff" as a string.

We can print the file content.

```
print(file_stuff)
```

```
This is line 1  
This is line 2  
This is line 3
```

```
file_stuff:
```

```
This is line 1 \n This is line 2 \n This is line 3
```

We will see the following. When we examine the raw string, we will see the slash-n. This is so Python knows to start a new line.

We can output every line as an element in a list using the **method "readlines."**

```
with open("Example1.txt","r") as file1:
```

```
    file_stuff=file1.readline()  
    print(file_stuff)
```

file\_stuff[0]

|   |   |   |   |  |   |   |  |   |   |   |   |  |   |  |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|--|
| T | h | i | s |  | i | s |  | l | i | n | e |  | 1 |  |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|--|

file\_stuff[1]

|   |   |   |   |  |   |   |  |   |   |   |   |  |   |  |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|--|
| T | h | i | s |  | i | s |  | l | i | n | e |  | 2 |  |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|--|

file\_stuff: ['This is line 1 \n', 'This is line 2 \n', 'This is line 3']

*Reading line by line*

```
with open("Example1.txt","r") as file1:
```

```
    file_stuff=file1.readline()  
    print(file_stuff)
```

file\_stuff[0]

|   |   |   |   |  |   |   |  |   |   |   |   |  |   |  |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|--|
| T | h | i | s |  | i | s |  | l | i | n | e |  | 1 |  |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|--|

file\_stuff[1]

|   |   |   |   |  |   |   |  |   |   |   |   |  |   |  |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|--|
| T | h | i | s |  | i | s |  | l | i | n | e |  | 2 |  |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|--|

file\_stuff: ['This is line 1 \n', 'This is line 2 \n', 'This is line 3']

Reading with a loop

```
with open("Example1.txt","r") as file1:
```

```
    for line in file1:
        print(line)
```

This is line 1  
This is line 2  
This is line 3

|   |   |   |   |  |   |   |  |   |   |   |   |  |   |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|
| T | h | i | s |  | i | s |  | l | i | n | e |  | 1 |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|

|   |   |   |   |  |   |   |  |   |   |   |   |  |   |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|
| T | h | i | s |  | i | s |  | l | i | n | e |  | 2 |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|

|   |   |   |   |  |   |   |  |   |   |   |   |  |   |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|
| T | h | i | s |  | i | s |  | l | i | n | e |  | 3 |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|

Reading a certain number of characters

```
with open("Example1.txt","r") as file1:
```

```
    file_stuff=file1.readline(16)
    print(file_stuff)
    file_stuff=file1.readline(5)
    print(file_stuff)
    file_stuff=file1.readline(9)
    print(file_stuff)
```

|   |   |   |   |  |   |   |  |   |   |   |   |  |   |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|
| T | h | i | s |  | i | s |  | l | i | n | e |  | 1 |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|

|   |   |   |   |
|---|---|---|---|
| T | h | i | s |
|---|---|---|---|

|   |   |  |   |   |   |   |  |   |
|---|---|--|---|---|---|---|--|---|
| i | s |  | l | i | n | e |  | 2 |
|---|---|--|---|---|---|---|--|---|

|   |   |   |   |  |   |   |  |   |   |   |   |  |   |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|
| T | h | i | s |  | i | s |  | l | i | n | e |  | 1 |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|

|   |   |   |   |  |   |   |  |   |   |   |   |  |   |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|
| T | h | i | s |  | i | s |  | l | i | n | e |  | 2 |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|

|   |   |   |   |  |   |   |  |   |   |   |   |  |   |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|
| T | h | i | s |  | i | s |  | l | i | n | e |  | 3 |
|---|---|---|---|--|---|---|--|---|---|---|---|--|---|

Writing

## File Object





We use the 'open' function. The first argument is the file **path**. This is made up of the **file name** (if you have that file in your directory, it **will be overwritten**,) and the file directory. We set the **mode** parameter to **W** for writing.

Finally, we have the **file object**. As before, we use the '**with**' **statement**. The code will run everything in the indent block, then close the file. We create the file object, File1. We use the **open function**. This creates a file Example2.txt in your directory. We use the **method write**, to write data into the file.

---

```
with open("/resources/data/Example2.txt", "w") as file1:
```

```
    file1.write ( "This is line A")
```

The argument is the text we would like input into the file. If we use the write method successively, each time it is called, it will write to the file. The first time it is called, we will write, "This is line A" with a slash-n to represent a new line. The second time we call the method, it will write, "This is line B" then it will close the file.

---

```
with open("/resources/data/Example2.txt", "w") as file1:
```

```
    file1.write ( "This is line A\n")
```

```
    file1.write ( "This is line B\n")
```



This is line B  
This is line A

Example2.txt

### Writing lines from a list

```
Lines=["This is line A\n","This is line B\n","This is line C\n"]
```

```
with open("/resources/data/Example2.txt", "w") as file1:
```

```
    for line in Lines:
```

```
        This is line B
```



```
        file1.write(line)
```

```
        This is line A
```

Example2.txt

### Adding content to a file

We can set the mode to appended using a **lowercase 'a'**. This will not create a new file but just use the existing file. If we call the method write, it will just write to the existing file, then add "This is line C" then close the file.

```
with open("/resources/data/Example2.txt", "a") as file1:
```

```
    file1.write ("This is line C")
```

```
    This is line A
```

```
    This is line B
```

```
    This is line C
```

Example2.txt

## Coyping files

```
with open("Example1.txt", "r") as readfile:
```

```
with open("Example3.txt", "w") as writefile:
```



```
for line in readfile:
```

```
    writefile.write(line)
```

This is line A  
This is line B  
This is line C

This is line A  
This is line B

## Other file object methods

- Truncate() will reduce the file to your data and delete everything that follows.
- Tell() returns the current position in bytes
- Seek(offset, from) changes the position by 'offset' bytes with respect to 'from'. From can take the value of 0,1,2 corresponding to beginning, relative to current position and end

## Open Modes

The following table provides an overview of different file modes, their syntax, and common use cases. Understanding these modes is essential when working with files in Python for various data manipulation tasks.

| Mode | Syntax | Description                                                                                                     |
|------|--------|-----------------------------------------------------------------------------------------------------------------|
| 'r'  | 'r'    | Read mode. Opens an existing file for reading. Raises an error if the file doesn't exist.                       |
| 'w'  | 'w'    | Write mode. Creates a new file for writing. Overwrites the file if it already exists.                           |
| 'a'  | 'a'    | Append mode. Opens a file for appending data. Creates the file if it doesn't exist.                             |
| 'x'  | 'x'    | Exclusive creation mode. Creates a new file for writing but raises an error if the file already exists.         |
| 'rb' | 'rb'   | Read binary mode. Opens an existing binary file for reading.                                                    |
| 'wb' | 'wb'   | Write binary mode. Creates a new binary file for writing.                                                       |
| 'ab' | 'ab'   | Append binary mode. Opens a binary file for appending data.                                                     |
| 'xb' | 'xb'   | Exclusive binary creation mode. Creates a new binary file for writing but raises an error if it already exists. |

|      |      |                                                                                                                              |
|------|------|------------------------------------------------------------------------------------------------------------------------------|
| 'rt' | 'rt' | Read text mode. Opens an existing text file for reading. (Default for text files)                                            |
| 'wt' | 'wt' | Write text mode. Creates a new text file for writing. (Default for text files)                                               |
| 'at' | 'at' | Append text mode. Opens a text file for appending data. (Default for text files)                                             |
| 'xt' | 'xt' | Exclusive text creation mode. Creates a new text file for writing but raises an error if it already exists.                  |
| 'r+' | 'r+' | Read and write mode. Opens an existing file for both reading and writing.                                                    |
| 'w+' | 'w+' | Write and read mode. Creates a new file for reading and writing. Overwrites the file if it already exists.                   |
| 'a+' | 'a+' | Append and read mode. Opens a file for both appending and reading. Creates the file if it doesn't exist.                     |
| 'x+' | 'x+' | Exclusive creation and read/write mode. Creates a new file for reading and writing but raises an error if it already exists. |

## Pandas

**Dependencies or libraries** are pre written code to help solve problems. You can import the library or dependency like pandas using the **import command** followed by the name of the library. You now have access to a large number of pre-built classes and functions.

## Importing Pandas

```
import pandas
```

pandas

read\_csv()

Series()

DataFrame

Values

:

:

:

Load a CSV file

## Importing Pandas

---

```
import pandas

csv_path='file1.csv'

df=pandas.read_csv(csv_path)
```

pandas

|            |
|------------|
| read_csv() |
| Series()   |
| DataFrame  |
| values     |
| :          |
| :          |
| :          |
|            |

Let us say we would like to load a CSV file using pandas built in function, read\_csv. A CSV is a typical file type used to store data. Simply type the word pandas, then a dot and the name of the function with all the inputs.

Typing pandas all the time make it difficult, so you can use the **As statement to shorten the name of the library**. In this case, use the standard abbreviation pd. Now type pd, next, type a dot followed by the name of the function you would like to use. In this case, read\_csv.

```
import pandas as pd

csv_path='file1.csv'

df= pd.read_csv(csv_path)
```

### Dataframes

One way pandas allows you to work with data is in a dataframe.

```

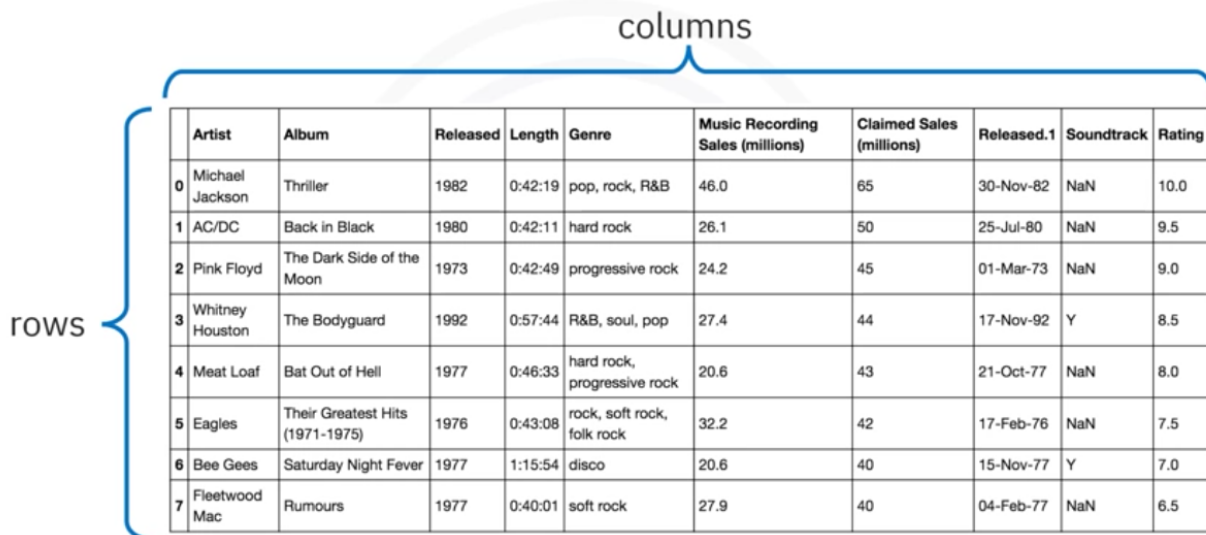
xlsx_path='file1.xlsx'
df= pd.read_excel(xlsx_path)
df.head()

```

|   | Artist          | Album                     | Released | Length  | Genre                       | Music Recording Sales (millions) | Claimed Sales (millions) | Released.1 | Soundtrack | Rating |
|---|-----------------|---------------------------|----------|---------|-----------------------------|----------------------------------|--------------------------|------------|------------|--------|
| 0 | Michael Jackson | Thriller                  | 1982     | 0:42:19 | pop, rock, R&B              | 46.0                             | 65                       | 30-Nov-82  | NaN        | 10.0   |
| 1 | AC/DC           | Back in Black             | 1980     | 0:42:11 | hard rock                   | 26.1                             | 50                       | 25-Jul-80  | NaN        | 9.5    |
| 2 | Pink Floyd      | The Dark Side of the Moon | 1973     | 0:42:49 | progressive rock            | 24.2                             | 45                       | 01-Mar-73  | NaN        | 9.0    |
| 3 | Whitney Houston | The Bodyguard             | 1992     | 0:57:44 | R&B, soul, pop              | 27.4                             | 44                       | 17-Nov-92  | Y          | 8.5    |
| 4 | Meat Loaf       | Bat Out of Hell           | 1977     | 0:46:33 | hard rock, progressive rock | 20.6                             | 43                       | 21-Oct-77  | NaN        | 8.0    |

Let us review the process of going from a CSV or excel file to a dataframe. The result is stored to the variable **df**. **This is short for dataframe**. Now that you have the data in a dataframe, you can work with it. You can use the method **head** to **examine the first five rows** of a dataframe.

A dataframe is comprised of **rows and columns**.



|   | Artist          | Album                           | Released | Length  | Genre                       | Music Recording Sales (millions) | Claimed Sales (millions) | Released.1 | Soundtrack | Rating |
|---|-----------------|---------------------------------|----------|---------|-----------------------------|----------------------------------|--------------------------|------------|------------|--------|
| 0 | Michael Jackson | Thriller                        | 1982     | 0:42:19 | pop, rock, R&B              | 46.0                             | 65                       | 30-Nov-82  | NaN        | 10.0   |
| 1 | AC/DC           | Back in Black                   | 1980     | 0:42:11 | hard rock                   | 26.1                             | 50                       | 25-Jul-80  | NaN        | 9.5    |
| 2 | Pink Floyd      | The Dark Side of the Moon       | 1973     | 0:42:49 | progressive rock            | 24.2                             | 45                       | 01-Mar-73  | NaN        | 9.0    |
| 3 | Whitney Houston | The Bodyguard                   | 1992     | 0:57:44 | R&B, soul, pop              | 27.4                             | 44                       | 17-Nov-92  | Y          | 8.5    |
| 4 | Meat Loaf       | Bat Out of Hell                 | 1977     | 0:46:33 | hard rock, progressive rock | 20.6                             | 43                       | 21-Oct-77  | NaN        | 8.0    |
| 5 | Eagles          | Their Greatest Hits (1971-1975) | 1976     | 0:43:08 | rock, soft rock, folk rock  | 32.2                             | 42                       | 17-Feb-76  | NaN        | 7.5    |
| 6 | Bee Gees        | Saturday Night Fever            | 1977     | 1:15:54 | disco                       | 20.6                             | 40                       | 15-Nov-77  | Y          | 7.0    |
| 7 | Fleetwood Mac   | Rumours                         | 1977     | 0:40:01 | soft rock                   | 27.9                             | 40                       | 04-Feb-77  | NaN        | 6.5    |

You can **create a dataframe out of a dictionary**. The keys correspond to the column labels. The values are lists corresponding to the rows. You can then cast the dictionary to a dataframe using the function `DataFrame`. Notice the direct correspondence between the table. The **keys correspond to the table headers**. The **values are lists corresponding to the rows**.

```
songs = { 'Album' : ['Thriller','Back in Black','The Dark Side of the Moon',\
                    'The Bodyguard','Bat Out of Hell'],
          'Released' : [1982,1980,1973,1992,1977],
          'Length' : ['00:42:19','00:42:11','00:42:49','00:57:44','00:46:33']}

songs_frame = pd.DataFrame(songs)
```

```
songs = {'Album': ['Thriller','Back in Black', 'The Dark Side of the Moon',\ 'The Bodyguard','Bat Out of Hell'],
         'Released': [1982,1980,1973,1992,1977],
         'Length': ['00:42:19','00:42:11','00:42:49','00:57:44','00:46:33']}
```

|   | Album                     | Length   | Released |
|---|---------------------------|----------|----------|
| 0 | Thriller                  | 00:42:19 | 1982     |
| 1 | Back in Black             | 00:42:11 | 1980     |
| 2 | The Dark Side of the Moon | 00:42:49 | 1973     |
| 3 | The Bodyguard             | 00:57:44 | 1992     |
| 4 | Bat Out of Hell           | 00:46:33 | 1977     |

```
songs = {'Album' : ['Thriller','Back in Black', 'The Dark Side of the Moon',\ 'The Bodyguard','Bat Out of Hell'],
         'Released': [1982,1980,1973,1992,1977],
         'Length': ['00:42:19','00:42:11','00:42:49','00:57:44','00:46:33']}
```

|   | Album                     | Length   | Released |
|---|---------------------------|----------|----------|
| 0 | Thriller                  | 00:42:19 | 1982     |
| 1 | Back in Black             | 00:42:11 | 1980     |
| 2 | The Dark Side of the Moon | 00:42:49 | 1973     |
| 3 | The Bodyguard             | 00:57:44 | 1992     |
| 4 | Bat Out of Hell           | 00:46:33 | 1977     |

### Dataframe column selection

Create a new dataframe consisting of one column. Enclose the dataframe name, in this case df, and the name of the column header in double brackets. The result is a **new dataframe comprised of the original column**.



```
x=df[ ['Length'] ]
```

|   | Artist          | Album                           | Released | Length  | Genre                       | Music Recording Sales (millions) | Claimed Sales (millions) | Released.1 | Soundtrack | Rating |   | Length  |
|---|-----------------|---------------------------------|----------|---------|-----------------------------|----------------------------------|--------------------------|------------|------------|--------|---|---------|
| 0 | Michael Jackson | Thriller                        | 1982     | 0:42:19 | pop, rock, R&B              | 46.0                             | 65                       | 30-Nov-82  | NaN        | 10.0   | 0 | 0:42:19 |
| 1 | AC/DC           | Back in Black                   | 1980     | 0:42:11 | hard rock                   | 26.1                             | 50                       | 25-Jul-80  | NaN        | 9.5    | 1 | 0:42:11 |
| 2 | Pink Floyd      | The Dark Side of the Moon       | 1973     | 0:42:49 | progressive rock            | 24.2                             | 45                       | 01-Mar-73  | NaN        | 9.0    | 2 | 0:42:49 |
| 3 | Whitney Houston | The Bodyguard                   | 1992     | 0:57:44 | R&B, soul, pop              | 27.4                             | 44                       | 17-Nov-92  | Y          | 8.5    | 3 | 0:57:44 |
| 4 | Meat Loaf       | Bat Out of Hell                 | 1977     | 0:46:33 | hard rock, progressive rock | 20.6                             | 43                       | 21-Oct-77  | NaN        | 8.0    | 4 | 0:46:33 |
| 5 | Eagles          | Their Greatest Hits (1971-1975) | 1976     | 0:43:08 | rock, soft rock, folk rock  | 32.2                             | 42                       | 17-Feb-76  | NaN        | 7.5    | 5 | 0:43:08 |
| 6 | Bee Gees        | Saturday Night Fever            | 1977     | 1:15:54 | disco                       | 20.6                             | 40                       | 15-Nov-77  | Y          | 7.0    | 6 | 1:15:54 |
| 7 | Fleetwood Mac   | Rumours                         | 1977     | 0:40:01 | soft rock                   | 27.9                             | 40                       | 04-Feb-77  | NaN        | 6.5    | 7 | 0:40:01 |

You can do the same thing for **multiple columns**.

```
y=df[ ['Artist', 'Length', 'Genre ' ] ]
```

|   | Artist          | Album                           | Released | Length  | Genre                       | Music Recording Sales (millions) | Claimed Sales (millions) | Released.1 | Soundtrack | Rating |   | Artist          | Length  | Genre                       |
|---|-----------------|---------------------------------|----------|---------|-----------------------------|----------------------------------|--------------------------|------------|------------|--------|---|-----------------|---------|-----------------------------|
| 0 | Michael Jackson | Thriller                        | 1982     | 0:42:19 | pop, rock, R&B              | 46.0                             | 65                       | 30-Nov-82  | NaN        | 10.0   | 0 | Michael Jackson | 0:42:19 | pop, rock, R&B              |
| 1 | AC/DC           | Back in Black                   | 1980     | 0:42:11 | hard rock                   | 26.1                             | 50                       | 25-Jul-80  | NaN        | 9.5    | 1 | AC/DC           | 0:42:11 | hard rock                   |
| 2 | Pink Floyd      | The Dark Side of the Moon       | 1973     | 0:42:49 | progressive rock            | 24.2                             | 45                       | 01-Mar-73  | NaN        | 9.0    | 2 | Pink Floyd      | 0:42:49 | progressive rock            |
| 3 | Whitney Houston | The Bodyguard                   | 1992     | 0:57:44 | R&B, soul, pop              | 27.4                             | 44                       | 17-Nov-92  | Y          | 8.5    | 3 | Whitney Houston | 0:57:44 | R&B, soul, pop              |
| 4 | Meat Loaf       | Bat Out of Hell                 | 1977     | 0:46:33 | hard rock, progressive rock | 20.6                             | 43                       | 21-Oct-77  | NaN        | 8.0    | 4 | Meat Loaf       | 0:46:33 | hard rock, progressive rock |
| 5 | Eagles          | Their Greatest Hits (1971-1975) | 1976     | 0:43:08 | rock, soft rock, folk rock  | 32.2                             | 42                       | 17-Feb-76  | NaN        | 7.5    | 5 | Eagles          | 0:43:08 | rock, soft rock, folk rock  |
| 6 | Bee Gees        | Saturday Night Fever            | 1977     | 1:15:54 | disco                       | 20.6                             | 40                       | 15-Nov-77  | Y          | 7.0    | 6 | Bee Gees        | 1:15:54 | disco                       |
| 7 | Fleetwood Mac   | Rumours                         | 1977     | 0:40:01 | soft rock                   | 27.9                             | 40                       | 04-Feb-77  | NaN        | 6.5    | 7 | Fleetwood Mac   | 0:40:01 | soft rock                   |

Additionally, we can extract column from a Dataframe to be a **Series** with a slightly different command `x=df['Length']`. Pay attention there is only one pair of square brackets

Individual element access: `iloc()` and `loc()`

One way to access unique elements is with the `iloc` method. The **first input** is an integer representing the **row index**, and the **second** is the integer representing the **column index**.



|   | Artist          | Album                           | Released | Length  | Genre                       | Music Recording Sales (millions) | Claimed Sales (millions) | Released.1 | Soundtrack | Rating |
|---|-----------------|---------------------------------|----------|---------|-----------------------------|----------------------------------|--------------------------|------------|------------|--------|
| 0 | Michael Jackson | Thriller                        | 1982     | 0:42:19 | pop, rock, R&B              | 46.0                             | 65                       | 30-Nov-82  | NaN        | 10.0   |
| 1 | AC/DC           | Back in Black                   | 1980     | 0:42:11 | hard rock                   | 26.1                             | 50                       | 25-Jul-80  | NaN        | 9.5    |
| 2 | Pink Floyd      | The Dark Side of the Moon       | 1973     | 0:42:49 | progressive rock            | 24.2                             | 45                       | 01-Mar-73  | NaN        | 9.0    |
| 3 | Whitney Houston | The Bodyguard                   | 1992     | 0:57:44 | R&B, soul, pop              | 27.4                             | 44                       | 17-Nov-92  | Y          | 8.5    |
| 4 | Meat Loaf       | Bat Out of Hell                 | 1977     | 0:46:33 | hard rock, progressive rock | 20.6                             | 43                       | 21-Oct-77  | NaN        | 8.0    |
| 5 | Eagles          | Their Greatest Hits (1971-1975) | 1976     | 0:43:08 | rock, soft rock, folk rock  | 32.2                             | 42                       | 17-Feb-76  | NaN        | 7.5    |
| 6 | Bee Gees        | Saturday Night Fever            | 1977     | 1:15:54 | disco                       | 20.6                             | 40                       | 15-Nov-77  | Y          | 7.0    |
| 7 | Fleetwood Mac   | Rumours                         | 1977     | 0:40:01 | soft rock                   | 27.9                             | 40                       | 04-Feb-77  | NaN        | 6.5    |

```
df.iloc[0,0]:'Michael Jackson'
```

```
df.iloc[1,0]:'AC/DC'
```

```
df.iloc[0,2]:1982
```

```
df.iloc[1,2]:1980
```

You can use the name of the row index and the column as well.

|   | Artist          | Album                           | Released | Length  | Genre                       | Music Recording Sales (millions) | Claimed Sales (millions) | Released.1 | Soundtrack | Rating |
|---|-----------------|---------------------------------|----------|---------|-----------------------------|----------------------------------|--------------------------|------------|------------|--------|
| 0 | Michael Jackson | Thriller                        | 1982     | 0:42:19 | pop, rock, R&B              | 46.0                             | 65                       | 30-Nov-82  | NaN        | 10.0   |
| 1 | AC/DC           | Back in Black                   | 1980     | 0:42:11 | hard rock                   | 26.1                             | 50                       | 25-Jul-80  | NaN        | 9.5    |
| 2 | Pink Floyd      | The Dark Side of the Moon       | 1973     | 0:42:49 | progressive rock            | 24.2                             | 45                       | 01-Mar-73  | NaN        | 9.0    |
| 3 | Whitney Houston | The Bodyguard                   | 1992     | 0:57:44 | R&B, soul, pop              | 27.4                             | 44                       | 17-Nov-92  | Y          | 8.5    |
| 4 | Meat Loaf       | Bat Out of Hell                 | 1977     | 0:46:33 | hard rock, progressive rock | 20.6                             | 43                       | 21-Oct-77  | NaN        | 8.0    |
| 5 | Eagles          | Their Greatest Hits (1971-1975) | 1976     | 0:43:08 | rock, soft rock, folk rock  | 32.2                             | 42                       | 17-Feb-76  | NaN        | 7.5    |
| 6 | Bee Gees        | Saturday Night Fever            | 1977     | 1:15:54 | disco                       | 20.6                             | 40                       | 15-Nov-77  | Y          | 7.0    |
| 7 | Fleetwood Mac   | Rumours                         | 1977     | 0:40:01 | soft rock                   | 27.9                             | 40                       | 04-Feb-77  | NaN        | 6.5    |

```
df.loc[0, 'Artist']:'Michael Jackson'
```

```
df.loc[1, 'Artist']:'AC/DC'
```

```
df.loc[0, 'Released']:'1982'
```

```
df.loc[1, 'Released']:'1980'
```

You can access the first row of the column named artist as follows. Similarly, you can access the second row of the column named artist. You can do the same for the column released.

**Loc() can also be used if the index is not an integer.**

```
df_new=df
df_new.index=['a','b','c','d','e','f','g','h']
```

```
df.loc['a', 'Artist']:'Michael Jackson'
```

```
df.loc['b', 'Artist']:'AC/DC'
```

```
df.loc['a', 'Released']:'1982'
```

```
df.loc['b', 'Released']:'1980'
```

|   | Artist          | Album                           | Released | Length   | Genre                       | Music Recording Sales (millions) | Claimed Sales (millions) | Released.1 | Soundtrack | Rating |
|---|-----------------|---------------------------------|----------|----------|-----------------------------|----------------------------------|--------------------------|------------|------------|--------|
| a | Michael Jackson | Thriller                        | 1982     | 00:42:19 | pop, rock, R&B              | 46.0                             | 65                       | 1982-11-30 | NaN        | 10.0   |
| b | AC/DC           | Back in Black                   | 1980     | 00:42:11 | hard rock                   | 26.1                             | 50                       | 1980-07-25 | NaN        | 9.5    |
| c | Pink Floyd      | The Dark Side of the Moon       | 1973     | 00:42:49 | progressive rock            | 24.2                             | 45                       | 1973-03-01 | NaN        | 9.0    |
| d | Whitney Houston | The Bodyguard                   | 1992     | 00:57:44 | R&B, soul, pop              | 27.4                             | 44                       | 1992-11-17 | Y          | 8.5    |
| e | Meat Loaf       | Bat Out of Hell                 | 1977     | 00:46:33 | hard rock, progressive rock | 20.6                             | 43                       | 1977-10-21 | NaN        | 8.0    |
| f | Eagles          | Their Greatest Hits (1971-1975) | 1976     | 00:43:08 | rock, soft rock, folk rock  | 32.2                             | 42                       | 1976-02-17 | NaN        | 7.5    |
| g | Bee Gees        | Saturday Night Fever            | 1977     | 01:15:54 | disco                       | 20.6                             | 40                       | 1977-11-15 | Y          | 7.0    |
| h | Fleetwood Mac   | Rumours                         | 1977     | 00:40:01 | soft rock                   | 27.9                             | 40                       | 1977-02-04 | NaN        | 6.5    |

We create a new dataframe called df\_new. We replace the index 1, 2, 3 and so on with a, b, c. You can access the index a, that is the first row of the column named artist as follows.

## Dataframe Slicing

You can also **slice dataframes and assign the values to a new dataframe.**

```
z=df.iloc[0:2, 0:3]
```

|   | Artist          | Album                           | Released | Length  | Genre                       | Music Recording Sales (millions) | Claimed Sales (millions) | Released.1 | Soundtrack | Rating |
|---|-----------------|---------------------------------|----------|---------|-----------------------------|----------------------------------|--------------------------|------------|------------|--------|
| 0 | Michael Jackson | Thriller                        | 1982     | 0:42:19 | pop, rock, R&B              | 46.0                             | 65                       | 30-Nov-82  | NaN        | 10.0   |
| 1 | AC/DC           | Back in Black                   | 1980     | 0:42:11 | hard rock                   | 26.1                             | 50                       | 25-Jul-80  | NaN        | 9.5    |
| 2 | Pink Floyd      | The Dark Side of the Moon       | 1973     | 0:42:49 | progressive rock            | 24.2                             | 45                       | 01-Mar-73  | NaN        | 9.0    |
| 3 | Whitney Houston | The Bodyguard                   | 1992     | 0:57:44 | R&B, soul, pop              | 27.4                             | 44                       | 17-Nov-92  | Y          | 8.5    |
| 4 | Meat Loaf       | Bat Out of Hell                 | 1977     | 0:46:33 | hard rock, progressive rock | 20.6                             | 43                       | 21-Oct-77  | NaN        | 8.0    |
| 5 | Eagles          | Their Greatest Hits (1971-1975) | 1976     | 0:43:08 | rock, soft rock, folk rock  | 32.2                             | 42                       | 17-Feb-76  | NaN        | 7.5    |
| 6 | Bee Gees        | Saturday Night Fever            | 1977     | 1:15:54 | disco                       | 20.6                             | 40                       | 15-Nov-77  | Y          | 7.0    |
| 7 | Fleetwood Mac   | Rumours                         | 1977     | 0:40:01 | soft rock                   | 27.9                             | 40                       | 04-Feb-77  | NaN        | 6.5    |

Z

|   | Artist          | Album                     | Released |
|---|-----------------|---------------------------|----------|
| 0 | Michael Jackson | Thriller                  | 1982     |
| 1 | AC/DC           | Back in Black             | 1980     |
| 2 | Pink Floyd      | The Dark Side of the Moon | 1973     |

Assign the first two rows and the first three columns to the variable z. The result is a dataframe comprised of the selected rows and columns.

You can also slice dataframes and assign the values to a new dataframe using loc.

```
z=df.loc[0:2, 'Artist':'Released']
```

|   | Artist          | Album                           | Released | Length  | Genre                       | Music Recording Sales (millions) | Claimed Sales (millions) | Released.1 | Soundtrack | Rating |
|---|-----------------|---------------------------------|----------|---------|-----------------------------|----------------------------------|--------------------------|------------|------------|--------|
| 0 | Michael Jackson | Thriller                        | 1982     | 0:42:19 | pop, rock, R&B              | 46.0                             | 65                       | 30-Nov-82  | NaN        | 10.0   |
| 1 | AC/DC           | Back in Black                   | 1980     | 0:42:11 | hard rock                   | 26.1                             | 50                       | 25-Jul-80  | NaN        | 9.5    |
| 2 | Pink Floyd      | The Dark Side of the Moon       | 1973     | 0:42:49 | progressive rock            | 24.2                             | 45                       | 01-Mar-73  | NaN        | 9.0    |
| 3 | Whitney Houston | The Bodyguard                   | 1992     | 0:57:44 | R&B, soul, pop              | 27.4                             | 44                       | 17-Nov-92  | Y          | 8.5    |
| 4 | Meat Loaf       | Bat Out of Hell                 | 1977     | 0:46:33 | hard rock, progressive rock | 20.6                             | 43                       | 21-Oct-77  | NaN        | 8.0    |
| 5 | Eagles          | Their Greatest Hits (1971-1975) | 1976     | 0:43:08 | rock, soft rock, folk rock  | 32.2                             | 42                       | 17-Feb-76  | NaN        | 7.5    |
| 6 | Bee Gees        | Saturday Night Fever            | 1977     | 1:15:54 | disco                       | 20.6                             | 40                       | 15-Nov-77  | Y          | 7.0    |
| 7 | Fleetwood Mac   | Rumours                         | 1977     | 0:40:01 | soft rock                   | 27.9                             | 40                       | 04-Feb-77  | NaN        | 6.5    |

Z

|   | Artist          | Album                     | Released |
|---|-----------------|---------------------------|----------|
| 0 | Michael Jackson | Thriller                  | 1982     |
| 1 | AC/DC           | Back in Black             | 1980     |
| 2 | Pink Floyd      | The Dark Side of the Moon | 1973     |

## Unique values

|   | Released |
|---|----------|
| 0 | 1982     |
| 1 | 1980     |
| 2 | 1973     |
| 3 | 1992     |
| 4 | 1977     |
| 5 | 1976     |
| 6 | 1977     |
| 7 | 1977     |

`df['Released'].unique()`

|      |
|------|
| 1982 |
| 1980 |
| 1973 |
| 1992 |
| 1977 |
| 1976 |

## Filtering rows

Filter songs from the 1980s and after. We can look at the column Released for songs made after 1979, then select the corresponding rows.

We can use the **inequality operators** for the entire data frame in Pandas. The result is a series of Boolean values. For our case, we simply specify the column Released and the inequality for the albums after 1979. The result is true when the condition is true and false otherwise.

`df['Released'] > 1979`

|   | Artist          | Album                           | Released | Length  | Genre                       | Music Recording Sales (millions) | Claimed Sales (millions) | Released.1 | Soundtrack | Rating |
|---|-----------------|---------------------------------|----------|---------|-----------------------------|----------------------------------|--------------------------|------------|------------|--------|
| 0 | Michael Jackson | Thriller                        | 1982     | 0:42:19 | pop, rock, R&B              | 46.0                             | 65                       | 30-Nov-82  | NaN        | 10.0   |
| 1 | AC/DC           | Back in Black                   | 1980     | 0:42:11 | hard rock                   | 26.1                             | 50                       | 25-Jul-80  | NaN        | 9.5    |
| 2 | Pink Floyd      | The Dark Side of the Moon       | 1973     | 0:42:49 | progressive rock            | 24.2                             | 45                       | 01-Mar-73  | NaN        | 9.0    |
| 3 | Whitney Houston | The Bodyguard                   | 1992     | 0:57:44 | R&B, soul, pop              | 27.4                             | 44                       | 17-Nov-92  | Y          | 8.5    |
| 4 | Meat Loaf       | Bat Out of Hell                 | 1977     | 0:46:33 | hard rock, progressive rock | 20.6                             | 43                       | 21-Oct-77  | NaN        | 8.0    |
| 5 | Eagles          | Their Greatest Hits (1971-1975) | 1976     | 0:43:08 | rock, soft rock, folk rock  | 32.2                             | 42                       | 17-Feb-76  | NaN        | 7.5    |
| 6 | Bee Gees        | Saturday Night Fever            | 1977     | 1:15:54 | disco                       | 20.6                             | 40                       | 15-Nov-77  | Y          | 7.0    |
| 7 | Fleetwood Mac   | Rumours                         | 1977     | 0:40:01 | soft rock                   | 27.9                             | 40                       | 04-Feb-77  | NaN        | 6.5    |

|   |       |
|---|-------|
| 0 | True  |
| 1 | True  |
| 2 | False |
| 3 | True  |
| 4 | False |
| 5 | False |
| 6 | False |
| 7 | False |

We can select the specified columns in one line. We simply use the data frames names and square brackets we placed the previously mentioned inequality and assign it to the variable df1. We now have a new data frame, where each album was released after 1979.

```
df1=df[df['Released']>1979]
```

|   | Artist          | Album         | Released | Length   | Genre          | Music Recording Sales (millions) | Claimed Sales (millions) | Released.1 | Soundtrack | Rating |
|---|-----------------|---------------|----------|----------|----------------|----------------------------------|--------------------------|------------|------------|--------|
| 0 | Michael Jackson | Thriller      | 1982     | 00:42:19 | pop, rock, R&B | 46.0                             | 65                       | 1982-11-30 | NaN        | 10.0   |
| 1 | AC/DC           | Back in Black | 1980     | 00:42:11 | hard rock      | 26.1                             | 50                       | 1980-07-25 | NaN        | 9.5    |
| 3 | Whitney Houston | The Bodyguard | 1992     | 00:57:44 | R&B, soul, pop | 27.4                             | 44                       | 1992-11-17 | Y          | 8.5    |

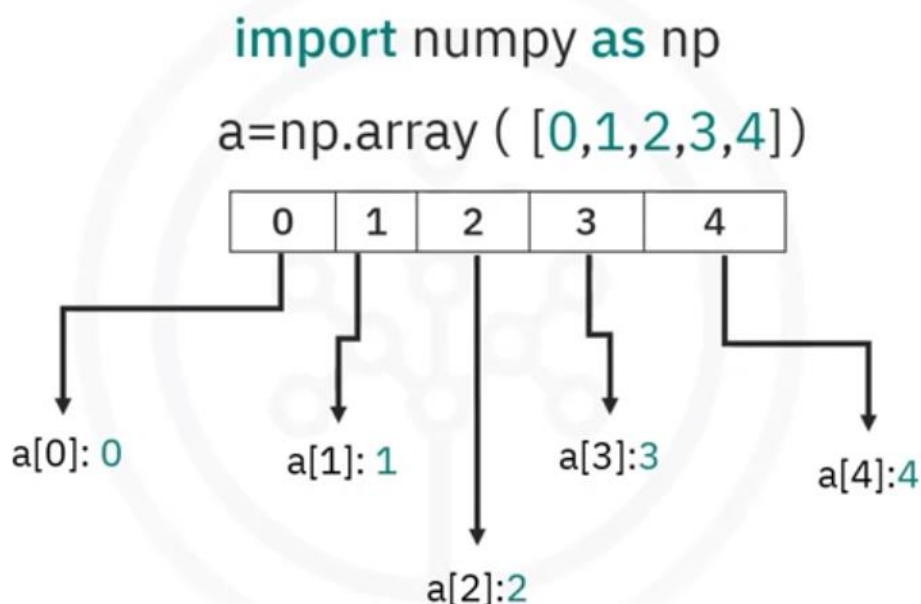
We can save the new data frame using the method `to_csv()`. Make sure you include a `.csv` extension. There are other functions to save the data frame in other formats.

## Numpy

NumPy stands for **Numerical Python** and it is an **open source project**. The array object in NumPy is called **ndarray**, it provides a lot of supporting functions that make working with ndarray very easy.

Numpy is a library for **scientific computing** (speed and memory). Numpy is also the **basis for pandas**.

A Numpy array or **ND array** is similar to a list. It's usually **fixed in size** and **each element is of the same type**, in this case integers.



We can cast a list to a Numpy array by first importing Numpy. We then cast the list as follows; we can access the data via an index. As with the list, we can access each element with an integer and a square bracket.

The value of a is stored as follows.

```
a=array([0,1,2,3, 4])      b=np.array([3.1,11.02,6.2, 213.2,5.2])
type(a): numpy.ndarray    type(b): numpy.ndarray
a.dtype:dtype('int64')    b.dtype:dtype('float64')
```

If we check the **type** of the array we get, numpy.ndarray. As Numpy arrays contain data of the same type, we can use the attribute **dtype** to obtain the data type of the array's elements. In this case a 64-bit integer.

```
a=np.array( [0,1,2,3,4] )
```

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|

```
a.size: 5
a.ndim: 1
a.shape: (5,)
```

Let's review some basic array attributes using the array a. The attribute **size** is the number of elements in the array. As there are five elements the result is five. The attribute **ndim** represents the number of array dimensions or the rank of the array. The attribute **shape** is a tuple of integers indicating the size of the array in each dimension.

## Indexing and Slicing

We can **change the first element of the array** to 100 as follows. The array's first value is now 100. We can change the fifth element of the array as follows. The fifth element is now zero.

```

c=np.array([20,1,2,3, 4])
c:array([20,1,2,3, 4])

c[0]=100
c:array([100,1,2,3,4])

c[4]=0
c:array([100,1,2,3,0])

```

Like lists and tuples we can slice a Numpy array. The elements of the array correspond to the following index. We can select the elements from one to three and assign it to a new Numpy array d as follows. The elements in d correspond to the index. Like lists, **we do not count the element corresponding to the last index.**

```

c:array([100,1,2,3,0])

```

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

```

d=c[1:4]
d:array([1, 2, 3])

```

```

c:array([100,1,2,3,0])
c[3:5]=300,400
c:array([100, 1, 2, 300, 400])

```

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

We can assign the corresponding indices to new values as follows. The array c now has new values.

We can also define the steps in slicing, like this: [start:end:step].

```

[16]: arr = np.array([1, 2, 3, 4, 5, 6, 7])
      print(arr[1:5:2])
      [2 4]

```

If we don't pass start its considered 0. If we don't pass end it considers till the length of array. If we don't pass step its considered 1

```
select = [0, 2, 3, 4]
```

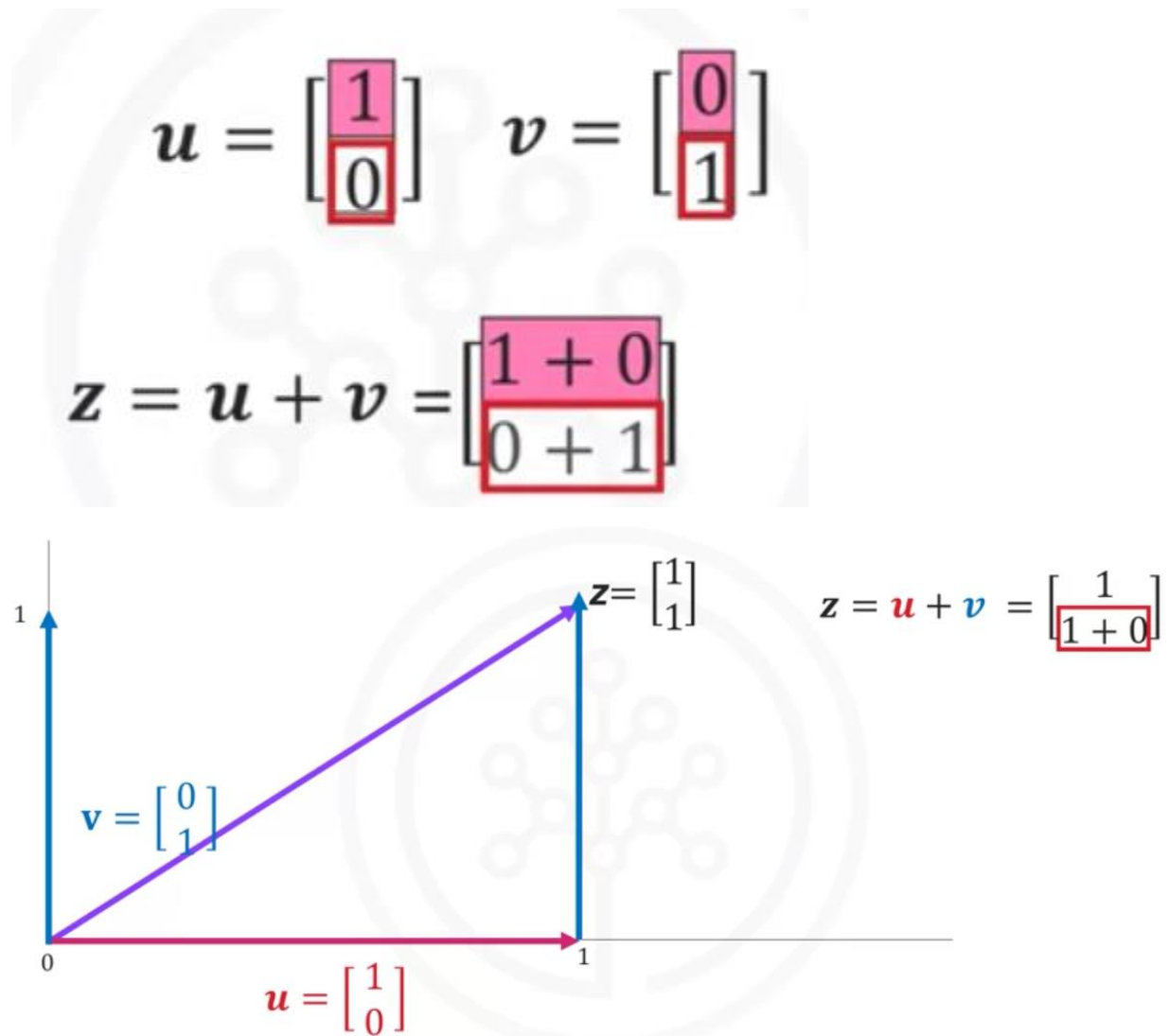
```
d = arr[select]
```

```
array([1, 3, 4, 5])
```

## Basic Operations

Numpy makes it easier to do many operations that are commonly performed in data science. The **same operations are usually computationally faster and require less memory in Numpy compared to regular Python**. We will look at many of the operations in the context of **Euclidian vectors** to make things more interesting.

### Vector Addition





Representing vector addition with line segment or arrows is helpful. The first vector is represented in red. The vector will point in the direction of the two components. The first component of the vector is one. As a result the arrow is offset one unit from the origin in the horizontal direction. The second component is zero, we represent this component in the vertical direction. As this component is zero, the vector does not point in the vertical direction. We represent the second vector in blue. The first component is zero, therefore the arrow does not point to the horizontal direction. The second component is one. As a result the vector points in the vertical direction one unit. When we add the vector  $u$  and  $v$ , we get the new vector  $z$ . We add the first component, this corresponds to the horizontal direction. We also add the second component. **It's helpful to use the tip to tail method when adding vectors, placing the tail of the vector  $v$  on the tip of vector  $u$ .** The new vector  $z$  is constructed by connecting the base of the first vector  $u$  with the tail of the second  $v$ . The following three lines of code will add the two lists and place the result in the list  $z$ .

We can also perform vector addition with **one line of Numpy code**. It would require multiple lines to perform vector addition on two lists as shown on the right side of the screen.

```
u=np.array([1,0])
v=np.array([0,1])

z=u+v
z:array([1, 1])
```

```
u=[1,0]
v=[0,1]
z=[ ]
for n, m in zip(u,v):
    z.append(n+m)
```

In addition, the Numpy code will run much faster. This is important if you have lots of data.

#### *Vector subtraction*

We can also perform vector subtraction by changing the addition sign to a subtraction sign.

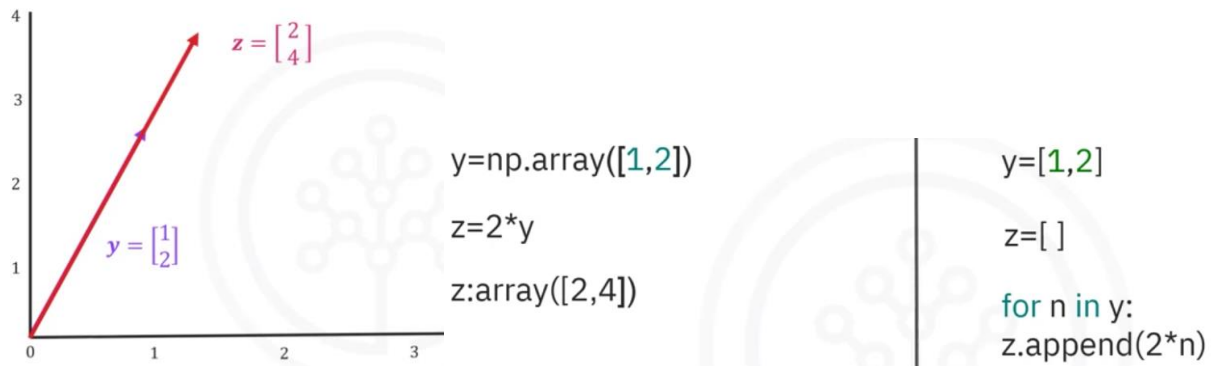
#### *Vector multiplication with a scalar*

$$y = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$z = 2y = \begin{bmatrix} 2(1) \\ 2(2) \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$



Vector multiplication with a scalar is another commonly performed operation. Consider the vector  $y$ , each component is specified by a different color. We simply multiply the vector by a scalar value, in this case two. Each component of the vector is multiplied by two, in this case each component is doubled.



Vector multiplication with a scalar **only requires one line of code using Numpy**. It would require multiple lines to perform the same task as shown with Python lists as shown on the right side of the screen. In addition, the operation would also be much slower.

### Hadamard product

$$u = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad v = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$$

$$z = u \circ v = \begin{bmatrix} 1 \times 3 \\ 2 \times 2 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

| Method       | Code                                                                    |
|--------------|-------------------------------------------------------------------------|
| Numpy        | <pre>u=np.array([1,2]) v=np.array([3,2]) z=u*v z:array([3, 4])</pre>    |
| Python Lists | <pre>u=[1,2] v=[3,2] z=[] for n, m in zip(u,v):     z.append(n*m)</pre> |

Consider the following two vectors,  $u$  and  $v$ . The Hadamard product of  $u$  and  $v$  is a new vector  $z$ . The first component of  $z$  is the product of the first element of  $u$  and  $v$ . Similarly, the second component is the product of the second element of  $u$  and  $v$ . The resultant vector consists of the entry wise product of  $u$  and  $v$ . We can also perform Hadamard product with one line of code in Numpy. It would require multiple lines to perform Hadamard product on two lists as shown on the right side of the screen.

### The dot Product

$$\mathbf{u} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \mathbf{v} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$
$$\mathbf{u}^T \mathbf{v} = 1 \times 3 + 2 \times 1 = 5$$

Consider the vector  $\mathbf{u}$  and  $\mathbf{v}$ , the dot product is a single number given by the following term and represents **how similar two vectors are**. We multiply the first component from  $\mathbf{v}$  and  $\mathbf{u}$ , we then multiply the second component and add the result together. The result is a number that represents how similar the two vectors are.

```
u=np.array([1,2])
v=np.array([3,1])
result=np.dot(u,v)
result:5
```

We can also perform dot product using the Numpy function `dot` and assign it with the variable `result` as follows.

### Scalar addition

```
u=np.array([1,2,3,-1])
z=u+1
z:array([2,3,4,0])
```

1, 2, 3, -1  
↓  
1+1, 2+1, 3+1, -1+1  
↓

Consider the array  $\mathbf{u}$ , the array contains the following elements. If we add a scalar value to the array, Numpy will add that value to each element. This property is known as **broadcasting**.

### Universal Functions

A universal function is a function that operates on ND arrays.

Mean(), max(), sin()

|                                              |                                                 |
|----------------------------------------------|-------------------------------------------------|
| <code>np.pi</code>                           | $\pi$                                           |
| <code>x=np.array([0, np.pi/2, np.pi])</code> | $x = [0, \frac{\pi}{2}, \pi]$                   |
| <code>y=np.sin(x)</code>                     | $y = [\sin(0), \sin(\frac{\pi}{2}), \sin(\pi)]$ |
| <code>y:array([ 0,1, 1.2e-16])</code>        | $y = [0,1,0]$                                   |

A useful function for plotting mathematical functions is linspace().



Line space returns evenly spaced numbers over a specified interval. We specify the starting point of the sequence, the ending point of the sequence. The parameter num indicates the number of samples to generate, in this case five. The space between samples is one. If we change the parameter num to nine, we get **nine evenly spaced numbers** over the interval from negative two to two. The result is the difference between subsequent samples is 0.5 as opposed to one as before.

### Plotting

We can use the function line space to generate 100 evenly spaced samples from the interval zero to two pi. We can use the Numpy function sine to map the array x to a new array y. We can import the library pyplot as plt to help us plot the function.

As we are using a Jupiter notebook, we use the **command matplotlib inline** to display the plot.

The following command plots a graph. The first input corresponds to the values for the horizontal or x-axis. The second input corresponds to the values for the vertical or y-axis.

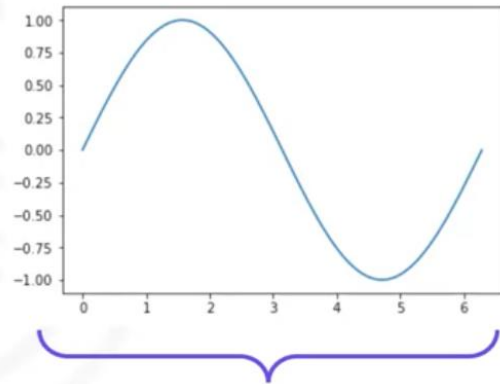
```
x=np.linspace( 0 , 2*np.pi,100)
```

```
y=np.sin(x)
```

```
import matplotlib.pyplot as plt
```

```
%matplotlib inline
```

```
plt.plot(x,y)
```



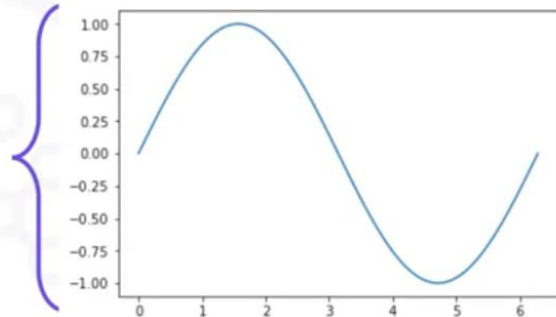
```
x=np.linspace( 0 , 2*np.pi,100)
```

```
y=np.sin(x)
```

```
import matplotlib.pyplot as plt
```

```
%matplotlib inline
```

```
plt.plot(x,y)
```



## Two Dimensional Numpy Arrays

We can create Numpy arrays with more than one dimension.

Consider the list `a`, the list contains three nested lists each of equal size. We can cast the list to a Numpy array as follows.

It is helpful to visualize the Numpy array as a rectangular array **each nested lists corresponds to a different row of the matrix.**

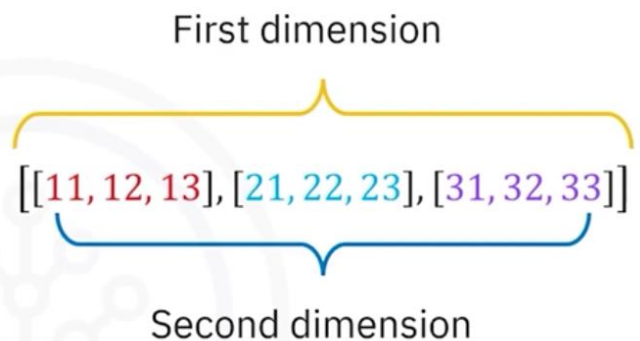
```
a = [[11, 12, 13], [21, 22, 23], [31, 32, 33]]
```

```
A = np.array(a)
```

```
A:  $\begin{bmatrix} 11 & 12 & 13 \\ 21 & 22 & 23 \\ 31 & 32 & 33 \end{bmatrix}$ 
```

We can use the attribute **ndim** to obtain the number of axes or dimensions referred to as the **rank**. The term rank **does not refer to the number of linearly independent columns like a matrix**. It's useful to think of **ndim as the number of nested lists**. The first list represents the first dimension. This list contains another set of lists. This represents the second dimension or axis. The number of lists the list contains does not have to do with the dimension but the shape of the list.

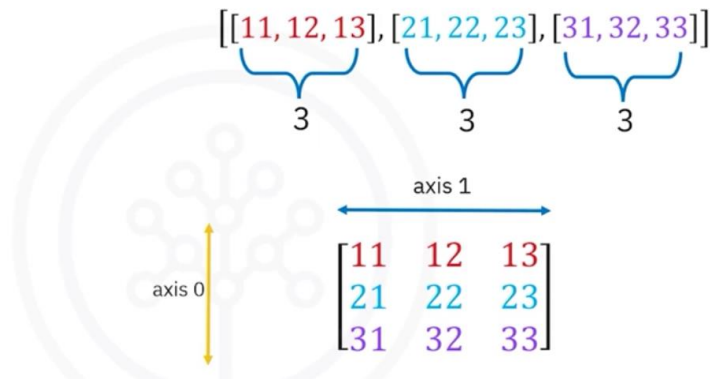
```
A.ndim:2
```



The attribute **shape** returns a tuple. It's helpful to use the rectangular representation as well. The **first element** in the tuple corresponds to the **number of nested lists** contained in the original list or the **number of rows** in the rectangular representation, in this case three. The second element corresponds to the size of each of the nested list or the number of columns in the rectangular array zero. The convention is to label this axis zero and this axis one as follows.

A.ndim:2

A.shape: (3,3)



We can also use the **attribute size to get the size of the array**. We see there are three rows and three columns. Multiplying the number of columns and rows together, we get the total number of elements, in this case **nine**.

We can use **rectangular brackets to access the different elements of the array**.

A:  $[A[0][0], A[0][1], A[0][2], A[1][0], A[1][1], A[1][2], A[2][0], A[2][1], A[2][2]]$



The following image demonstrates the relationship between the indexing conventions for the lists like representation. The index in the first bracket corresponds to the different nested lists each a different color. The second bracket corresponds to the index of a particular element within the nested list. Using the rectangular representation, the **first index** corresponds to the **row index**. The **second index** corresponds to the **column index**.

We could also use **a single bracket to access the elements** as follows.

A:  $\left[ \left[ A[0,0], A[0,1], A[0,2] \right], \left[ A[1,0], A[1,1], A[1,2] \right], \left[ A[2,0], A[2,1], A[2,2] \right] \right]$

$$\begin{bmatrix} A[0,0] & A[0,1] & A[0,2] \\ A[1,0] & A[1,1] & A[1,2] \\ A[2,0] & A[2,1] & A[2,2] \end{bmatrix}$$

Consider the following syntax. This index corresponds to the second row, and this index the third column, the value is 23.

$A = \left[ \left[ 11, 12, 13 \right], \left[ 21, 22, 23 \right], \left[ 31, 32, 33 \right] \right]$

$A[1][2] : 23$

|   | 0  | 1  | 2  |
|---|----|----|----|
| 0 | 11 | 12 | 13 |
| 1 | 21 | 22 | 23 |
| 2 | 31 | 32 | 33 |

We can also use **slicing** in Numpy arrays.

$A[0,0:2] : \text{array}([11, 12])$

|   | 0  | 1  | 2  |
|---|----|----|----|
| 0 | 11 | 12 | 13 |
| 1 | 21 | 22 | 23 |
| 2 | 31 | 32 | 33 |

The first index corresponds to the first row. The second index accesses the first two columns.

$$A = \begin{bmatrix} [11, 12, 13], [21, 22, 23], [31, 32, 33] \end{bmatrix}$$

`A[0:2,2]:array([13, 23])`

|   | 0  | 1  | 2  |
|---|----|----|----|
| 0 | 11 | 12 | 13 |
| 1 | 21 | 22 | 23 |
| 2 | 31 | 32 | 33 |

Consider this example, the first index corresponds to the first two rows. the second index accesses the last column.

We can also **add arrays**, the process is identical to matrix addition.

$$X = \begin{bmatrix} \begin{matrix} 1 & 0 \\ 0 & 1 \end{matrix} \end{bmatrix} \quad Y = \begin{bmatrix} \begin{matrix} 2 & 1 \\ 1 & 2 \end{matrix} \end{bmatrix}$$

$$X + Y = \begin{bmatrix} \begin{matrix} 1+2 & 0+1 \\ 0+1 & 1+2 \end{matrix} \end{bmatrix} = \begin{bmatrix} \begin{matrix} 3 & 1 \\ 1 & 3 \end{matrix} \end{bmatrix}$$

Consider the matrix X, each element is colored differently. Consider the matrix Y. Similarly, each element is colored differently. We can add the matrices. This corresponds to adding the elements in the same position, i.e adding elements contained in the same color boxes together. The result is a **new matrix that has the same size as matrix Y or X**. Each element in this new matrix is the sum of the corresponding elements in X and Y.



```
X=np.array([[1,0],[0,1]])
Y=np.array([[2,1],[1,2]])
Z=X+Y;
Z:array([[3,1],
        [1,3]])
```

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$Y = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

$$Z = X + Y$$

$$Z = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$$

To add two arrays in Numpy, we define the array in this case X. Then we define the second array Y, we add the arrays. The result is identical to matrix addition.

```
Y=np.array([[2,1],[1,2]])
```

$$Y = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

```
Z=2*Y;
```

$$Z = 2Y = \begin{bmatrix} (2)2 & (2)1 \\ (2)1 & (2)2 \end{bmatrix}$$

```
Z:array([[4,2],
        [2,4]])
```

$$Z = \begin{bmatrix} 4 & 2 \\ 2 & 4 \end{bmatrix}$$

**Multiplying a Numpy array by a scalar** is identical to multiplying a matrix by a scalar. Consider the matrix Y. If we multiply the matrix by this scalar two, we simply multiply every element in the matrix by two. The result is a new matrix of the same size where each element is multiplied by two. Consider the array Y. We first define the array, we multiply the array by a scalar as follows and assign it to the variable Z. The result is a new array where each element is multiplied by two.

```
X=np.array([[1,0],[0,1]])
```

```
Y=np.array([[2,1],[1,2]])
```

```
Z=X*Y;
```

```
Z:array([[2,0],
        [0,2]])
```

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$Y = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

$$Z = X \circ Y = \begin{bmatrix} (1)2 & (0)1 \\ (0)1 & (1)2 \end{bmatrix}$$

$$Z = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

**Multiplication of two arrays** corresponds to an element-wise product, or Hadamard product. Consider array X and array Y. Hadamard product corresponds to multiplying each of the elements in the same position i.e multiplying elements contained in the same color boxes together. The result is a new matrix that is the same size as matrix Y or X. Each element in this new matrix is the product of the corresponding elements in X and Y. Consider the array X and Y. We can find the product of two arrays X and Y in one line, and assign it to the variable Z as follows. The result is identical to Hadamard product.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix} \quad B = \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ -1 & 1 \end{bmatrix} \quad \left. \begin{matrix} 1 \\ 2 \\ 3 \end{matrix} \right\} 3$$

$A \quad B$

We can also perform **matrix multiplication** with Numpy arrays. Consider the matrix A consider the matrix B In linear algebra, before we multiply matrix A by matrix B, we must **make sure that the number of columns** in matrix A in this case three is **equal to the number of rows** in matrix B, in this case three.

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix} \quad B = \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ -1 & 1 \end{bmatrix}$$

$$0 \times 1 + 1 \times 1 - 1 \times 1 = 0$$

$$AB = \begin{bmatrix} 0 & \end{bmatrix}$$

From matrix multiplication, to obtain the  $i$ th row and  $j$ th column of the new matrix, we take the dot product of the  $i$ th row of A with the  $j$ th columns of B. For the first column, first row we take the dot product of the first row of A with the first column of B as follows. The result is zero.

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix} \quad B = \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ -1 & 1 \end{bmatrix}$$

$$0 \times 1 + 1 \times 1 + 1 \times 1 = 2$$

$$AB = \begin{bmatrix} 0 & 2 \end{bmatrix}$$

For the first row and the second column of the new matrix, we take the dot product of the first row of the matrix A, but this time we use the second column of matrix B, the result is two.

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ -1 & 1 \end{bmatrix}$$

$$1 \times 1 + 0 \times 1 + 1 \times (-1) = 0$$

$$AB = \begin{bmatrix} 0 & 2 \\ 0 & \end{bmatrix}$$

For the second row and the first column of the new matrix, we take the dot product of the second row of the matrix A. With the first column of matrix B, the result is zero.

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ -1 & 1 \end{bmatrix}$$

$$1 \times 1 + 0 \times 1 + 1 \times 1 = 2$$

$$AB = \begin{bmatrix} 0 & 2 \\ 0 & 2 \end{bmatrix}$$

Finally, for the second row and the second column of the new matrix, we take the dot product of the second row of the matrix A with the second column of matrix B, the result is two.

```
A=np.array([[0,1,1],[1,0,1]])  
B=np.array([[1,1],[1,1],[-1,1]])  
C=np.dot(A,B);  
C:array([[0,2],  
        [0,2]])
```

In Numpy, we can define the Numpy arrays A and B. We can perform matrix multiplication and assign it to array C. The result is the array C. It corresponds to the matrix multiplication of array A and B.

Check out [numpy.org](https://numpy.org)

## Summary.

- Python uses the `open()` function and allows you to read and write files, providing access to the content within the file for reading. It also allows overwriting it for writing and specifies the file mode (for example, `r` for reading, `w` for writing, `a` for appending).
  - To read a file, Python uses an `open` function along with `r`.
  - Python uses the **`open with`** function to read and process a file attribute, that is, from `open` to `close`.
  - In Python, you use the **`open`** method to edit or overwrite a file.
  - To write a file, Python uses the **`open`** function along with `w`.
  - In Python, `"a"` indicates that the program has appended to the file.
  - In Python, `"\n"` signifies that the code should start on a new line.
  - Python uses various methods to print lines from attributes.
- Pandas is a powerful Python library for data manipulation and analysis, providing data structures and functions to work with structured data like data frames and series.

- You import the file (panda) by using the import command followed by the file name.
- In Python, you use the **as** command to provide a shorter name for the file.
- In Pandas, you use a data frame (df) to specify the files to read.
- DataFrames consist of rows and columns.
- You can create new DataFrames by using the column or columns of a specific DataFrame.
- We can work with data in a DataFrames and save the results in different formats.
- In Python, you use the **Unique** method to determine unique elements in a column of the DataFrames.
- You use the inequality operator along with df to assign a Boolean value to the selected column in DataFrames.
- You save a new DataFrame as a different DataFrame, which may contain values from an earlier DataFrame.
- NumPy is a Python library for numerical and matrix operations, offering multidimensional array objects and a variety of mathematical functions to work with data efficiently.
  - NumPy is a basis for Pandas.
  - A NumPy array or ND array is similar to a list, usually of a fixed size with the same kind of element.
- A one-dimensional NumPy array is a linear sequence of elements with a single axis, like a traditional list, but optimized for numerical computations and array operations.
  - You can access elements in a NumPy using an index.
  - You use the attribute **dtype** to get the data type of the array elements.
  - You use **ndsize** and **ndim** to get the size and dimension of the array, respectively.
  - You can use indexing and slicing methods in NumPy.
  - Vector additions are widely used operations in Python.
  - Representing vector addition with line segments or arrows is useful.
  - NumPy codes work much faster, which is helpful with lots of data.
  - You perform vector subtraction by replacing the addition sign with a negative sign.
  - Multiplying an array by a scalar in Python entails multiplying each element of the array by the scalar value, leading to a new array in which each element scales by the scalar.

- Hadamard product refers to the element-wise multiplication of two arrays of the same shape, resulting in a new array where each element is the product of the corresponding elements in the input arrays.
- The dot product in Python is the sum of the element-wise products of two arrays, often used for vector and matrix operations to find the scalar result of multiplying corresponding elements and summing them.
- When working with NumPy, it is common to utilize libraries like Matplotlib to create graphs and visualizations from numerical data stored in NumPy arrays.
- A two-dimensional NumPy array is a grid-like structure with rows and columns suitable for representing data as a matrix or a table for numerical computations.
  - In NumPy, "shape" refers to an array's dimensions (number of rows and columns), indicating its size and structure.
  - You use the attribute "size" to obtain the size of an array.
  - You use rectangular attributes to access the various elements in an array.
  - You use a scalar to multiply elements in NumPy.

## Cheatsheet, Grades Quiz, Glossary

## Module 5: Apis and Data Collection

This module delves into the unique ways to collect data by the use of APIs and web scraping. It further explores data collection by explaining how to read and collect data when dealing with different file formats.

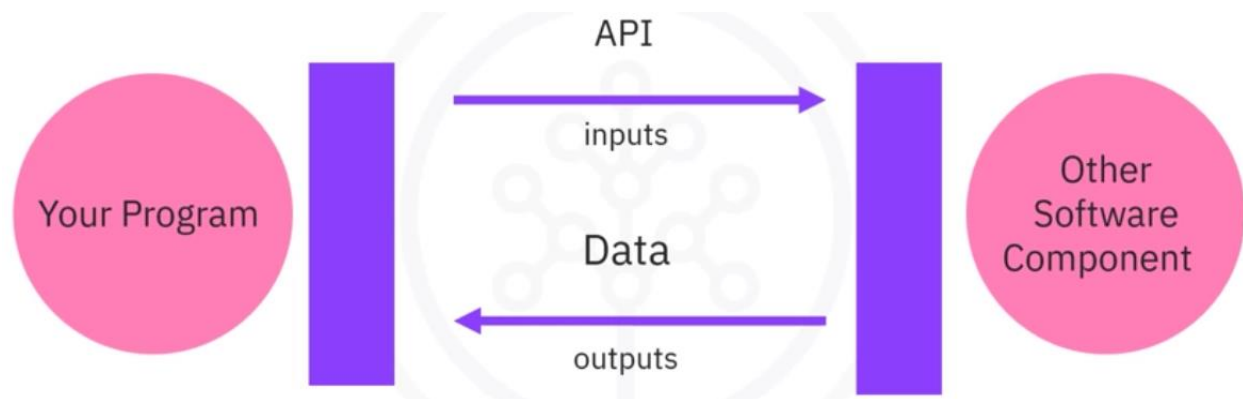
### Learning Objectives

- Explain the use of the HTTP protocol using the Requests Library method.
- Describe how the URL Request Response HTTP protocol works.
- Invoke simple, open-source APIs.
- Perform basic web scraping using Python.
- Work with different file formats using Python.

- Explain the difference between APIs and REST APIs.
- Summarize how APIs receive and send information.

## Simple APIs

An API **lets two pieces of software talk to each other**. For example, you have your program, you have some data, you have other software components. You use the API to communicate with other software **via inputs and outputs**. Just like a function, you **don't have to know how the API works**, just its inputs and outputs.



Pandas is a set of software components, much of which **are not even written in Python**. You have some data. You have a set of software components. We use the pandas API to process the data by communicating with the other software components. Let's clean up the diagram. When you create a dictionary and then create a panda's object with the DataFrame constructor, in API lingo, **this is an "instance."** The data in the dictionary is passed along to the pandas API. You then use the dataframe to communicate with the API. When you call the method **head**, **the dataframe communicates with the API displaying the first few rows of the dataframe**. When you call the method **mean** the API will calculate the mean and return the values.



```
import pandas as pd
```

```
dict_ = {'a':[11, 21, 31], 'b':[12, 22, 32]}
```

```
df = pd.DataFrame(dict_)
```

```
df.head()
```

|   | a  | b  |
|---|----|----|
| 0 | 11 | 12 |
| 1 | 21 | 22 |
| 2 | 31 | 32 |

```
df.mean()
```

|   |      |
|---|------|
| a | 21.0 |
| b | 22.0 |

dtype: float64

df

Your  
Python  
Program

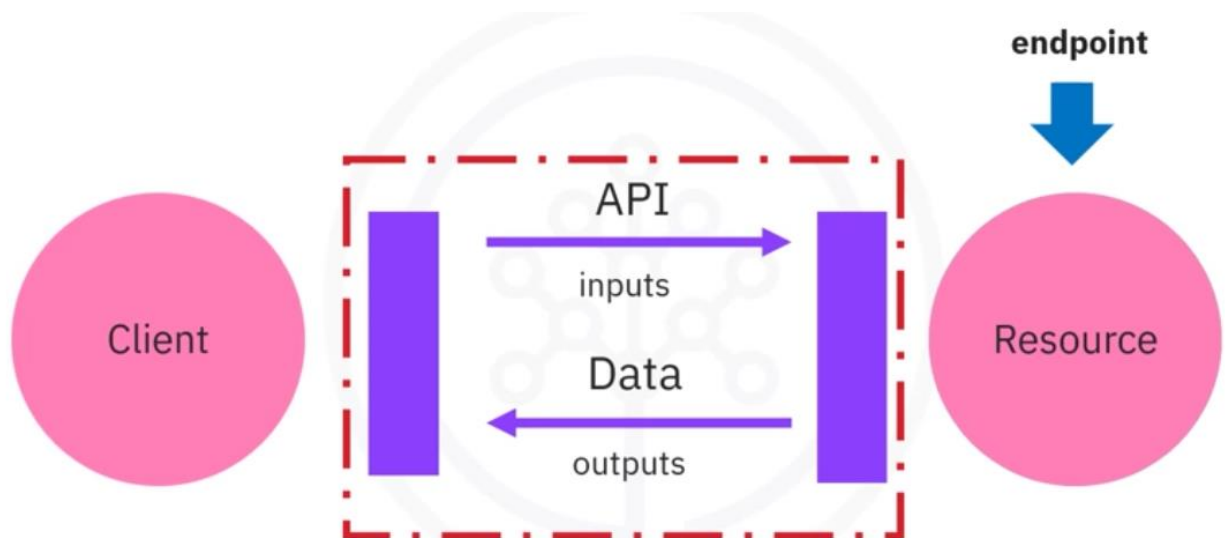
Pandas

## REST API

REST APIs are another popular type of API; they allow you to communicate **through the internet** allowing you to take advantage of resources like storage, access more data, artificial intelligence algorithms, and much more. The **RE** stands for **Representational**, the **S** for **State**, and **T** for **Transfer**.

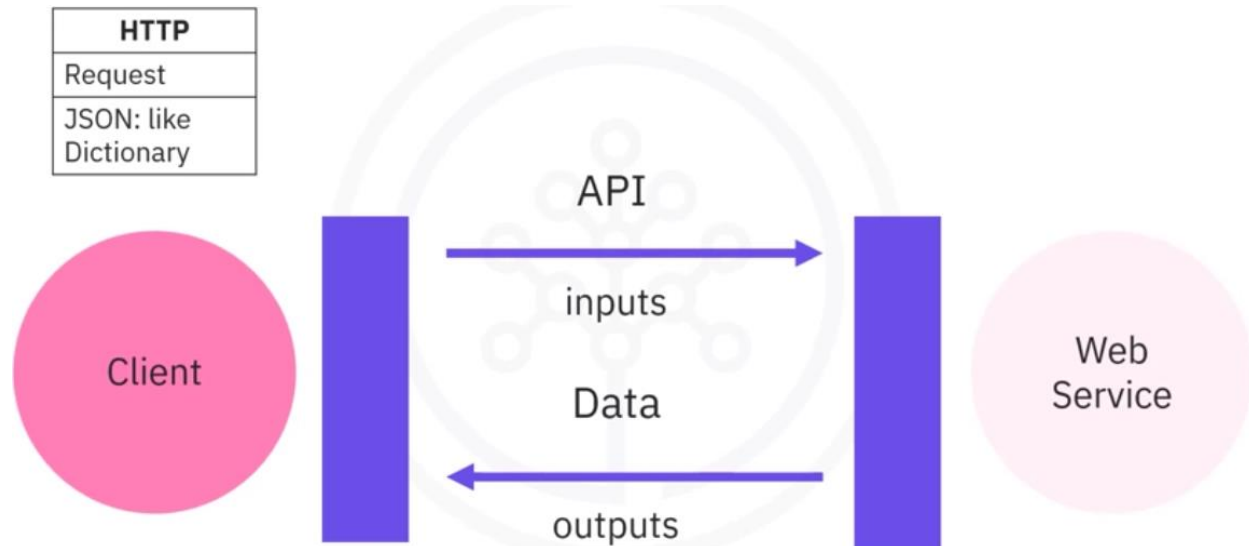
In REST APIs your program is called the **client**. The API communicates with a **web service** you call through the internet. There is a **set of rules regarding communication, input or request, and output or response**. Here are some common terms.

You or your code can be thought of as a **client**. The web service is referred to as a **resource**. The client finds the service via an **endpoint**. The client sends **requests** to the resource and the **response** to the client.

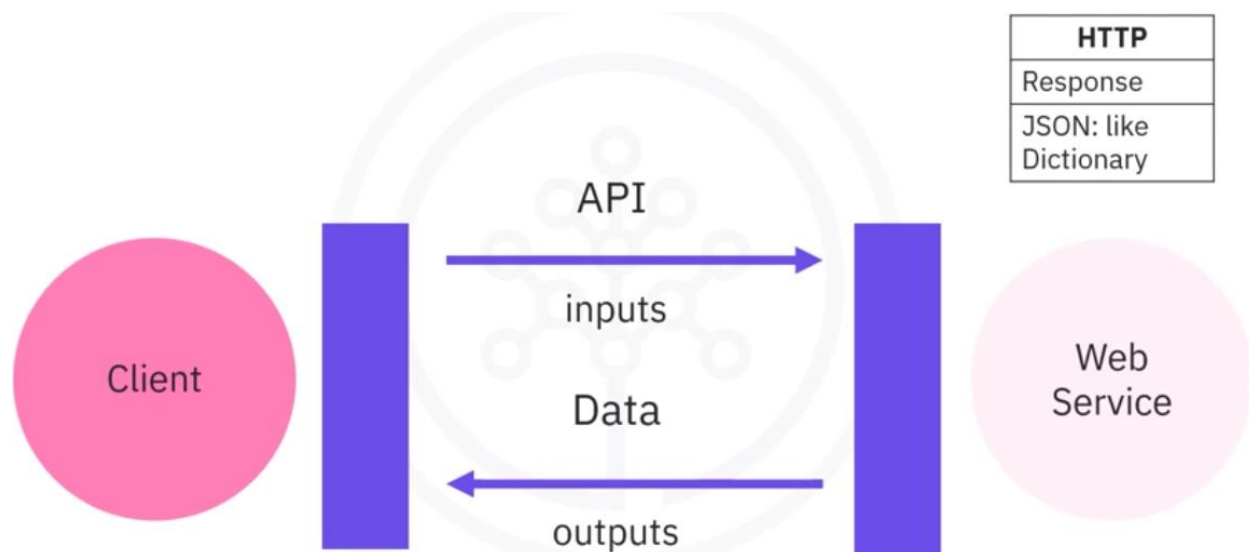


HTTP methods are a way of transmitting data over the internet. We tell the REST APIs what to do by sending a request. The **request** is usually communicated via an HTTP message. The **HTTP message usually contains a JSON file**. This contains **instructions** for what operation

we would like the service to perform. This operation is transmitted to the web service via the internet.



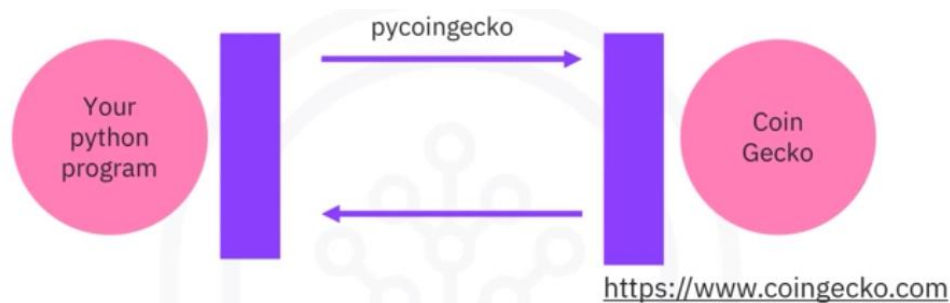
The service performs the operation. In a similar manner, the web service returns a response via an HTTP message, where the information is usually returned via a JSON file. This information is transmitted back to the client.



## USE CASE

Cryptocurrency data is excellent to be used in an API because it is constantly updated and it is vital to cryptocurrency trading. We will use the PyCoinGecko Python client/wrapper for the CoinGecko API, updated every minute by CoinGecko.

**We use the wrapper/client because it is easy to use so you can focus on the task of collecting data,** we will also introduce pandas time series functions for dealing with time series data. Using PyCoinGecko to collect data is simple.



```

!pip install pycoingecko
from pycoingecko import CoinGeckoAPI
cg = CoinGeckoAPI()
bitcoin_data = cg.get_coin_market_chart_by_id(id = 'bitcoin',
vs_currency = 'usd', days=30)
  
```

All we need is to install and import the library, then create a client object, and finally use a function to request our data. In this function we are getting data on bitcoin, in US dollars, for the past 30 days.

In this case our response is a JSON expressed as a Python dictionary of nested lists including **price, market cap, and total volumes**, which contain the UNIX **timestamp** and the price at that time. We are only interested in price so that is what we will select using the **key price**.

bitcoin data:

```

{'prices': [[1705514514561, 42374.46162247654],
[1705518104954, 42549.27806320843],
[1705521707969, 42594.69487372182],
[1705525307604, 42549.30786322405],
[1705528909875, 42635.68516739609],
[1705532503246, 42699.215928408215],
[1705536108559, 42752.74932167083],
[1705539698713, 42636.22685895103],
[1705543290579, 42548.84177998278],
[1705546889967, 42612.90498225697],
[1705550498004, 42538.92367999053],
[1705554105418, 42615.07396771357],
[1705557705611, 42733.060348224586],
[1705561301606, 42745.25542409034],
[1705564902264, 42847.485319378924],
  
```

bitcoin\_data['prices']

```

[[1705514514561, 42374.46162247654],
[1705518104954, 42549.27806320843],
[1705521707969, 42594.69487372182],
[1705525307604, 42549.30786322405],
[1705528909875, 42635.68516739609],
[1705532503246, 42699.215928408215],
[1705536108559, 42752.74932167083],
[1705539698713, 42636.22685895103],
[1705543290579, 42548.84177998278],
[1705546889967, 42612.90498225697],
[1705550498004, 42538.92367999053],
[1705554105418, 42615.07396771357],
[1705557705611, 42733.060348224586],
[1705561301606, 42745.25542409034],
[1705564902264, 42847.485319378924],
  
```

To make things simple, we can convert our nested list to a DataFrame, with the columns timestamp and price it's difficult to understand the column timestamp.

```
import pandas as pd
```

```
data = pd.DataFrame(bitcoin_data['prices'], columns=['TimeStamp', 'Price'])
```

We will convert it to a more readable format using the **pandas' function to\_datetime**. Using the to\_datetime function, we create readable time data, the input is the timestamp column, unit of time is set to milliseconds. We append the output to the new column, date.

```
data['Date'] = pd.to_datetime(data['TimeStamp'], unit = 'ms')
```

|     | TimeStamp     | Price        | Date                    |
|-----|---------------|--------------|-------------------------|
| 0   | 1705514514561 | 42374.461622 | 2024-01-17 18:01:54.561 |
| 1   | 1705518104954 | 42549.278063 | 2024-01-17 19:01:44.954 |
| 2   | 1705521707969 | 42594.694874 | 2024-01-17 20:01:47.969 |
| 3   | 1705525307604 | 42549.307863 | 2024-01-17 21:01:47.604 |
| 4   | 1705528909875 | 42635.685167 | 2024-01-17 22:01:49.875 |
| ... | ...           | ...          | ...                     |
| 716 | 1708092116602 | 52314.974836 | 2024-02-16 14:01:56.602 |
| 717 | 1708095707720 | 51894.782944 | 2024-02-16 15:01:47.720 |
| 718 | 1708099200913 | 51774.070077 | 2024-02-16 16:00:00.913 |
| 719 | 1708102896477 | 52086.168479 | 2024-02-16 17:01:36.477 |
| 720 | 1708104968000 | 51920.829189 | 2024-02-16 17:36:08.000 |

Now we want to create a candlestick plot.

```
candlestick_data = data.groupby(data.Date.dt.date).agg({'Price': ['min',  
'max', 'first', 'last']})
```

To get the data for the daily candlesticks we will group by the date to find the minimum, maximum, first, and last price of each day.

| Date       | Price        |              |              |              |
|------------|--------------|--------------|--------------|--------------|
|            | min          | max          | first        | last         |
| 2024-01-17 | 42374.461622 | 42699.215928 | 42374.461622 | 42699.215928 |
| 2024-01-18 | 40894.520842 | 42847.485319 | 42752.749322 | 41321.666233 |
| 2024-01-19 | 40377.501822 | 41752.772512 | 41260.439354 | 41636.055733 |
| 2024-01-20 | 41467.100392 | 41777.258891 | 41623.930438 | 41777.258891 |
| 2024-01-21 | 41570.971370 | 41839.988102 | 41661.368099 | 41601.179213 |
| 2024-01-22 | 39727.783338 | 41612.829218 | 41545.548426 | 39820.593582 |
| 2024-01-23 | 38678.188080 | 40049.095760 | 39518.636999 | 39446.548800 |
| 2024-01-24 | 39563.706549 | 40441.366290 | 39852.047871 | 39868.700723 |
| 2024-01-25 | 39669.928827 | 40149.639037 | 40061.723693 | 39949.220119 |
| 2024-01-26 | 39876.531792 | 42069.029347 | 39917.164425 | 41883.891094 |
| 2024-01-27 | 41439.024787 | 42147.820174 | 41863.006876 | 42127.570805 |
| 2024-01-28 | 41749.655704 | 42718.873130 | 42119.610495 | 41821.000815 |
| 2024-01-29 | 41920.844894 | 43212.003823 | 42037.094218 | 43156.865352 |
| 2024-01-30 | 43196.664459 | 43644.372959 | 43285.811667 | 43337.837788 |
| 2024-01-31 | 42404.142121 | 43654.517128 | 42950.147697 | 42648.584147 |

Finally, we will use plotly to create the candlestick chart and plot it.

```
import plotly.graph_objects as go
import plotlyfig = go.Figure(data=[go.Candlestick(x = candlestick_data.index,
                                                    open = candlestick_data['Price']['first'],
                                                    high = candlestick_data['Price']['max'],
                                                    low = candlestick_data['Price']['min'],
                                                    close = candlestick_data['Price']['last']
                                                    )
                                ])
fig.update_layout(xaxis_rangeslider_visible = False, xaxis_title = 'Date',
yaxis_title = 'Price (USD $)', title = 'Bitcoin Candlestick Chart Over Past 30
Days')

plotly.offline.plot(fig, filename = './bitcoin_candlestick_graph.html',
auto_open=False)
```

Now we can view the candlestick chart by opening the HTML file and clicking Trust HTML in the top left of the tab. It should look something like this.

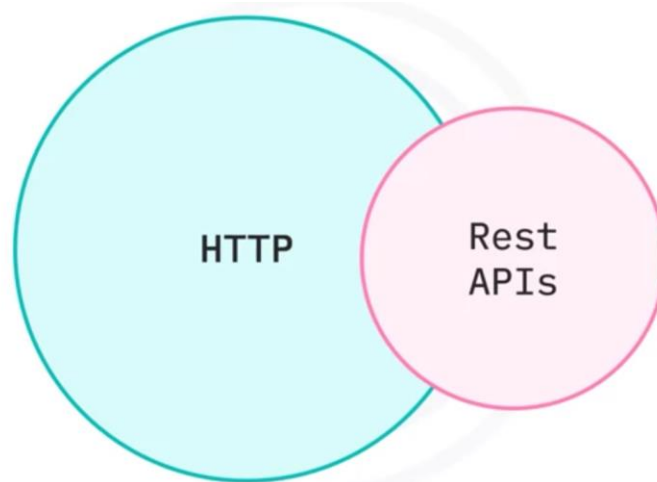
Bitcoin Candlestick Chart Over Past 30 Days



Rest APIs, Web scraping, and working with files

## HTTP Protocol

The HTTP protocol can be thought of as a **general protocol of transferring information through the web**. This **includes many types of REST APIs**.



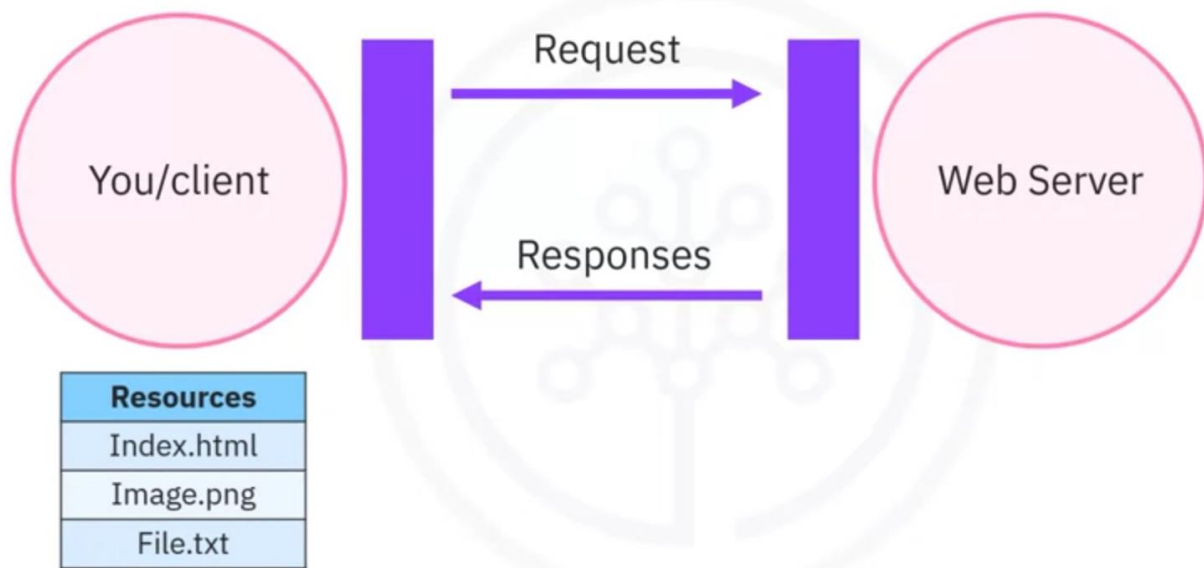
Recall that REST APIs function by sending a request, and the request is communicated via HTTP message. The HTTP message usually contains a JSON file.



When you, the client, use a web page **your browser sends an HTTP request to the server** where the page is hosted. The server tries to find the desired resource by default "index.html". If your request is successful, **the server will send the object to the client in an HTTP response**; this includes information like the **type of the resource**, the **length of the resource**, and other information.

## HTTP Request

---



The table under the Web server represents a list of resources stored in the web server. In this case, an HTML file, png image, and txt file. When the request is made for the information, the web servers sends the the requested information, that is, one of the files.

### URL

A **uniform resource locator** (URL) is the most popular way to find resources on the web.

We can break the URL into three parts:

- **The scheme:** this is the protocol, and for this lab it will always be <http://>
- **The internet address or Base URL:** this will be used to find the location; some examples include [www.ibm.com](http://www.ibm.com) and [www.gitlab.com](http://www.gitlab.com)
- **The route:** this is the location on the web server, for example: `/images/IDSNlogo.png`

- **Scheme:** the protocol, for this lab, it will always be **http://**
- **Internet address or Base URL:** used to find the location, for example: **www.ibm.com** and **www.gitlab.com**
- **Route:** location on the web server, for example: **/images/IDSNlogo.png**

**http://www.ibm.com /images/IDSNlogo.png**

### *Request*

In the **start line** we have the HTTP GET method requesting the file index.html.

The Request **header** passes additional information with an HTTP request. In the GET method the Request header is empty.

Some Requests have a body; we will have an example of a request body later.

|                           |                                                                           |
|---------------------------|---------------------------------------------------------------------------|
| <b>Request Start line</b> | /Get/index.html HTTP/1.0                                                  |
| <b>Request Header</b>     | User-Agent: python-requests/2.21.0<br>Accept-Encoding: gzip, deflate<br>: |
| <b>Request Body</b>       |                                                                           |

### *Reponse*

The response start line contains the **version number** followed by a descriptive phrase. In this case, HTTP/1.0 a **status code** (200) meaning success, and the **descriptive phrase**, OK. The response header contains information. Finally, we have the **response body containing the requested file**, in this case an HTML document.



|                            |                                                                                                                      |
|----------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>Response Start line</b> | HTTP/1.0 200 OK                                                                                                      |
| <b>Response Header</b>     | Server: Apache-Cache:<br>UNCACHEABLE                                                                                 |
| <b>Response Body</b>       | <!DOCTYPE html><br><html><br><body><br><h1>My First Heading</h1><br><p>My first paragraph.</p><br></body><br></html> |

### Status code

The prefix indicates the class;

The 100s are **informational responses**; 100 indicates that everything is OK so far.

The 200s are **Successful responses**: For example, 200 The request has succeeded.

The 400s are **bad news**. 401 means the request is unauthorized.

The 500's stands for server errors, like 501 for not Implemented.

|     |                         |
|-----|-------------------------|
| 1XX | Informational           |
| 100 | Everything So Far Is OK |
| 2xx | Success                 |
| 200 | OK                      |
| 3XX | Redirection             |
| 300 | Multiple Choices        |

|     |                |
|-----|----------------|
| 4XX | Client Error   |
| 401 | Unauthorized   |
| 403 | Forbidden      |
| 404 | Not Found      |
| 500 | Server Error   |
| 501 | No Implemented |

### Methods

When an HTTP request is made, an HTTP method is sent. This tells the server what action to perform. A list of several HTTP methods is shown here.

| HTTP Methods | Description                    |
|--------------|--------------------------------|
| GET          | Retrieves data from the server |
| POST         | Submits data to server         |
| PUT          | Updates data already on server |
| DELETE       | Deletes data from server       |

## Python Requests Library

The **requests** library is one of several libraries, including **httplib** and **urllib**, that can work with the HTTP protocol. Requests is a Python library that **allows you to send HTTP/1.1 requests easily**. We can import the library as follows:

You can **make a GET request via the method get()** to [www.ibm.com](https://www.ibm.com/): We have the **response object "r"**. This has information about the request, like the status of the request. We can view the status code using the attribute **status\_code**, which is 200 for OK. You can view the **request headers**: You can view the request body in the following line.

```
import requests
url='https://www.ibm.com/'
r=requests.get(url)
r.status_code :200
r.request.headers

{'User-Agent': 'python-requests/2.24.0', 'Accept-Encoding': 'gzip, deflate',
'Accept': '*/*', 'Connection': 'keep-alive', 'Cookie':
'_abck=A5C90067E0241F8BBD3ECC70ECDB1EC0~-
1~YAAQLc2U0ZaliKB1AQAAwucY4QT77mnX/GJQJvRGOV48PDVR70euYZ9FTBsFbF3z4kwEcKgttV+Ot6
SzP+sjsPDIFni428WfQn/yUYVddQ2e1ejPilhGVE+eNJl4xbIRXxE1U59jQLvi0jd/Q8e6ClE7mw27Qz
b5nCGdMcd4ZnULKeu6U5QSVLFRUPnhMLvOM40+1WFW1CNH36WxccSUtoKkTsExTltvbsvxIbthin3znf
L1racKtV9WJFJlAEdqajkt0qX/frGBULGLK/r47j78DuZHabdMhdalss9J1MpfXj6kXCFe~-1~-1~-1;
bm_sz=564D34683F98DB203E92B73A05A3324C~YAAQLc2U0ZWliKB1AQAAwucY4Qn7jwZFH9N07QY5q
55JvdADHcL95RCSPsAGLlx2RAR+iIH6hEt/frpWT8RXkjWBcCtS29lu91pIEz6zjed+o0yyTH7aL/qkC
9nzVmdZfvAh0jFKnFzewR7xguYLBc7XlAAeG8GrkPFHq0vMT038GwTMCC+xGYQYpM6y' }
```

As there is **no body for a GET request**, we get a **None**.

```
r.request.body: None
```

```
header=r.headers
```

```
header:
```

```
{'Server': 'Apache', 'x-drupal-dynamic-cache': 'UNCACHEABLE', 'Link':  
'<https://www.ibm.com/ca-en>; rel="canonical", <https://www.ibm.com/ca-en>; rel="revision",  
<https://www.ibm.com/ca-en>; rel="revision", <https://1.cms.s81c.com>; rel=preconnect;  
crossorigin, <https://1.cms.s81c.com>; rel=dns-prefetch', 'x-ua-compatible': 'IE=edge', 'Content-  
Language': 'en-ca', 'x-generator': 'Drupal 8 (https://www.drupal.org)', 'x-dns-prefetch-  
control': 'on', 'x-drupal-cache': 'MISS', 'Last-Modified': 'Thu, 19 Nov 2020 10:32:43 GMT',  
'ETag': '"1605781963"', 'Content-Type': 'text/html; charset=UTF-8', 'x-acquia-host':  
'www.ibm.com', 'x-acquia-path': '/ca-en', 'x-acquia-site': '', 'x-acquia-purge-tags': '',  
'x-varnish': '18482279 12683819', 'x-cache-hits': '9', 'x-age': '7030', 'Accept-Ranges':  
'bytes', 'Content-Encoding': 'gzip', 'Cache-Control': 'public, max-age=300', 'Expires':  
'Thu, 19 Nov 2020 15:26:47 GMT', 'X-Akamai-Transformed': '9 11615 0 pmb=mTOE,1', 'Date':  
'Thu, 19 Nov 2020 15:21:47 GMT', 'Content-Length': '11725', 'Connection': 'keep-alive',  
'Vary': 'Accept-Encoding', 'x-content-type-options': 'nosniff', 'X-XSS-Protection': '1;  
mode=block', 'Content-Security-Policy': 'upgrade-insecure-requests', 'Strict-Transport-  
Security': 'max-age=31536000', 'x-ibm-trace': 'www-dipatcher: dynamic rule'}
```

You can view the HTTP response header using the attribute **headers**. This returns a **python dictionary** of HTTP response headers. We can look at the dictionary values. We can obtain the **date** the request was sent by using the key Date. The key **Content-Type** indicates the type of data. Using the response object 'r', we can also check the **encoding**. As the Content-Type is text/html, we can use the attribute **text to display the HTML** in the body. We can review the first 100 characters. You can also download other content, see the lab for more.

```
header['date']:'Thu, 19 Nov 2020 15:21:47 GMT'
```

```
header['Content-Type']:'text/html; charset=UTF-8'
```

```
r.encoding:'UTF-8'
```

```
r.text[0:100]:
```

```
'<!DOCTYPE html>\n<html lang="en-ca" dir="ltr">\n<head>\n <meta charset="utf-8"  
>\n<script>digitalD'
```

## GET request with URL parameters

You can use the GET method to modify the results of your query. For example, retrieving data from an API.

| Base URL        | Route |
|-----------------|-------|
| httpbin.org     | /get  |
| httpbin.org/get |       |

In the lab we will use httpbin.org A simple HTTP Request & Response Service. We send a GET request to the server. Like before, we have the Base URL in the Route; we append /get. This indicates we would like to preform a GET request. This is demonstrated in the following table:

| Start of Query                             | Parameter Name |   | Value  |   | Parameter Name |   | Value |
|--------------------------------------------|----------------|---|--------|---|----------------|---|-------|
| ?                                          | name           | = | Joseph | & | ID             | = | 123   |
| http://httpbin.org/get? Name=Joseph&ID=123 |                |   |        |   |                |   |       |

After GET is requested we have the **query string**. This is a part of a uniform resource locator (URL) and this sends other information to the web server. The **start of the query is a ?**, followed by a series of **parameter and value pairs**, as shown in the table below. The first parameter name is "name" and the value is "Joseph." The second parameter name is "ID" and the Value is "123." Each pair, parameter, and value is **separated by an equal sign, "="**. The series of pairs is **separated by the ampersand, "&."**



# Create query string

```
url_get='http://httpbin.org/get'
payload={"name": "Joseph", "ID": "123"}
r=requests.get(url_get,params=payload)

r.url:'http://httpbin.org/get?name=Joseph&ID=123'
r.request.body : None
r.status_code: 200
```

```
r.text      { "args": { "ID": "123", "name": "Joseph" },
               "headers": { "Accept": "*/*", "Accept-Encoding":
               "gzip, deflate", "Host": "httpbin.org", "User-
               Agent": "python-requests/2.24.0", "X-Amzn-Trace-
               Id": "Root=1-5fbbd03e-106584ab42513a6d5cef39a9"
               }, "origin": "99.228.137.181", "url":
               "http://httpbin.org/get?name=Joseph&ID=123" }
```

```
r.headers['Content-Type'] 'application/json'
```

```
r.json()     {'args': {'ID': '123', 'name': 'Joseph'},
               'headers': {'Accept': '*/*', 'Accept-Encoding':
               'gzip, deflate', 'Host': 'httpbin.org', 'User-
               Agent': 'python-requests/2.24.0', 'X-Amzn-Trace-
               Id': 'Root=1-5fbbd03e-106584ab42513a6d5cef39a9'},
               'origin': '99.228.137.181', 'url':
               'http://httpbin.org/get?name=Joseph&ID=123'}
```

```
r.json()['args']    {'ID': '123', 'name': 'Joseph'}
```

Let's complete an example in python. We have the Base URL with GET appended to the end. **To create a Query string, we use the dictionary payload.** The keys are the parameter names, and the values are the value of the Query string. Then, we pass the dictionary **payload to the params parameter** of the get() function. We can print out the URL and see the name and values. We can see the request body. As the info is sent in the URL, the body has a value of None. We can print out the **status code**. We can view the response as **text**:

We can look at the key '**Content-Type**' to look at the content type. As the content 'Content-Type' is in the JSON, **we format it using the method json()** . It returns a Python dict: The key 'args' has the name and values for the query string.

## POST request

Like a GET request, a POST request is used to **send data to a server**, but the POST request sends the data **in a request body, not the url**.

# POST

---



```
url_post="http://httpbin.org/post"
```

```
payload={"name": "Joseph", "ID": "123"}
```

```
r_post=requests.post(url_post,data=payload)
```

In order to send the Post Request in the URL, we **change the route to POST**: This endpoint will expect data and it is a convenient way to configure an HTTP request to send data to a server. We have the Payload dictionary. To make a POST request, we use the post() function. The variable **payload is passed to the parameter data**.

# Compare POST and GET

---

```
print("POST request URL:", r_post.url )  
print("GET request URL:", r.url)
```

```
POST request URL: http://httpbin.org/post  
GET request URL:  
http://httpbin.org/get?name=Joseph&ID=123
```

Comparing the URL using the attribute url from the response object of the GET and POST request, we see the **POST request has no name or value pairs in its url.**

# Compare POST and GET

---

```
print("POST request body:", r_post.request.body)  
print("GET request body:", r.request.body)
```

```
POST request body: name=Joseph&ID=123  
GET request body: None
```

```
r_post.json()['form']
```

```
{ 'ID': '123', 'name': 'Joseph' }
```

We can compare the POST and GET request body. We see **only the POST request has a body, and we can view the key form to get the payload.**

In this video, we will discuss the HTTP protocol using the Requests Library a popular method: Added to Selection. Press [CTRL + S] to save as a note

## HTML for web Scrapping

Lots of useful data is available on web pages. If you have an **understanding of HTML**, you can use Python **to extract this information**.

# HTML composition

---

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>
<h3><b id='boldest'> LeBron James</b></h3>
<p> Salary: $ 92,000,000 </p>
<h3> Stephen Curry</h3>
<p> Salary: $85,000, 000 </p>
<h3> Kevin Durant </h3>
<p> Salary: $73,200, 000</p>
</body>
</html>
```

**Lebron James**

Salary \$92,000,000

**Stephen Curry**

Salary \$85,000,000

**Kevin Durant**

Salary \$73,200,000

Let's say you were asked to find the name and salary of players in a National Basketball League from the following page. The web page is comprised of HTML. It consists of text surrounded by a series of blue text elements enclosed in angle brackets called tags. The tags tell the browser how to display the content. The data we require is in this text. The first portion contains the **"DOCTYPE html"** which declares this document is an HTML document. The "html" element is the **root element** of an HTML page, and "head" element contains **meta information** about the HTML page. Next, we have the **body**, this is what's displayed on the web page. This is usually **the data** we are interested in,

We see the elements with an "h3", this means **type 3 heading**, makes the text larger and bold. These tags have the names of the players, notice the data is enclosed in the elements. It **starts** with a **h3** in brackets and **ends** in a **slash h3** in brackets.



## HTML elements

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>
<h3><b id='boldest'> LeBron James</b></h3>
<p> Salary: $ 92,000,000 </p>
<h3> Stephen Curry</h3>
<p> Salary: $85,000, 000 </p>
<h3> Kevin Durant </h3>
<p> Salary: $73,200, 000</p>
</body>
</html>
```

### Lebron James

Salary \$92,000,000

### Stephen Curry

Salary \$85,000,000

### Kevin Durant

Salary \$73,200,000

There is also a different tag “p”, this means **paragraph**, each p tag contains a player's salary.

## HTML paragraph tags

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>
<h3><b id='boldest'> LeBron James</b></h3>
<p> Salary: $ 92,000,000 </p>
<h3> Stephen Curry</h3>
<p> Salary: $85,000, 000 </p>
<h3> Kevin Durant </h3>
<p> Salary: $73,200, 000</p>
</body>
</html>
```

### Lebron James

Salary \$92,000,000

### Stephen Curry

Salary \$85,000,000

### Kevin Durant

Salary \$73,200,000

### HTML Tag

### Element

```
<a href="https://www.ibm.com"> IBM webpage </a>
```

Here is an example of an **HTML Anchor tag**. It will display IBM and when you click it, it will send you to IBM.com. We have the **tag name**, in this case “a”. This tag defines a hyperlink, which is used to link from one page to another. It’s helpful to think of each tag name as a class in Python and each individual tag as an instance. We have the opening or **start tag** and we have the **end tag**. This has the tag name preceded by a slash.

Opening/start tag

```
<a href="https://www.ibm.com"> IBM webpage </a>
```

End tag

```
<a href="https://www.ibm.com"> IBM webpage </a>
```

Content

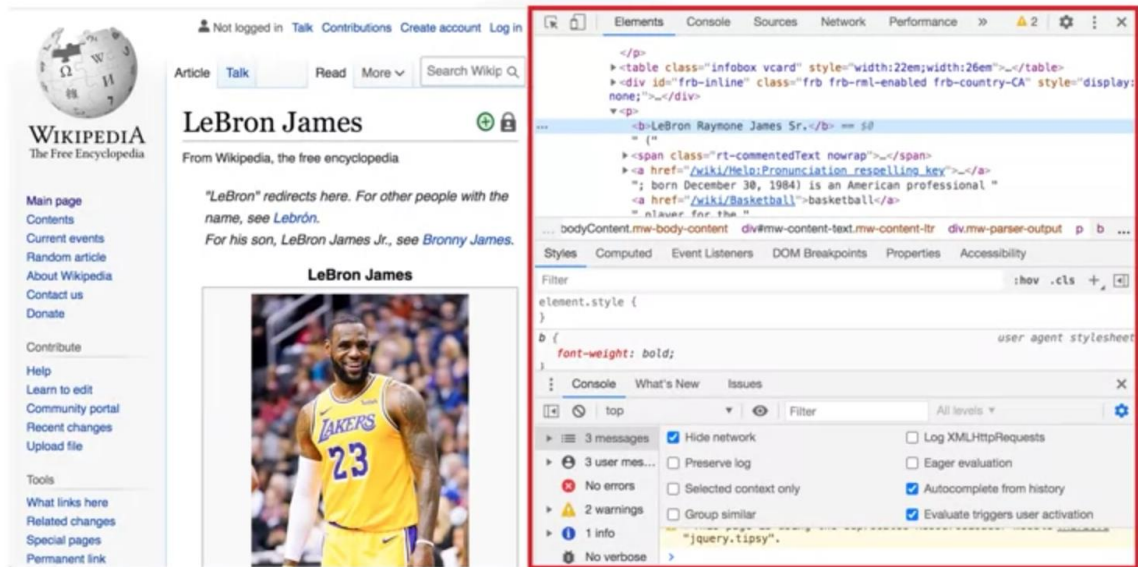
Attribute value

```
<a href="https://www.ibm.com"> IBM webpage </a>
```

Attribute name

These tags contain the **content**, in this case what's displayed on the web page. We have the **attribute**, this is composed of the Attribute Name and Attribute Value. In this case it the url to the destination web page. Real web pages are more complex, depending on your browser you can select the HTML element, then click Inspect. The result will give you the ability to inspect the HTML. There is also other types of content such as CSS and JavaScript that we will not go over in this course.

# Inspect HTML

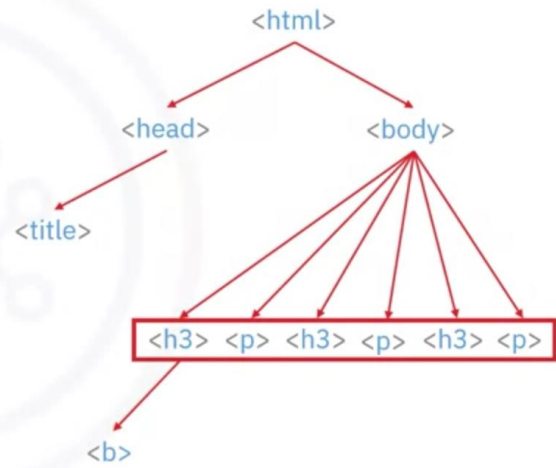


## HTML Trees

The actual element is shown here. **Each HTML document can actually be referred to as a document tree.** Let's go over a simple example. Tags may contain strings and other tags. These elements are the tag's children. We can represent this as a family tree. **Each nested tag is a level in the tree.** The tag **HTML tag contains the head and body tag.** The Head and body tag are the descendants of the html tag. In particular they are the children of the HTML tag. HTML tag is their parent. The **head and body tag are siblings** as they are on the same level. Title tag is the **child** of the head tag and its **parent** is the head tag. The title tag is a **descendant** of the HTML tag but not its child. The heading and paragraph tags are the children of the body tag; and as they are all children of the body tag they are siblings of each other. The bold tag is a child of the heading tag, the content of the tag is also part of the tree but this can get unwieldy to draw.

# Document tree

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    <h3><b id='boldest'>Lebron James</b></h3>
    <p>Salary: $ 92,000,000 </p>
    <h3>Stephen Curry</h3>
    <p>Salary: $85,000, 000 </p>
    <h3>Kevin Durant </h3>
    <p>Salary: $73,200, 000</p>
  </body>
</html>
```



HTML Table

## HTML tables

```
<table >
  <tr>
    <td>Pizza Place</td>
    <td>Orders</td>
    <td>Slices </td>
  </tr>
  <tr>
    <td>Domino Pizza</td>
    <td>10</td>
    <td>100</td>
  </tr>
  <tr>
    <td>Little Caesars</td>
    <td>12</td>
    <td>144 </td>
  </tr>
</table>
```

| Pizza Place    | Orders | Slices |
|----------------|--------|--------|
| Domino's Pizza | 10     | 100    |
| Little Caesars | 12     | 144    |

Next, let's review HTML tables.

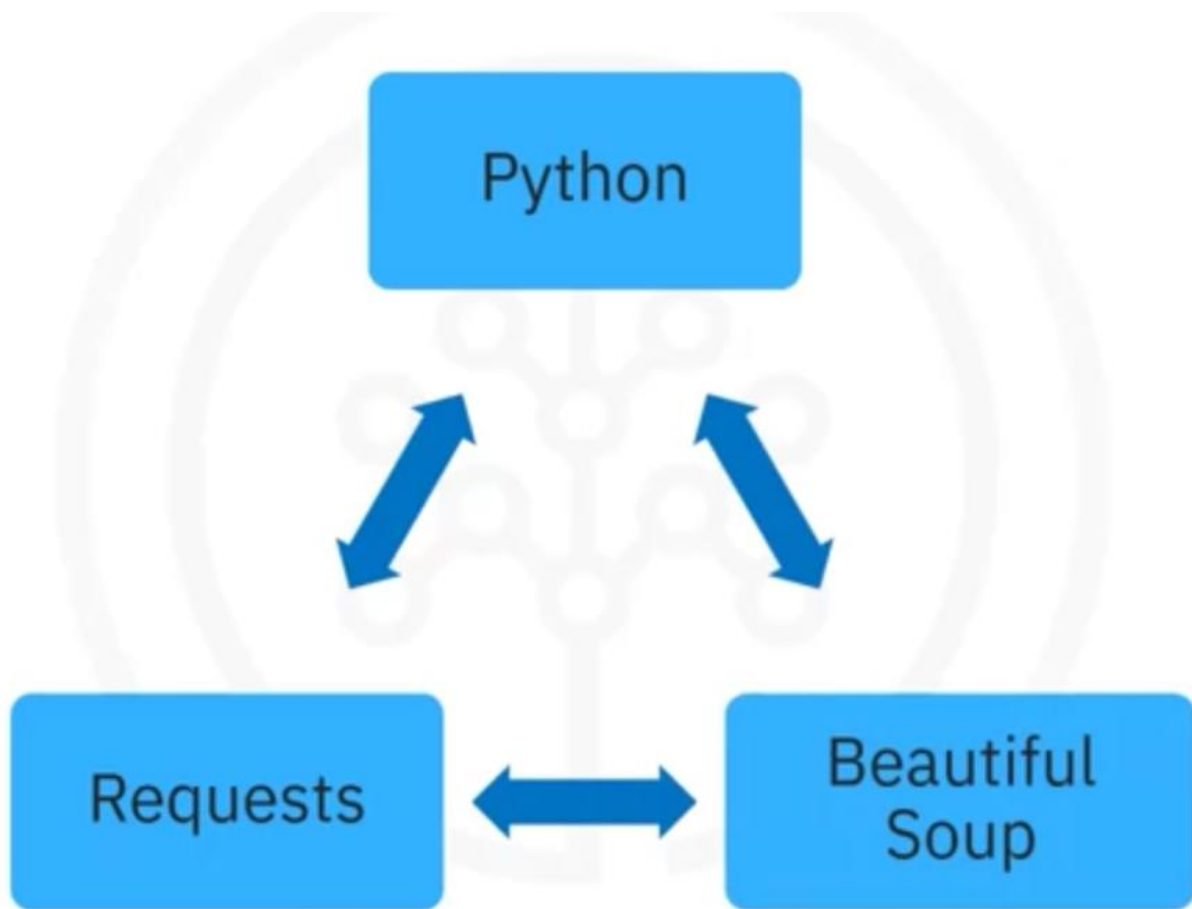
To define an **HTML table** we have the table tag. Each **table row** is defined with a "tr" tag, you can also use a **table header tag** for the first row. The table row cell contains a set of "td" tags,

each defines a **table cell**. For the first row first cell we have; for the first row second cell we have; and so on.

For the second row we have; and for the second row first cell we have; for the second row second cell we have; and so on.

We now have some basic knowledge of HTML. Now let's try and extract some data from a web page.

## WEB Scrapping with Python



What would you do if you wanted to analyze **hundreds of points of data** to find the best players of a sports team? Would you start manually copying and pasting information from different websites into a spreadsheet? Spending hours trying to find the right data, and eventually giving up because the task was too overwhelming? That's where web scraping can help.

Web scraping is a process that can be used to **automatically extract information from a website**, and can easily be accomplished **within a matter of minutes and not hours**. To get

started, we just need a little Python code and the help of two modules named Requests and BeautifulSoup.

Let's say you were asked to find the name and salary of players in a National Basketball League, from the following webpage.

```
from bs4 import BeautifulSoup

html = "<!DOCTYPE html><html><head><title>Page  
Title</title></head><body><h3><b id='boldest'>Lebron  
James</b></h3><p> Salary: $ 92,000,000 </p><h3> Stephen  
Curry</h3><p> Salary: $85,000, 000 </p><h3> Kevin Durant </h3><p>  
Salary: $73,200, 000</p></body></html>"

soup = BeautifulSoup(html, 'html5lib')
```

First we **import** BeautifulSoup. We can **store the webpage HTML as a string in the variable HTML**. To parse a document, pass it into the BeautifulSoup constructor. We get the **Beautiful Soup object, soup**, which represents the **document as a nested data structure**. BeautifulSoup represents HTML as a set of Tree like objects with methods used to parse the HTML.

### *BeautifulSoup Objects*

We will review the BeautifulSoup object using the BeautifulSoup object, soup, we created.

```
tag_object=soup.title
tag_object:
<title>Page Title</title>

tag_object=soup.h3
tag_object:
<h3><b id='boldest'>Lebron James</b></h3>
<h3> Stephen Curry</h3>
<h3> Kevin Durant </h3>
<p> Salary: $73,200, 000</p>
</h3>
```

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>
<h3><b id='boldest'> Lebron James</b></h3>
<p> Salary: $ 92,000,000 </p>
<h3> Stephen Curry</h3>
<p> Salary: $85,000, 000 </p>
<h3> Kevin Durant </h3>
<p> Salary: $73,200, 000</p>
</body>
</html>
```

The tag object **corresponds to an HTML tag in the original document**. For example, the tag "title."

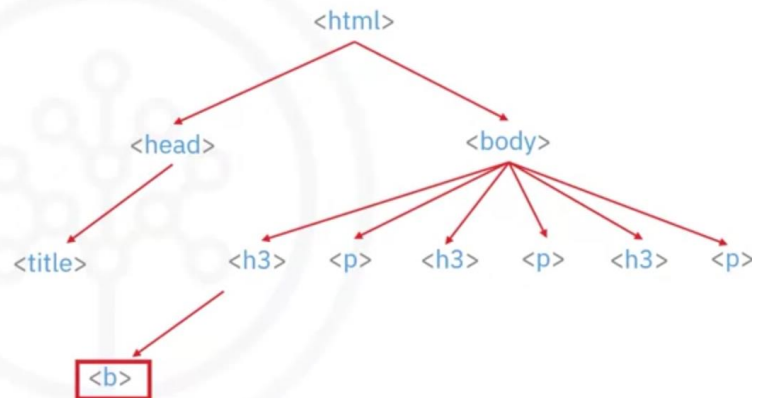


Consider the tag h3. **If there is more than one tag with the same name, the first element with that tag is selected.** In this case with Lebron James, we see the name is Enclosed in the bold attribute "b". **To extract it, use the Tree representation.**

```
tag_object: <h3><b id="boldest">Lebron James</b></h3>
```

```
tag_child =tag_object.b  
tag_child:
```

```
<b id="boldest">Lebron James</b>
```



Let's use the Tree representation.

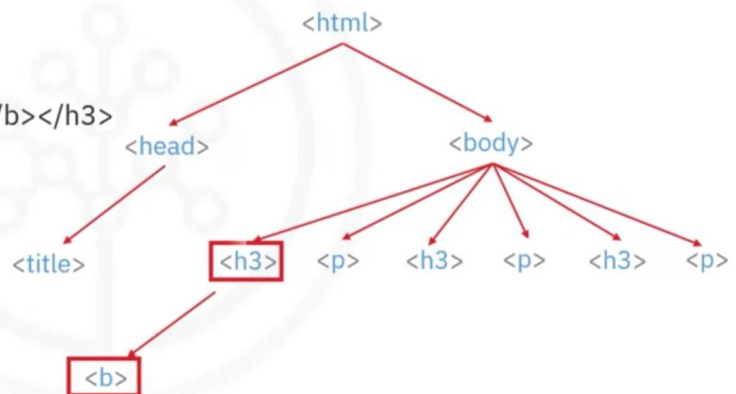
The variable tag-object is located here. We can access the child of the tag or navigate down the branch as follows:

You can navigate up the tree by using the **parent attribute**.

```
tag_child:<b id="boldest">Lebron James</b>
```

```
parent_tag=tag_child.parent  
parent_tag:
```

```
<h3><b id="boldest">Lebron James</b></h3>
```



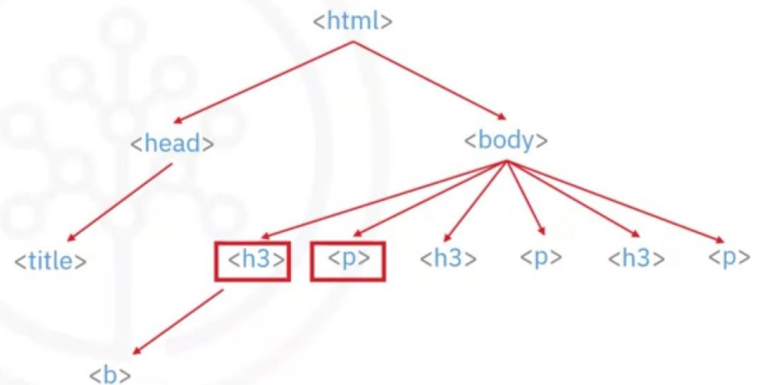
The variable tag child is located here. We can access the parent. This is the original tag object.

We can find the sibling of "tag object."

tag\_object: <h3><b id="boldest">Lebron James</b></h3>

sibling\_1= tag\_object.next\_sibling  
sibling\_1:

<p> Salary: \$ 92,000,000 </p>

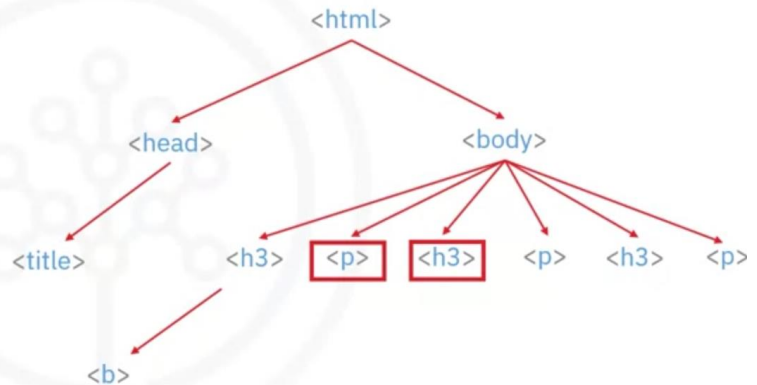


We simply use the **next-sibling** attribute.

sibling\_1: <p> Salary: \$ 92,000,000 </p>

sibling\_2=sibling\_1.next\_sibling  
sibling\_2:

<h3> Stephen Curry</h3>



We can find the sibling of sibling one. We simply use the next sibling attribute.

Consider the tag child object. You can access the **attribute name and value as a key value pair in a dictionary** as follows. You can return the content as a Navigable string, this is like a Python string that supports BeautifulSoup functionality.



tag\_child:<b id="boldest">Lebron James</b>

tag\_child.attrs:

{'id': 'boldest'}

tag\_child.string:

'Lebron James'

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>
<h3><b id='boldest'> Lebron
James</b></h3>
<p> Salary: $ 92,000,000 </p>
<h3> Stephen Curry</h3>
<p> Salary: $85,000, 000 </p>
<h3> Kevin Durant </h3>
<p> Salary: $73,200, 000</p>
</body>
</html>
```

*Find\_all()*

This is a filter, you can use filters to **filter based on a tag's name**, its attributes, the text of a string, or on some combination of these.

Consider the list of pizza places.

```
<table >
  <tr>
    <td>Pizza Place</td>
    <td>Orders</td>
    <td>Slices </td>
  </tr>
  <tr>
    <td>Domino Pizza</td>
    <td>10</td>
    <td>100</td>
  </tr>
  <tr>
    <td>Little
Caesars</td>
    <td>12</td>
    <td>144 </td>
  </tr>
</table>
```

| Pizza Place    | Orders | Slices |
|----------------|--------|--------|
| Domino's Pizza | 10     | 100    |
| Little Caesars | 12     | 144    |

Like before, create a BeautifulSoup object. But this time, name it table.

```
from bs4 import BeautifulSoup
```

```
html= "<table><tr><td>Pizza Place</td><td>Orders</td> <td>Slices</td></tr><tr><td>Domino's Pizza</td><td>10</td><td>100</td></tr><tr><td>Little Caesars</td><td>12</td><td>144</td></tr></table>"
```

```
table = BeautifulSoup(html, 'html5lib')
```

The `find_all()` method looks through a tag's descendants and **retrieves all descendants that match your filters**. Apply it to the table with the tag `tr`. The result is a Python iterable just like a list, each element is a tag object for `tr`. This corresponds to each row in the list including the table header.

```
table_row=table.find_all(name='tr')
```

```
table_row:
```

| Pizza Place    | Orders | Slices |
|----------------|--------|--------|
| Domino's Pizza | 10     | 100    |
| Little Caesars | 12     | 144    |

```
[<tr><td>Pizza Place</td><td>Orders</td><td>Slices</td></tr>,<br><tr><td>Domino's Pizza</td><td>10</td><td>100</td></tr>,<br><tr><td>Little Caesars</td><td>12</td><td>144</td></tr>,<br>]
```

Each element is a tag object. Consider the first row. For example, we can extract the first table cell.

```
first_row = table_row[0]
```

```
first_row:
```

```
<tr><td>Pizza Place</td><td>Orders</td> <td>Slices </td></tr>
```

```
first_row : |
```

```
<td>Pizza Place</td>
```

We can also iterate through each table cell. First, we iterate through the list “table rows,” via the variable `row`. Each element corresponds to a row in the table. We can apply the method `find_all` to find all the table cells, then we can iterate through the variable `cells` for each row.

For each iteration, the variable cell corresponds to an element in the table for that particular row. We continue to iterate through each element and repeat the process for each row.

```
for i,row in enumerate(table_rows):  
    print("row", i)  
    cells=row.find_all("td")  
    for j,cell in enumerate(cells):  
        print("column", j , "cell", cell)
```

| Pizza Place    | Orders | Slices |
|----------------|--------|--------|
| Domino's Pizza | 10     | 100    |
| Little Caesars | 12     | 144    |
| Papa John's    | 15     | 166    |

Let's see how to apply BeautifulSoup to a webpage.

```
import requests  
from bs4 import BeautifulSoup  
  
page = requests.get("http://EnterWebsiteURL...").text  
  
#Creates a BeautifulSoup object  
soup = BeautifulSoup(page, "html.parser")  
  
# Pulls all instances of <a> tag  
artists = soup.find_all('a')  
  
#Clears data of all tags  
for artist in artists:  
    names = artist.contents[0]  
    fullLink = artist.get('href')  
    print(names)  
    print(fullLink)
```

To scrape a webpage we also need the Requests library. The first step is to import the modules that are needed. Use the get method from the requests library to download the webpage. The input is the URL. Use the text attribute to get the text and assign it to the variable page. Then, create a BeautifulSoup object 'soup' from the variable page. It will allow you to parse through the HTML page. You can now scrape the Page. Check out the labs for more.

## Summary. Cheatsheet, Grades Quiz, Glossary

Pandas has a `read_html()` function that extracts table data found in a URL

Congratulations! You have completed this module. At this point, you know that:

- Simple APIs in Python are application programming interfaces that provide straightforward and easy-to-use methods for interacting with services, libraries, or data, often with minimal configuration or complexity.
  - An API lets two pieces of software talk to each other.
  - Using an API library in Python entails importing the library, calling its functions or methods to make HTTP requests, and parsing the responses to access data or services provided by the API.
  - Pandas API processes the data by communicating with the other software components.
  - An Instance forms when you create a dictionary and then use the DataFrames constructor to create a Pandas object.
  - Method “`head()`” will display the mentioned number of rows from the top (default 5) of DataFrames, while method “`mean()`” will calculate the mean and return the values
- Rest APIs allow you to communicate through the internet, taking advantage of resources like storage, access more data, AI algorithms, and so on.
  - HTTP methods transmit data over the internet.
  - An HTTP message typically includes a JSON file with instructions for operations.
  - HTTP messages containing JSON files are returned to the client as a response from web services.
  - Dealing with time series data involves using the Pandas time series function.
  - You can get data for daily candlesticks and plot the chart using Plotly with the candlestick plot.
- The HTTP (HyperText Transfer Protocol) transfers data, including web pages and resources, between a client (a web browser) and a server on the World Wide Web.
  - The HTTP protocol is commonly used for implementing various types of REST APIs.
  - An HTTP response includes information like the type of resource, length of resource, and so on
  - Uniform resource locator (URL) is the most popular way to find resources on the web.

- URL is divided into three parts: scheme, internet address or base URL, and route
- The GET method is one of the popular methods of requesting information. Some other methods may also include the body.
- Response method contains the version and body of the response.
- POST submits data to the server, PUT updates data already on the server, DELETE deletes data from the server
- Requests is a Python library that allows you to send HTTP/1.1 requests easily
  - You can modify the results of your query with the GET method.
  - You can obtain multiple requests from a URL like name, ID, and so on with a Query string.
- Web scraping in Python involves extracting and parsing data from websites to gather information for various applications, using libraries like BeautifulSoup and requests.
  - HTML comprises text surrounded by blue text elements enclosed in angular brackets called tags.
  - You can select an HTML element on a web page to inspect the webpage.
  - Web pages may also contain CSS and JavaScript along with HTML elements.
  - Each HTML document is like an HTML Tree, which may contain strings and other tags.
  - Each HTML table is comprised of table tags and is structured with elements such as rows, headers, body and so on.
- Tabular data can also be extracted from web pages using the `read_html` method in Pandas.
- BeautifulSoup in Python is a library for parsing and navigating HTML and XML documents, making extracting, and manipulating data from web pages more accessible.
- To parse a document, pass it through the BeautifulSoup constructor to get a beautiful soup object representing the document as a nested data structure.
- BeautifulSoup represents HTML as a set of tree-like objects with methods to parse the HTML.
- Navigable string is like a Python string that supports beautiful soup functionality.
- `find_all` is a method used to extract content based on the tag's name, its attributes, the text of a string, or some combination of these.
- The `find_all` method looks through a tag's descendants and retrieves all descendants that match your filters.
- The result is a Python iterable like a list.

- File formats refer to the specific structure and encoding rules used to store and represent data in files, such as .txt for plain text or .csv for comma-separated values.
- Python works with different file formats such as CSV, XML, JSON, xlsx, and so on
- The extension of a file name will let you know what type of file it is and what it needs to open with.
- To access data from CSV files, we can use Python libraries such as Pandas.
- Similarly, different methods help parse JSON, XML, and other files.

## Final Exam

## Course Wrap up

## Glossary

| Term                          | Definition                                                                                                                                                                                                      |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AI                            | AI (artificial intelligence) is the ability of a digital computer or computer-controlled robot to perform tasks commonly associated with intelligent beings.                                                    |
| Application development       | Application development, or app development, is the process of planning, designing, creating, testing, and deploying a software application to perform various business operations.                             |
| Arithmetic Operations         | Arithmetic operations are the basic calculations we make in everyday life like addition, subtraction, multiplication and division. It is also called as algebraic operations or mathematical operations.        |
| Array of numbers              | Set of numbers or objects that follow a pattern presented as an arrangement of rows and columns to explain multiplication.                                                                                      |
| Assignment operator in Python | Assignment operator is a type of Binary operator that helps in modifying the variable to its left with the use of its value to the right. The symbol used for assignment operator is "=".                       |
| Asterisk                      | Symbol "*" used to perform various operations in Python.                                                                                                                                                        |
| Backslash                     | A backslash is an escape character used in Python strings to indicate that the character immediately following it should be treated in a special way, such as being treated as escaped character or raw string. |

|                          |                                                                                                                                                                                                                                                                            |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Boolean                  | Denoting a system of algebraic notation used to represent logical propositions by means of the binary digits 0 (false) and 1 (true).                                                                                                                                       |
| Colon                    | A colon is used to represent an indented block. It is also used to fetch data and index ranges or arrays.                                                                                                                                                                  |
| Concatenate              | Link (things) together in a chain or series.                                                                                                                                                                                                                               |
| Data engineering         | Data engineers are responsible for turning raw data into information that an organization can understand and use. Their work involves blending, testing, and optimizing data from numerous sources.                                                                        |
| Data science             | Data Science is an interdisciplinary field that focuses on extracting knowledge from data sets which are typically huge in amount. The field encompasses analysis, preparing data for analysis, and presenting findings to inform high-level decisions in an organization. |
| Data type                | Data type refers to the type of value a variable has and what type of mathematical, relational or logical operations can be applied without causing an error.                                                                                                              |
| Double quote             | Symbol “ ” used to represent strings in Python.                                                                                                                                                                                                                            |
| Escape sequence          | An escape sequence is two or more characters that often begin with an escape character that tell the computer to perform a function or command.                                                                                                                            |
| Expression               | An expression is a combination of operators and operands that is interpreted to produce some other value.                                                                                                                                                                  |
| Float                    | Python float () function is used to return a floating-point number from a number or a string representation of a numeric value.                                                                                                                                            |
| Forward slash            | Symbol “/” used to perform various operation sin Python                                                                                                                                                                                                                    |
| Foundational             | Denoting an underlying basis or principle; fundamental.                                                                                                                                                                                                                    |
| Immutable                | Immutable Objects are of in-built datatypes like int, float, bool, string, Unicode, and tuple. In simple words, an immutable object can't be changed after it is created.                                                                                                  |
| Integer                  | An integer is the number zero (0), a positive natural number (1, 2, 3, and so on) or a negative integer with a minus sign (-1, -2, -3, and so on.)                                                                                                                         |
| Manipulate               | Is the process of modifying a string or creating a new string by making changes to existing strings.                                                                                                                                                                       |
| Mathematical conventions | A mathematical convention is a fact, name, notation, or usage which is generally agreed upon by mathematicians.                                                                                                                                                            |
| Mathematical expressions | Expressions in math are mathematical statements that have a minimum of two terms containing numbers or variables, or both, connected by an operator in between.                                                                                                            |
| Mathematical operations  | The mathematical “operation” refers to calculating a value using operands and a math operator.                                                                                                                                                                             |
| Negative indexing        | Allows you to access elements of a sequence (such as a list, a string, or a tuple) from the end, using negative numbers as indexes.                                                                                                                                        |
| Operands                 | The quantity on which an operation is to be done.                                                                                                                                                                                                                          |

|                     |                                                                                                                                                                                               |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Operators in Python | Operators are used to perform operations on variables and values.                                                                                                                             |
| Parentheses         | Parentheses is used to call an object.                                                                                                                                                        |
| Replicate           | To make an exact copy of.                                                                                                                                                                     |
| Sequence            | A sequence is formally defined as a function whose domain is an interval of integers.                                                                                                         |
| Single quote        | Symbol ' ' used to represent strings in python.                                                                                                                                               |
| Slicing in Python   | Slicing is used to return a portion from defined list.                                                                                                                                        |
| Special characters  | A special character is one that is not considered a number or letter. Symbols, accent marks, and punctuation marks are considered special characters.                                         |
| Stride value        | Stride is the number of bytes from one row of pixels in memory to the next row of pixels in memory.                                                                                           |
| Strings             | In Python, Strings are arrays of bytes representing Unicode characters.                                                                                                                       |
| Substring           | A substring is a sequence of characters that are part of an original string.                                                                                                                  |
| Type casting        | The process of converting one data type to another data type is called Typecasting or Type Coercion or Type Conversion.                                                                       |
| Types in Python     | Data types are the classification or categorization of data items. It represents the kind of value that tells what operations can be performed on a particular data.                          |
| Variables           | Variables are containers for storing data values.                                                                                                                                             |
| Aliasing            | Aliasing refers to giving another name to a function or a variable.                                                                                                                           |
| Ampersand           | A character typically "&" standing for the word "and."                                                                                                                                        |
| Compound elements   | Compound statements contain (groups of) other statements; they affect or control the execution of those other statements in some way.                                                         |
| Delimiter           | A delimiter in Python is a character or sequence of characters used to separate or mark the boundaries between elements or fields within a larger data structure, such as a string or a file. |
| Dictionaries        | A dictionary in Python is a data structure that stores a collection of key-value pairs, where each key is unique and associated with a specific value.                                        |
| Function            | A function is a block of code, defining a set procedure, which is executed only when it is called.                                                                                            |
| Immutable           | Immutable Objects are of in-built datatypes like int, float, bool, string, Unicode, and tuple. In simple words, an immutable object can't be changed after it is created.                     |
| Intersection        | The intersection of two given sets is the largest set, which contains all the elements that are common to both sets.                                                                          |



|                      |                                                                                                                                                                                                                                                            |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Keys                 | The keys () method in Python Dictionary, returns a view object that displays a list of all the keys in the dictionary in order of insertion using Python.                                                                                                  |
| Lists                | A list is any list of data items, separated by commas, inside square brackets.                                                                                                                                                                             |
| Logic operations     | In Python, logic operations refer to the use of logical operators such as "and," "or," and "not" to perform logical operations on Boolean values (True or False).                                                                                          |
| Mutable              | Immutable objects are of in-built datatypes like int, float, bool, string, Unicode, and tuple. A mutable object can be changed after it is created.                                                                                                        |
| Nesting              | A nested function is simply a function within another function and is sometimes called an "inner function".                                                                                                                                                |
| Ratings in python    | Ratings in Python typically refer to a numerical or qualitative measure assigned to something to indicate its quality, performance, or value.                                                                                                              |
| Set operations       | Set operations in Python refer to mathematical operations performed on sets, which are unordered collections of unique elements.                                                                                                                           |
| Sets in python       | A set is an unordered collection of unique elements.                                                                                                                                                                                                       |
| Syntax               | The rules that define the structure of the language for python is called its syntax.                                                                                                                                                                       |
| Tuples               | These are used store multiple items in a single variable.                                                                                                                                                                                                  |
| Type casting         | In python, this is converting one data type to another.                                                                                                                                                                                                    |
| Variables            | In python, a variable is a symbolic name or identifier used to store and manipulate data. Variables serve as containers for values, and these values can be of various data types, including numbers, strings, lists, and more.                            |
| Venn diagram         | A Venn diagram is a graphical representation that uses overlapping circles to illustrate the relationships and commonalities between sets or groups of items.                                                                                              |
| Versatile data       | Versatile data, in a general context, refers to data that can be used in multiple ways, is adaptable to different applications or purposes, and is not restricted to a specific use case.                                                                  |
| Analogy              | Refers to a concept or comparison outside the scope of the programming language itself, used to explain or relate one concept to another in a more understandable way.                                                                                     |
| Attributes           | Attributes in Python refer to the characteristics or properties of an object, and they can be accessed using dot notation.                                                                                                                                 |
| Branching            | Branching in Python is a process of altering the flow of a program based on conditions, typically using if, elif, and else statements.                                                                                                                     |
| Comparison operators | Comparison operators in Python are used to compare values and return Boolean results (True or False), including operators like == (equal), != (not equal), < (less than), > (greater than), <= (less than or equal to), and >= (greater than or equal to). |

|                          |                                                                                                                                                                                       |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Conditions               | Conditions in Python are used to make decisions in code, executing specific blocks of code based on whether a given expression evaluates to True or False.                            |
| Enumerate                | In Python, "enumerate" is a built-in function that adds a counter to an iterable, allowing you to loop through both the elements and their corresponding indices.                     |
| Exception handling       | Exception handling in Python is a mechanism for gracefully managing and responding to errors or exceptional conditions that may occur during program execution.                       |
| Explicitly               | In Python, the term "explicitly" refers to performing an action or specifying something in a clear, unambiguous, and direct manner.                                                   |
| For loops                | For loops in Python are used for iterating over a sequence (such as a list, tuple, or string) or other iterable objects, executing a set of statements for each item in the sequence. |
| Global variable          | Global variables in Python are variables defined outside of any function or block and can be accessed and modified from any part of the code.                                         |
| Incremented              | "Incremented" in Python means to increase the value of a variable by a specified amount, typically done using the += operator or by adding a fixed value.                             |
| Indent                   | In Python, "indent" refers to the use of whitespace at the beginning of a line to signify the structure and scope of code blocks, such as loops and functions.                        |
| Indices                  | In Python, "indices" refer to the position or location of elements in a sequence, like a string, list, or tuple, starting with 0 for the first element.                               |
| Iterate                  | In Python, "iterate" means to repeatedly perform a set of operations or steps on each item in a collection, such as a list, tuple, or dictionary, typically using loops or iterators. |
| Local variables          | Local variables in Python are variables defined within a specific function or block of code and are only accessible within that function or block.                                    |
| Logic operators          | Logic operators in Python are used to perform logical operations on Boolean values, including operators like and (logical AND), or (logical OR), and not (logical NOT).               |
| Loops                    | Loops in Python are constructs for repeating a block of code, enabling the execution of the same code multiple times.                                                                 |
| Parameters               | Parameters in Python are placeholders in a function definition, used to accept and work with values provided to the function when it is called.                                       |
| Programming Fundamentals | Programming fundamentals in Python involve variables, control structures, functions, data structures, input/output, and error handling for building software.                         |

|                       |                                                                                                                                                                                                                                        |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Range function        | The range function in Python generates a sequence of numbers that can be used for iterating in a loop and is typically used as range (start, stop, step), where it creates numbers from start to stop-1 with the given step increment. |
| Scope of function     | The "scope of a function" in Python refers to the region of code where a variable defined within that function is accessible or visible.                                                                                               |
| Sequences             | Sequences in Python are ordered collections of items that can include data types like strings, lists, and tuples, allowing for indexing and iteration.                                                                                 |
| Syntax                | In Python, "explicitly" means to state something clearly and directly, leaving no room for ambiguity or implicit interpretation.                                                                                                       |
| While loops           | While loops in Python are used to repeatedly execute a block of code as long as a specified condition is true.                                                                                                                         |
| .csv file             | A .csv (Comma-Separated Values) file is a plain text file format for storing tabular data, where each line represents a row and uses commas to separate values in different columns.                                                   |
| .txt file             | A .txt (Text) file is a common file format that contains plain text without specific formatting, making it suitable for storing and editing textual data.                                                                              |
| Append                | To "append" means to add or attach something to the end of an existing object, typically used in the context of adding data to a file or elements to a data structure like a list in Python.                                           |
| Attribute             | An "attribute" in Python refers to a property or characteristic associated with an object, which can be accessed using dot notation.                                                                                                   |
| Broadcasting in NumPy | Broadcasting in NumPy allows arrays with different shapes to be combined in element-wise operations by automatically extending smaller arrays to match the shape of larger ones, making operations more flexible.                      |
| Component             | In NumPy, a "component" typically refers to a specific element or value within a multi-dimensional array, which can be accessed using indexing.                                                                                        |
| Computation           | Computation in NumPy involves performing numerical operations on arrays and matrices, making it a powerful library for mathematical and scientific computing in Python.                                                                |
| Data analysis         | Data analysis is the process of inspecting, cleaning, transforming, and interpreting data to discover useful information, draw conclusions, and support decision-making.                                                               |
| DataFrames            | A DataFrames in Pandas is a two-dimensional, tabular data structure for storing and analyzing data, consisting of rows and columns.                                                                                                    |
| Dependencies          | Dependencies in Pandas are external libraries or modules, such as NumPy, that Pandas rely on for fundamental data manipulation and analysis functionality.                                                                             |

|                                 |                                                                                                                                                                                                                                                         |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| File attribute                  | File attributes generally refer to properties or metadata associated with files, which are managed at the operating system level.                                                                                                                       |
| File object                     | A "file object" in Python represents an open file, allowing reading from or writing to the file.                                                                                                                                                        |
| Grid                            | In Python, a "grid" typically refers to a two-dimensional structure composed of rows and columns, often used to represent data in a tabular format or for organizing objects in a coordinate system.                                                    |
| Hadamard Product                | The Hadamard product is a mathematical operation that involves element-wise multiplication of two matrices or arrays of the same shape, producing a new matrix with each element being the product of the corresponding elements in the input matrices. |
| Importing pandas                | To import Pandas in Python, you use the statement: <code>import pandas as pd</code> , which allows you to access Pandas functions and data structures using the abbreviation "pd."                                                                      |
| Index                           | In Python, an "index" typically refers to a position or identifier used to access elements within a sequence or data structure, such as a list or string.                                                                                               |
| Libraries                       | Libraries in Python are collections of pre-written code modules that provide reusable functions and classes to simplify and enhance software development.                                                                                               |
| Linespace                       | In Python, "linespace" refers to a NumPy function that generates an array of evenly spaced values within a specified range.                                                                                                                             |
| NumPy                           | NumPy in Python is a fundamental library for numerical computing that provides support for large, multi-dimensional arrays and matrices, as well as a variety of high-level mathematical functions to operate on these arrays.                          |
| One dimensional NumPy           | A one-dimensional NumPy array is a linear data structure that stores elements in a single sequence, often used for numerical computations and data manipulation.                                                                                        |
| Open function                   | In Python, the "open" function is used to access and manipulate files, allowing you to read from or write to a specified file.                                                                                                                          |
| Pandas                          | Pandas is a popular Python library for data manipulation and analysis, offering data structures and tools for working with structured data like tables and time series.                                                                                 |
| Pandas library                  | Pandas library in Python refer to the various modules and functions within the Pandas library, which provides powerful data structures and data analysis tools for working with structured data.                                                        |
| Plotting Mathematical Functions | Plotting mathematical functions in Python involves using libraries like Matplotlib to create graphical representations of mathematical equations, aiding visualization, and analysis.                                                                   |
| Shape                           | In NumPy, "shape" refers to an array's dimensions (number of rows and columns), describing its size and structure.                                                                                                                                      |

|                        |                                                                                                                                                                                                                |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Slicing                | Slicing in NumPy entails extracting specific portions of an array by specifying a range of indices, enabling you to work with subsets of the data.                                                             |
| Two dimensional NumPy  | A two-dimensional NumPy array is a structured data representation with rows and columns, resembling a matrix or table, ideal for various data manipulation and analysis tasks.                                 |
| Universal Functions    | Universal functions (ufuncs) in NumPy are functions that operate element-wise on arrays, providing efficient and vectorized operations for a wide range of mathematical and logical operations.                |
| Vector addition        | Vector addition in Python involves adding corresponding elements of two or more vectors, producing a new vector with the sum of their components.                                                              |
| Visualizations         | Visualizations in Python refer to the creation of graphical representations, such as charts, plots, and graphs, to illustrate and communicate data and trends visually.                                        |
| API Key                | An API key in Python is a secure access token or code used to authenticate and authorize access to an API or web service, enabling the user to make authenticated requests.                                    |
| APIs                   | APIs (Application Programming Interfaces) are a set of rules and protocols that enable different software applications to communicate and interact, facilitating the exchange of data and functionality.       |
| Audio file             | An audio file is a digital recording or representation of sound, often stored in formats like MP3, WAV, or FLAC, allowing playback and storage of audio content.                                               |
| Authorize              | In Python, "authorize" often means granting permission or access to a user or system to perform specific actions or access particular resources, often related to authentication and authorization mechanisms. |
| Beautiful Soup Objects | Beautiful Soup objects in Python are representations of parsed HTML or XML documents, allowing easy navigation, searching, and manipulation of the document's elements and data.                               |
| Bitcoin currency       | Bitcoin is a decentralized digital currency that operates without a central authority, allowing peer-to-peer transactions on a blockchain network.                                                             |
| Browser                | A browser is a software application that enables users to access and interact with web content, displaying websites and web applications.                                                                      |
| Candlestick plot       | A candlestick plot in Python visually represents stock price movements over time, using rectangles to illustrate the open, close, high, and low prices for a given period.                                     |
| Client/Wrapper         | A client or wrapper in Python is a software component that simplifies interaction with external services or APIs, encapsulating                                                                                |

|                  |                                                                                                                                                                                                           |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  | communication and providing higher-level functionality for developers.                                                                                                                                    |
| CoinGecko API    | The CoinGecko API is a web service that provides cryptocurrency market data and information, allowing developers to access real-time and historical data for various cryptocurrencies.                    |
| DELETE Method    | The DELETE method in Python is an HTTP request method used to request the removal or deletion of a resource on a web server.                                                                              |
| Endpoint         | In Python, an "endpoint" refers to a specific URL or URI that a web service or API exposes to perform a particular function or access a resource.                                                         |
| File extension   | A file extension is a suffix added to a filename to indicate the file's format or type, often used by operating systems and applications to determine how to handle the file.                             |
| find_all         | In Python, find_all is a BeautifulSoup method used to search and extract all occurrences of a specified HTML or XML element, returning a list of matching elements.                                       |
| GET method       | The GET method in Python is an HTTP request method used to retrieve data from a web server by appending parameters to the URL.                                                                            |
| HTML             | HTML (Hypertext Markup Language) is the standard language for creating and structuring content on web pages, using tags to define the structure and presentation of documents.                            |
| HTML Anchor tags | HTML anchor tags in Python are used to create hyperlinks within web pages, linking to other web pages or resources using the <a> element with the href attribute.                                         |
| HTML Tables      | HTML tables in Python are used to organize and display data in a structured grid format on a web page, constructed with <table>, <tr>, <th>, and <td> elements.                                           |
| HTML Tag         | An HTML tag in Python is a specific code enclosed in angle brackets used to define elements within an HTML document, specifying how content should be presented or structured.                            |
| HTML Trees       | HTML trees in Python refer to the hierarchical structure created when parsing an HTML document, representing its elements and their relationships, typically used for manipulation or extraction of data. |
| HTTP             | HTTP (HyperText Transfer Protocol) is the foundation of data communication on the World Wide Web, used for transmitting and retrieving web content between clients and servers.                           |
| httplib          | A library that provides a set of functions and classes to send and handle HTTP and HTTPS requests.                                                                                                        |
| Identify         | In Python, "identify" usually means determining if two variables or objects refer to the same memory location, which can be checked using the is operator.                                                |

|                  |                                                                                                                                                                                                              |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Instance         | In Python, an "instance" typically refers to a specific occurrence of an object or class, created from a class blueprint, with its own unique set of data and attributes.                                    |
| JSON file        | A JSON (JavaScript Object Notation) file is a lightweight data interchange format that stores structured data in a human-readable text format, commonly used for configuration, data exchange, and web APIs. |
| Mean value       | The mean value in Python is the average of a set of numerical values, calculated by adding all values and dividing by the total number of values.                                                            |
| Navigable string | In Python, a Navigable String is a BeautifulSoup object representing a string within an HTML or XML document, allowing for navigation and manipulation of the text content.                                  |
| Plotly           | Plotly is a Python library for creating interactive and visually appealing web-based data visualizations and dashboards.                                                                                     |
| PNG file         | A PNG (Portable Network Graphics) file is a lossless image format in Python that is commonly used for high-quality graphics with support for transparency and compression.                                   |
| POST method      | The POST method in Python is an HTTP request method used to send data to a web server, often used for submitting form data and creating or updating resources.                                               |
| Post request     | A POST request in Python is an HTTP method used to send data to a web server for the purpose of creating or updating a resource, typically used in web applications and APIs.                                |
| PUT method       | The PUT method in Python is an HTTP request method used to update an existing resource on a web server by replacing or modifying it.                                                                         |
| Py-Coin-Gecko    | Py-Coin-Gecko is a Python library that provides a convenient interface for accessing cryptocurrency data and information from the CoinGecko API.                                                             |
| Python iterable  | A Python iterable is an object that can be looped over, typically used in for loops, and includes data structures like lists, tuples, and dictionaries.                                                      |
| Query string     | A query string in Python is a part of a URL that contains data or parameters to be sent to a web server, typically used in HTTP GET requests to retrieve specific information.                               |
| rb mode          | In Python, "rb" mode is used when opening a file to read it in binary mode, allowing you to read and manipulate non-text files like images or binary data.                                                   |
| Resource         | In Python, a "resource" typically refers to an external entity such as a file, database connection, or network object that can be managed and manipulated within a program.                                  |
| Rest API         | A REST API in Python is a web-based interface that follows the principles of Representational State Transfer (REST), allowing                                                                                |

|                                |                                                                                                                                                                                                             |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                | communication and data exchange over HTTP using standard HTTP methods and data formats.                                                                                                                     |
| Service instance               | In Python, a "service instance" typically refers to an instantiated object or entity representing a service, enabling interaction with that service in a program or application.                            |
| Timestamp                      | A timestamp is a representation of a specific moment in time, often expressed as a combination of date and time, used for record-keeping and data tracking.                                                 |
| Transcribe                     | "Transcribe" typically means converting spoken language or audio into written text, often using automatic speech recognition (ASR) technology.                                                              |
| Unix timestamp                 | A UNIX timestamp is a numerical value representing the number of seconds that have elapsed since January 1, 1970, 00:00:00 UTC, used for time-keeping in Unix-based systems and programming.                |
| url (Uniform Resource Locator) | In Python, a URL (Uniform Resource Locator) is a web address that specifies the location of a resource on the internet, typically consisting of a protocol, domain, and path.                               |
| urllib                         | The "urllib" library in Python is used for working with URLs and making HTTP requests, including functions for fetching web content, handling cookies, and more.                                            |
| Web service                    | Web services in Python are software components that allow applications to communicate over the internet by sending and receiving data in a standardized format, typically using protocols like HTTP or XML. |
| Web scraping                   | Web scraping in Python is the process of extracting data from websites by parsing and analyzing their HTML structure, often done with libraries like BeautifulSoup or Scrapy.                               |
| xlsx                           | An XLSX file is a file format used for storing spreadsheet data in Excel, containing worksheets, cells, and formulas in a structured manner.                                                                |
| xml                            | XML (Extensible Markup Language) is a text-based format for storing and structuring data using tags, often used for data interchange and configuration files.                                               |